

Swelio Library

Belgian Electronic ID card access library

Copyright (c) 2013 - 2026 Serhiy Perevoznyk. All rights reserved.

Made with **Doc-O-Matic**.

Table of Contents

Symbol Reference 1

Functions **1**

ActivateCard Function 10
ActivateCardEx Function 11
AddFileToContainer Function 11
AddRemoveMessageFilter Function 12
AllocateBuffer Function 12
AllocateDefaultHWNDA Function 13
AllocateDefaultHWNDW Function 13
AllocateHWNDA Function 13
AllocateHWNDW Function 14
AllocateLayeredWindowA Function 14
AllocateLayeredWindowW Function 14
AllocateWindowClassA Function 15
AllocateWindowClassW Function 15
AlphaBlendBitmap Function 15
AlphaBlendNative Function 16
BringWindowToFront Function 16
CardDecryptFileA Function 17
CardDecryptFileW Function 17
CardEncryptFileA Function 17
CardEncryptFileW Function 18
CardSignCMS Function 18
CardSignCMSEx Function 19
CardSignCadesT Function 19
CardSignCadesTEx Function 20
CertSignCMS Function 20
CertSignCMSData Function 21
CertSignCMSEx Function 21
CertSignCadesT Function 22
CertSignCadesTData Function 22
CertSignCadesTEx Function 23
CheckMD5 Function 23
CheckSHA1 Function 24
CheckSHA256 Function 24
ClearFileAttributesA Function 25
ClearFileAttributesW Function 25
ClearUnusedMemory Function 25

CloneFont Function 26
ContainerCertificate Function 26
ContainerEidCertificate Function 26
ContainerPickCertificate Function 27
CopyNativeBitmap Function 27
CreateCardBuffer Function 28
CreateNativeBitmap Function 28
CreateUnicodeFileA Function 28
CreateUnicodeFileW Function 29
CreateWindowsFont Function 29
CurrentIPAddressA Function 29
CurrentIPAddressW Function 30
DeactivateCard Function 30
DeactivateCardEx Function 30
DeallocateBuffer Function 31
DeallocateHWNDA Function 31
DeallocateHWNDW Function 31
DecryptFileAESA Function 32
DecryptFileAESW Function 32
DeleteCardBuffer Function 33
DeleteToRecycleBinA Function 33
DeleteToRecycleBinW Function 33
DestroyFont Function 34
DestroyImageBuffer Function 34
DirectoryExistsA Function 34
DirectoryExistsW Function 35
DisplayCertificate Function 35
DpiY Function 36
DrawAlphaText Function 36
DrawAlphaTextRect Function 36
DrawLayeredWindow Function 37
DrawNativeBitmap Function 37
DrawTextDirect Function 38
DrawTextDirectEx Function 38
DrawTextGlow Function 38
DrawTextLine Function 39
DrawTextOutline Function 39
DrawTextRect Function 40
EmToPixels Function 40
EmptyRecycleBin Function 41
EncodeCertificate Function 41
EncodePhoto Function 41

EncryptFileAESA Function 42
EncryptFileAESW Function 42
FileCloseA Function 43
FileCloseW Function 43
FileCopyA Function 43
FileCopyW Function 44
FileCreateRewriteA Function 44
FileCreateRewriteW Function 44
FileDeleteA Function 45
FileDeleteW Function 45
FileExistsA Function 46
FileExistsW Function 46
FileExtensionIsA Function 46
FileExtensionIsW Function 47
FileGetSizeA Function 47
FileGetSizeW Function 47
FileIsExeA Function 48
FileIsExeW Function 48
FileIsIconA Function 49
FileIsIconW Function 49
FileIsImageA Function 49
FileIsImageW Function 50
FileIsLink Function 50
FileOrFolderExistsA Function 50
FileOrFolderExistsW Function 51
FileRenameA Function 51
FileRenameW Function 51
FileWriteA Function 52
FileWriteCharA Function 52
FileWriteCharW Function 52
FileWriteNewLineA Function 53
FileWriteNewLineW Function 53
FileWriteW Function 53
FreeContainer Function 54
FullPathA Function 54
FullPathW Function 54
GenerateAuthenticationSignatureA Function 55
GenerateAuthenticationSignatureExA Function 55
GenerateAuthenticationSignatureExW Function 56
GenerateAuthenticationSignatureW Function 56
GenerateBMPA Function 57
GenerateBMPW Function 57

GenerateNonRepudiationSignatureA Function 58
GenerateNonRepudiationSignatureExA Function 58
GenerateNonRepudiationSignatureExW Function 59
GenerateNonRepudiationSignatureW Function 59
GeneratePNGA Function 60
GeneratePNGW Function 60
GenerateQRCodeA Function 60
GenerateQRCodeExA Function 61
GenerateQRCodeExW Function 61
GenerateQRCodeW Function 62
GetAllFiles Function 62
GetCardBufferA Function 62
GetCardBufferSize Function 63
GetCardBufferW Function 63
GetCardSerialNumber Function 64
GetCardVersion Function 64
GetContainerError Function 65
GetContainerErrorsCount Function 65
GetEncodedCertificateSize Function 65
GetEncodedPhotoSize Function 66
GetFileMD5A Function 66
GetFileMD5W Function 67
GetFileSHA1A Function 67
GetFileSHA1W Function 68
GetFileSHA256A Function 68
GetFileSHA256W Function 68
GetFileSHA384A Function 69
GetFileSHA384W Function 69
GetFileSHA512A Function 70
GetFileSHA512W Function 70
GetFilesCountA Function 71
GetFilesCountW Function 71
GetHBitmapA Function 71
GetHBitmapW Function 72
GetISOCodeA Function 72
GetISOCodeW Function 73
GetMD5 Function 73
GetPNGA Function 74
GetPNGW Function 74
GetReaderIndexA Function 75
GetReaderIndexW Function 75
GetReaderNameA Function 75

GetReaderNameLenA Function 76
GetReaderNameLenW Function 76
GetReaderNameW Function 76
GetReadersCount Function 77
GetSHA1 Function 77
GetSHA256 Function 78
GetSHA384 Function 78
GetSHA512 Function 79
GetSelectedReaderIndex Function 79
GetStartupA Function 79
GetStartupW Function 80
GetSupportSIS Function 80
GetTextLineSize Function 80
GetTextSize Function 81
GetTextSizeEx Function 81
GetTimeServer Function 81
HibernateWindows Function 82
IgnoreHardwareEvents Function 82
IgnoreServiceEvents Function 82
IgnoreUnknownCards Function 83
InitializeContainer Function 83
IsAnimatedGIFA Function 84
IsAnimatedGIFW Function 84
IsCardActivated Function 84
IsCardActivatedEx Function 85
IsCardPresent Function 85
IsCardPresentEx Function 85
IsCardStillInserted Function 86
IsCardStillInsertedEx Function 86
IsCitrixSession Function 86
IsConnectedToInternet Function 87
IsDirectoryA Function 87
IsDirectoryW Function 87
IsEIDCard Function 88
IsEIDCardEx Function 88
IsEngineActive Function 88
IsFemaleA Function 89
IsFemaleW Function 89
IsMaleA Function 89
IsMaleW Function 90
IsMediaCenter Function 90
IsMetroActive Function 91

IsMultiTouchReady Function 91
IsNativeWin64 Function 91
IsRemoteSession Function 92
IsSISCard Function 92
IsSISCardEx Function 92
IsTabletPC Function 93
IsUnicodeFileA Function 93
IsUnicodeFileW Function 93
IsValidFileNameA Function 94
IsValidFileNameW Function 94
IsValidPathNameA Function 95
IsValidPathNameW Function 95
IsWindows10 Function 95
IsWindows11 Function 96
IsWindows7 Function 96
IsWindows8 Function 96
IsWindowsVista Function 97
IsWindowsXP Function 97
IsWindowsXPSP2 Function 97
IsWow64 Function 98
LayeredWndProcA Function 98
LayeredWndProcW Function 98
LoadAddressA Function 98
LoadAddressW Function 99
LoadBitmapJPG Function 99
LoadBitmapPNG Function 100
LoadCertificateA Function 100
LoadCertificateW Function 100
LoadIdentityA Function 101
LoadIdentityW Function 101
LoadPNGResource Function 101
LoadPhotoA Function 102
LoadPhotoW Function 102
MakeCompatibleBitmap Function 103
MakeSoundFromFileA Function 103
MakeSoundFromFileW Function 103
MakeSoundFromResourceA Function 104
MakeSoundFromResourceW Function 104
NonRepudiationSignatureAlgoA Function 104
NonRepudiationSignatureAlgoW Function 105
PointsToPixels Function 105
PortAvailable Function 105

ReadAddressA Function 106
ReadAddressExA Function 106
ReadAddressExW Function 106
ReadAddressW Function 107
ReadAuthenticationCertificate Function 107
ReadAuthenticationCertificateEx Function 107
ReadBufferFromFileA Function 108
ReadBufferFromFileW Function 108
ReadCaCertificate Function 109
ReadCaCertificateEx Function 109
ReadIdentityA Function 109
ReadIdentityExA Function 110
ReadIdentityExW Function 110
ReadIdentityW Function 110
ReadNonRepudiationCertificate Function 111
ReadNonRepudiationCertificateEx Function 111
ReadPhoto Function 112
ReadPhotoAsBitmap Function 112
ReadPhotoAsBitmapEx Function 112
ReadPhotoEx Function 113
ReadRootCaCertificate Function 113
ReadRootCaCertificateEx Function 114
ReadRrnCertificate Function 114
ReadRrnCertificateEx Function 114
ReadSISCardA Function 115
ReadSISCardExA Function 115
ReadSISCardExW Function 116
ReadSISCardW Function 116
RecycleBinEmpty Function 117
ReloadReadersList Function 117
RemoveCallback Function 117
RemoveStartupA Function 117
RemoveStartupW Function 118
RestoreWindowSubclassA Function 118
RestoreWindowSubclassW Function 118
SaveAddressA Function 119
SaveAddressW Function 119
SaveAuthenticationCertificateA Function 119
SaveAuthenticationCertificateExW Function 120
SaveAuthenticationCertificateW Function 120
SaveCaCertificateA Function 120
SaveCaCertificateExW Function 121

SaveCaCertificateW Function 121
SaveCardToToXMLStreamExA Function 122
SaveCardToToXMLStreamExW Function 122
SaveCardToXmlA Function 122
SaveCardToXmlExA Function 123
SaveCardToXmlExW Function 123
SaveCardToXmlW Function 124
SaveContainer Function 124
SaveIdentityA Function 124
SaveIdentityW Function 125
SaveNonRepudiationCertificateA Function 125
SaveNonRepudiationCertificateExW Function 126
SaveNonRepudiationCertificateW Function 126
SavePersonCsvToStreamA Function 126
SavePersonCsvToStreamW Function 127
SavePersonToCsvA Function 127
SavePersonToCsvExA Function 128
SavePersonToCsvExW Function 128
SavePersonToCsvW Function 128
SavePhotoA Function 129
SavePhotoAsBitmapA Function 129
SavePhotoAsBitmapExA Function 130
SavePhotoAsBitmapExW Function 130
SavePhotoAsBitmapW Function 130
SavePhotoAsJpegA Function 131
SavePhotoAsJpegExA Function 131
SavePhotoAsJpegExW Function 131
SavePhotoAsJpegW Function 132
SavePhotoW Function 132
SaveRootCaCertificateA Function 133
SaveRootCaCertificateExW Function 133
SaveRootCaCertificateW Function 133
SaveRrnCertificateA Function 134
SaveRrnCertificateExW Function 134
SaveRrnCertificateW Function 134
SelectReader Function 135
SelectReaderByNameA Function 135
SelectReaderByNameW Function 136
SendAPDU Function 136
ServerAccessible Function 136
SetCallback Function 137
SetContainerCanonization Function 137

SetContainerTimeServer Function 137
SetMWCompatibility Function 138
SetStartupA Function 138
SetStartupW Function 138
SetSupportSIS Function 139
SetTimeServer Function 139
ShellCopyFileA Function 140
ShellCopyFileW Function 140
ShutdownWindows Function 140
SignPdfFile Function 141
SignPdfFileEx Function 141
StartEngine Function 142
StopEngine Function 142
StretchNativeBitmap Function 142
StripFileNameA Function 143
StripFileNameW Function 143
SuspendWindows Function 144
TurnMonitorOff Function 144
TurnMonitorOn Function 144
UpdateWindowPosition Function 144
VerifyContainer Function 145
VerifyPinA Function 145
VerifyPinExA Function 145
VerifyPinExW Function 146
VerifyPinW Function 146
VerifySignature Function 146
WriteBufferToFileA Function 147
WriteBufferToFileW Function 147
fpreset Function 148

Structs, Records, Enums **148**

structErrorInformation Structure 149
tagCardEventType Enumeration 150
tagEidAddressA Structure 150
tagEidAddressW Structure 150
tagEidCertificate Structure 151
tagEidIdentityA Structure 151
tagEidIdentityExA Structure 153
tagEidIdentityExW Structure 154
tagEidIdentityW Structure 156
tagEidPicture Structure 157
tagSISRecordA Structure 157
tagSISRecordW Structure 158

CardEventType Enumeration 159
EidAddressA Structure 159
EidAddressW Structure 160
EidCertificate Structure 160
EidIdentityA Structure 160
EidIdentityExA Structure 162
EidIdentityExW Structure 163
EidIdentityW Structure 165
EidPicture Structure 166
ErrorInformation Structure 167
PEidAddressA Structure 167
PEidAddressW Structure 167
PEidCertificate Structure 168
PEidIdentityA Structure 168
PEidIdentityExA Structure 170
PEidIdentityExW Structure 171
PEidIdentityW Structure 173
PErrorInformation Structure 174
PSISRecordA Structure 174
PSISRecordW Structure 175
PeidPicture Structure 176
SISRecordA Structure 176
SISRecordW Structure 177

Files **178**

Algorithms.h 178
CardEvents.h 178
CardStructures.h 178
Encryption.h 179
FileOperations.h 180
Graphics.h 182
NationalityConverter.h 183
Network.h 183
Swelio.h 183
System.h 188
SystemInfo.h 189
quricol.h 190

Index **a**

1 Symbol Reference

1.1 Functions

The following table lists functions in this documentation.

Functions

	Name	Description
	ActivateCard (see page 10)	Established communication between the card and the reader
	ActivateCardEx (see page 11)	Established communication between the card and the reader
	AddFileToContainer (see page 11)	Add existing file to the container
	AddRemoveMessageFilter (see page 12)	Adds or removes a message from the User Interface Privilege Isolation (UIPI) message filter.
	AllocateBuffer (see page 12)	Allocates the buffer in memory
	AllocateDefaultHWNDA (see page 13)	This function creates the invisible tool window
	AllocateDefaultHWNDW (see page 13)	This function creates the invisible tool window
	AllocateHWNDA (see page 13)	This function creates the invisible tool window using the provided window procedure
	AllocateHWNDW (see page 14)	This function creates the invisible tool window using the provided window procedure
	AllocateLayeredWindowA (see page 14)	This function creates the layered window using the provided window class name
	AllocateLayeredWindowW (see page 14)	This function creates the layered window using the provided window class name
	AllocateWindowClassA (see page 15)	This function creates the standard window using the provided window class name
	AllocateWindowClassW (see page 15)	This function creates the standard window using the provided window class name
	AlphaBlendBitmap (see page 15)	Performs alpha blending of a bitmap onto a device context
	AlphaBlendNative (see page 16)	Performs alpha blending between two device contexts
	BringWindowToFront (see page 16)	This function brings the specified window to the top of the z-order.
	CardDecryptFileA (see page 17)	Decrypt file using Belgian Id card
	CardDecryptFileW (see page 17)	Decrypt file using Belgian Id card
	CardEncryptFileA (see page 17)	Encrypt file using Belgian Id card
	CardEncryptFileW (see page 18)	Encrypt file using Belgian Id card
	CardSignCMS (see page 18)	Sign data with eID card according to CMS standard
	CardSignCMSEx (see page 19)	Sign data with eID card according to CMS standard
	CardSignCadesT (see page 19)	Sign data with eID card according to CADES-T standard
	CardSignCadesTEx (see page 20)	Sign data with eID card according to CADES-T standard
	CertSignCMS (see page 20)	Sign data with the certificate file according to CMS standard
	CertSignCMSData (see page 21)	Sign data with the certificate according to CMS standard
	CertSignCMSEx (see page 21)	Sign data with the certificate file according to CMS standard

	CertSignCadesT (see page 22)	Sign data with the certificate file according to CADES-T standard
	CertSignCadesTData (see page 22)	Sign data with the certificate according to CADES-T standard
	CertSignCadesTEx (see page 23)	Sign data with the certificate file according to CADES-T standard
	CheckMD5 (see page 23)	Checks the MD5 hash value of the memory buffer
	CheckSHA1 (see page 24)	Checks the SHA1 hash value of the memory buffer
	CheckSHA256 (see page 24)	Checks the SHA256 hash value of the memory buffer
	ClearFileAttributesA (see page 25)	This function sets the file attributes to normal.
	ClearFileAttributesW (see page 25)	This function sets the file attributes to normal.
	ClearUnusedMemory (see page 25)	Clears unused memory and minimized the application memory usage
	CloneFont (see page 26)	Clones an existing font
	ContainerCertificate (see page 26)	Assign certificate for signing ASIC container
	ContainerEidCertificate (see page 26)	Select EID card certificate to sign ASIC container
	ContainerPickCertificate (see page 27)	Pick certificate to sign ASIC container
	CopyNativeBitmap (see page 27)	Copies a native bitmap to a device context
	CreateCardBuffer (see page 28)	Creates XML buffer
	CreateNativeBitmap (see page 28)	Creates a new native bitmap
	CreateUnicodeFileA (see page 28)	Creates UNICODE file
	CreateUnicodeFileW (see page 29)	Creates UNICODE file
	CreateWindowsFont (see page 29)	Creates a Windows font with specified properties
	CurrentIPAddressA (see page 29)	Returns the IP address
	CurrentIPAddressW (see page 30)	Returns the IP address
	DeactivateCard (see page 30)	Terminates a cennection between a smart card and a reader
	DeactivateCardEx (see page 30)	Terminates a connection between a smart card and a reader
	DeallocateBuffer (see page 31)	Deallocates the memory buffer
	DeallocateHWNDA (see page 31)	This function destroys the specified window.
	DeallocateHWNDW (see page 31)	This function destroys the specified window.
	DecryptFileAESA (see page 32)	Decrypts file using AES algorithm.
	DecryptFileAESW (see page 32)	Decrypts file using AES algorithm.
	DeleteCardBuffer (see page 33)	Deletes XML buffer
	DeleteToRecycleBinA (see page 33)	Deletes file to the Windows Recycle Bin
	DeleteToRecycleBinW (see page 33)	Deletes file to WIndows Recycle Bin
	DestroyFont (see page 34)	Destroys a font and releases its resources
	DestroyImageBuffer (see page 34)	Destroys the memory buffer
	DirectoryExistsA (see page 34)	Determines whether a specified directory exists.
	DirectoryExistsW (see page 35)	Determines whether a specified directory exists.
	DisplayCertificate (see page 35)	Displays the dialog window with certificate information

	DpiY (see page 36)	Gets the horizontal DPI of the display Returns: The horizontal DPI value
	DrawAlphaText (see page 36)	Draws text with alpha blending onto a device context
	DrawAlphaTextRect (see page 36)	Draws text with alpha blending within a specified rectangle on a device context
	DrawLayeredWindow (see page 37)	Repaints the surface of the layered window
	DrawNativeBitmap (see page 37)	Draws a native bitmap onto another bitmap
	DrawTextDirect (see page 38)	Draws text directly onto a device context
	DrawTextDirectEx (see page 38)	Draws text directly onto a device context with specified background color
	DrawTextGlow (see page 38)	Draws text with a glowing effect onto a device context
	DrawTextLine (see page 39)	Draws a single line of text onto a device context
	DrawTextOutline (see page 39)	Draws text with an outline effect onto a device context
	DrawTextRect (see page 40)	Draws text within a specified rectangle on a device context
	EmToPixels (see page 40)	Converts em units to pixels
	EmptyRecycleBin (see page 41)	Empties the recycle bin
	EncodeCertificate (see page 41)	Performs Base64 encoding of the certificate
	EncodePhoto (see page 41)	Performs Base64 encoding of the photo
	EncryptFileAESA (see page 42)	Encrypts file using AES algorithm.
	EncryptFileAESW (see page 42)	Encrypts file using AES algorithm.
	FileCloseA (see page 43)	Concludes input/output (I/O) to a file opened using the FileCreateRewrite function.
	FileCloseW (see page 43)	Concludes input/output (I/O) to a file opened using the FileCreateRewrite function.
	FileCopyA (see page 43)	The CopyFile function copies an existing file to a new file.
	FileCopyW (see page 44)	The CopyFile function copies an existing file to a new file.
	FileCreateRewriteA (see page 44)	Creates new or overwrites existing file
	FileCreateRewriteW (see page 44)	Creates new or overwrites existing file
	FileDeleteA (see page 45)	Deletes a file from disk.
	FileDeleteW (see page 45)	Deletes a file from disk.
	FileExistsA (see page 46)	Tests whether a specified file exists.
	FileExistsW (see page 46)	Tests whether a specified file exists.
	FileExtensionIsA (see page 46)	Checks the file extension
	FileExtensionIsW (see page 47)	Checks the file extension
	FileGetSizeA (see page 47)	Retrieves the size of a specified file.
	FileGetSizeW (see page 47)	Retrieves the size of a specified file.
	FileIsExeA (see page 48)	Checks if the file is a Windows executable
	FileIsExeW (see page 48)	Checks if the file is a Windows executable
	FileIsIconA (see page 49)	Checks if the file is a Windows icon (.ico) file
	FileIsIconW (see page 49)	Checks if the file is a Windows icon (.ico) file
	FileIsImageA (see page 49)	Checks if the file is an image file
	FileIsImageW (see page 50)	Checks if the file is an image file
	FileIsLink (see page 50)	Checks to see if the file specified by file name is a Microsoft Windows shortcut (.Lnk) file (and is neither a file nor a folder).
	FileOrFolderExistsA (see page 50)	Checks if the file or folder with the given name exists
	FileOrFolderExistsW (see page 51)	Checks if the file or folder with the given name exists
	FileRenameA (see page 51)	Renames the file

	FileRenameW (see page 51)	Renames the file
	FileWriteA (see page 52)	Writes string to the file
	FileWriteCharA (see page 52)	Writes one character to the file
	FileWriteCharW (see page 52)	Writes one character to the file
	FileWriteNewLineA (see page 53)	Writes new line sequence to the file
	FileWriteNewLineW (see page 53)	Writes new line sequence to the file
	FileWriteW (see page 53)	Writes string to the file
	FreeContainer (see page 54)	Deallocates ASIC container
	FullPathA (see page 54)	Gets the full path to the file based on file name
	FullPathW (see page 54)	Gets the full path to the file based on file name
	GenerateAuthenticationSignatureA (see page 55)	Generate authentication signature
	GenerateAuthenticationSignatureExA (see page 55)	Generate authentication signature
	GenerateAuthenticationSignatureExW (see page 56)	Generate authentication signature
	GenerateAuthenticationSignatureW (see page 56)	Generate authentication signature
	GenerateBMPA (see page 57)	Generates Windows Bitmap file with QR Code image
	GenerateBMPW (see page 57)	Generates Windows Bitmap file with QR Code image
	GenerateNonRepudiationSignatureA (see page 58)	Generate non repudiation signature
	GenerateNonRepudiationSignatureExA (see page 58)	Generate non repudiation signature
	GenerateNonRepudiationSignatureExW (see page 59)	Generate non repudiation signature
	GenerateNonRepudiationSignatureW (see page 59)	Generate non repudiation signature
	GeneratePNGA (see page 60)	Generates PNG file with QR Code image
	GeneratePNGW (see page 60)	Generates PNG file with QR Code image
	GenerateQRCodeA (see page 60)	Read eID card and save the identity information and address to PNG QR Code file
	GenerateQRCodeExA (see page 61)	Read eID card and save the identity information and address to PNG QR Code file
	GenerateQRCodeExW (see page 61)	Read eID card and save the identity information and address to PNG QR Code file
	GenerateQRCodeW (see page 62)	Read eID card and save the identity information and address to PNG QR Code file
	GetAllFiles (see page 62)	Returns the names of files in a specified directory.
	GetCardBufferA (see page 62)	Gets XML or CSV information from the memory buffer
	GetCardBufferSize (see page 63)	This function returns the size of the buffer needed to hold the information from the eID card in the XML or CSV format
	GetCardBufferW (see page 63)	Gets XML or CSV information from the memory buffer
	GetCardSerialNumber (see page 64)	Get the serial number of EID card
	GetCardVersion (see page 64)	Get the applet version number for card in the reader with specified number
	GetContainerError (see page 65)	Gets the value of the container validation error message based on the message index
	GetContainerErrorsCount (see page 65)	Get the number of error messages availavle in the container validation report
	GetEncodedCertificateSize (see page 65)	Returns the size of the Base64 encoded certificate

	GetEncodedPhotoSize (see page 66)	Calculates buffer size for Base64 encoded photo
	GetFileMD5A (see page 66)	Gets the MD5 hash value for the file
	GetFileMD5W (see page 67)	Gets the MD5 hash value for the file
	GetFileSHA1A (see page 67)	Gets the SHA1 hash value for the file
	GetFileSHA1W (see page 68)	Gets the SHA1 hash value for the file
	GetFileSHA256A (see page 68)	Gets the SHA256 hash value for the file
	GetFileSHA256W (see page 68)	Gets the SHA256 hash value for the file
	GetFileSHA384A (see page 69)	Gets the SHA384 hash value for the file
	GetFileSHA384W (see page 69)	Gets the SHA384 hash value for the file
	GetFileSHA512A (see page 70)	Gets the SHA512 hash value for the file
	GetFileSHA512W (see page 70)	Gets the SHA512 hash value for the file
	GetFilesCountA (see page 71)	Calculates the number of files in the given folder
	GetFilesCountW (see page 71)	Calculates the number of files in the given folder
	GetHBitmapA (see page 71)	Generates Windows Bitmap in memory with QR Code image
	GetHBitmapW (see page 72)	Generates Windows Bitmap in memory with QR Code image
	GetISOCodeA (see page 72)	Returns the country ISO code based on the nationality string
	GetISOCodeW (see page 73)	Returns the country ISO code based on the nationality string
	GetMD5 (see page 73)	Gets the MD5 hash value for the content of the memory buffer
	GetPNGA (see page 74)	Writes PNG image to the memory buffer.
	GetPNGW (see page 74)	Writes PNG image to the memory buffer.
	GetReaderIndexA (see page 75)	Returns the zero-based reader index with specified name
	GetReaderIndexW (see page 75)	Returns the zero-based reader index with specified name
	GetReaderNameA (see page 75)	Returns the name of the card reader
	GetReaderNameLenA (see page 76)	Returns the length of the reader name
	GetReaderNameLenW (see page 76)	Returns the length of the reader name
	GetReaderNameW (see page 76)	Returns the name of the card reader
	GetReadersCount (see page 77)	Get number of card readers connected to PC
	GetSHA1 (see page 77)	Gets the SHA1 hash value for the content of the memory buffer
	GetSHA256 (see page 78)	Gets the SHA256 hash value for the content of the memory buffer
	GetSHA384 (see page 78)	Gets the SHA384 hash value for the content of the memory buffer
	GetSHA512 (see page 79)	Gets the SHA512 hash value for the content of the memory buffer
	GetSelectedReaderIndex (see page 79)	Returns the index of the active smart card reader
	GetStartupA (see page 79)	Checks if the application is registered to run when Windows starts
	GetStartupW (see page 80)	Checks if the application is registered to run when Windows starts
	GetSupportSIS (see page 80)	Checks if the SIS cards are supported by the engine
	GetTextLineSize (see page 80)	Calculates the size of a single line of text in pixels
	GetTextSize (see page 81)	Calculates the size of the text in pixels
	GetTextSizeEx (see page 81)	Gets the size of text when rendered with a specific font
	GetTimeServer (see page 81)	Get the time server URI for digital signature of the ASIC container
	HibernateWindows (see page 82)	Hibernates Windows
	IgnoreHardwareEvents (see page 82)	Ignore USB reader insert / remove events
	IgnoreServiceEvents (see page 82)	Ignore smartcard service stop events when reporting readers list change
	IgnoreUnknownCards (see page 83)	Ignore Unknown cards insert / remove events
	InitializeContainer (see page 83)	Initializes ASIC container
	IsAnimatedGIFA (see page 84)	Checks if the file is an animated GIF image file
	IsAnimatedGIFW (see page 84)	Checks if the file is an animated GIF image file
	IsCardActivated (see page 84)	Checks the connection between a smart card and a reader

	IsCardActivatedEx (see page 85)	Checks the connection between a smart card and a reader
	IsCardPresent (see page 85)	Checks if the card is present in the card reader
	IsCardPresentEx (see page 85)	Checks if the card is present in the card reader
	IsCardStillInserted (see page 86)	Checks if the card is still inserted in the card reader
	IsCardStillInsertedEx (see page 86)	Checks if the card is still inserted in the card reader
	IsCitrixSession (see page 86)	Checks if application is running in Citrix session
	IsConnectedToInternet (see page 87)	Checks if PC is connected to Internet
	IsDirectoryA (see page 87)	Verifies that a path is a valid directory.
	IsDirectoryW (see page 87)	Verifies that a path is a valid directory.
	IsEIDCard (see page 88)	Check if Belgian EID card is inserted into card reader
	IsEIDCardEx (see page 88)	Check if Belgian EID card is inserted into card reader
	IsEngineActive (see page 88)	Checks if the Swelio Engine is activated
	IsFemaleA (see page 89)	Checks if the card owner is female
	IsFemaleW (see page 89)	Checks if the card owner is female
	IsMaleA (see page 89)	Checks if the card owner is male
	IsMaleW (see page 90)	Checks if the card owner is male
	IsMediaCenter (see page 90)	Checks if the Media Center version of Windows is installed
	IsMetroActive (see page 91)	Checks if metro interface is active
	IsMultiTouchReady (see page 91)	Checks if the system is multi touch ready
	IsNativeWin64 (see page 91)	Checks if the application is native 64 bit executable
	IsRemoteSession (see page 92)	Checks if application is running in RDP session
	IsSISCard (see page 92)	Check if Belgian SIS card is inserted into card reader
	IsSISCardEx (see page 92)	Check if Belgian SIS card is inserted into card reader
	IsTabletPC (see page 93)	Checks if the application is running on the Tablet PC
	IsUnicodeFileA (see page 93)	Checks if the file is UNICODE file
	IsUnicodeFileW (see page 93)	Checks if the file is UNICODE file
	IsValidFileNameA (see page 94)	Checks if provided string is a valid file name
	IsValidFileNameW (see page 94)	Checks if provided string is a valid file name
	IsValidPathNameA (see page 95)	Checks if provided string is a valid file path
	IsValidPathNameW (see page 95)	Checks if provided string is a valid file path
	IsWindows10 (see page 95)	Checks if PC is running Windows 10 or better
	IsWindows11 (see page 96)	Checks if PC is running Windows 11 or better
	IsWindows7 (see page 96)	Checks if PC is running Windows 7 or better
	IsWindows8 (see page 96)	Checks if PC is Running Windows 8 or better
	IsWindowsVista (see page 97)	Checks if PC is running Windows Vista or better
	IsWindowsXP (see page 97)	Checks if PC is running Windows XP
	IsWindowsXPSP2 (see page 97)	Checks if PC is running Windows XP with Service Pack 2 installed
	IsWow64 (see page 98)	Checks if the 32 bit application runs on 64 bit Windows
	LayeredWndProcA (see page 98)	The default window procedure for the layered window
	LayeredWndProcW (see page 98)	The default window procedure for the layered window
	LoadAddressA (see page 98)	Reads the raw address information from a file
	LoadAddressW (see page 99)	Reads the raw address information from a file
	LoadBitmapJPG (see page 99)	Loads a JPEG bitmap from a file
	LoadBitmapPNG (see page 100)	Loads a PNG bitmap from a file
	LoadCertificateA (see page 100)	Reads the certificate from a file
	LoadCertificateW (see page 100)	Reads the certificate from a file
	LoadIdentityA (see page 101)	Reads the raw identity information from a file
	LoadIdentityW (see page 101)	Reads the raw identity information from a file
	LoadPNGResource (see page 101)	Loads a PNG bitmap from a resource

	LoadPhotoA (see page 102)	Loads photo from a file
	LoadPhotoW (see page 102)	Loads photo from a file
	MakeCompatibleBitmap (see page 103)	Creates a bitmap compatible with the specified source bitmap
	MakeSoundFromFileA (see page 103)	Plays the wave sound from the file
	MakeSoundFromFileW (see page 103)	Plays the wave sound from the file
	MakeSoundFromResourceA (see page 104)	Plays the wave sound from the resource
	MakeSoundFromResourceW (see page 104)	Plays the wave sound from the resource
	NonRepudiationSignatureAlgoA (see page 104)	This is function NonRepudiationSignatureAlgoA.
	NonRepudiationSignatureAlgoW (see page 105)	This is function NonRepudiationSignatureAlgoW.
	PointsToPixels (see page 105)	Converts points to pixels
	PortAvailable (see page 105)	Checks if the port with specified number is available
	ReadAddressA (see page 106)	Read address information from Belgian eID card
	ReadAddressExA (see page 106)	Read address information from Belgian eID card
	ReadAddressExW (see page 106)	Read address information from Belgian eID card
	ReadAddressW (see page 107)	Read address information from Belgian eID card
	ReadAuthenticationCertificate (see page 107)	Read Authentication Certificate to memory
	ReadAuthenticationCertificateEx (see page 107)	Read Authentication Certificate to memory
	ReadBufferFromFileA (see page 108)	Reads the content of the file to the memory buffer
	ReadBufferFromFileW (see page 108)	Reads the content of the file to the memory buffer
	ReadCaCertificate (see page 109)	Read Ca Certificate to memory
	ReadCaCertificateEx (see page 109)	Read Ca Certificate to memory
	ReadIdentityA (see page 109)	Read identity information from Belgian eID card
	ReadIdentityExA (see page 110)	Read identity information from Belgian eID card
	ReadIdentityExW (see page 110)	Read identity information from Belgian eID card
	ReadIdentityW (see page 110)	Read identity information from Belgian eID card
	ReadNonRepudiationCertificate (see page 111)	Read Non Repudiation Certificate to memory
	ReadNonRepudiationCertificateEx (see page 111)	Read Non Repudiation Certificate to memory
	ReadPhoto (see page 112)	Reads a photo from a card
	ReadPhotoAsBitmap (see page 112)	Reads the picture from the card, converts it to bitmap and returns the bitmap handle Decription: Reads the photo from the Belgian eID card and returns the bitmap handle Reading the photo from the card is a time consuming operation. Do it only when needed.
	ReadPhotoAsBitmapEx (see page 112)	Reads the picture from the card, converts it to bitmap and returns the bitmap handle Decription: Reads the photo from the Belgian eID card and returns the Windows bitmap handle Reading the photo from the card is a time consuming operation. Do it only when needed.
	ReadPhotoEx (see page 113)	Reads a photo from a card
	ReadRootCaCertificate (see page 113)	Read Root Ca Certificate to memory
	ReadRootCaCertificateEx (see page 114)	Read Root Ca Certificate to memory
	ReadRrnCertificate (see page 114)	Read Rrn Certificate to memory
	ReadRrnCertificateEx (see page 114)	Read Rrn Certificate to memory

	ReadSISCardA (see page 115)	Read Belgian SIS card.
	ReadSISCardExA (see page 115)	Read Belgian SIS card.
	ReadSISCardExW (see page 116)	Read Belgian SIS card.
	ReadSISCardW (see page 116)	Read Belgian SIS card.
	RecycleBinEmpty (see page 117)	Checks if Windows Recycle Bin is empty
	ReloadReadersList (see page 117)	Reloads the list of the available card readers
	RemoveCallback (see page 117)	Remove callback procedure for card events
	RemoveStartupA (see page 117)	Removes the application from the list of the automatically started applications
	RemoveStartupW (see page 118)	Removes the application from the list of the automatically started applications
	RestoreWindowSubclassA (see page 118)	Restores window standard procedure
	RestoreWindowSubclassW (see page 118)	Restores window standard procedure
	SaveAddressA (see page 119)	Saves address information to a file
	SaveAddressW (see page 119)	Saves address information to a file
	SaveAuthenticationCertificateA (see page 119)	Save Authentication Certificate to a file
	SaveAuthenticationCertificateExW (see page 120)	Save Authentication Certificate to a file
	SaveAuthenticationCertificateW (see page 120)	Save Authentication Certificate to a file
	SaveCaCertificateA (see page 120)	Save Ca Certificate to a file
	SaveCaCertificateExW (see page 121)	Save Ca Certificate to a file
	SaveCaCertificateW (see page 121)	Save Ca Certificate to a file
	SaveCardToToXMLStreamExA (see page 122)	Read eID card and save the information to XML buffer
	SaveCardToToXMLStreamExW (see page 122)	Read eID card and save the information to XML buffer
	SaveCardToXmlA (see page 122)	Read eID card and save the information to XML file
	SaveCardToXmlExA (see page 123)	Read eID card and save the information to XML file
	SaveCardToXmlExW (see page 123)	Read eID card and save the information to XML file
	SaveCardToXmlW (see page 124)	Read eID card and save the information to XML file
	SaveContainer (see page 124)	Save container to the file
	SaveIdentityA (see page 124)	Saves indentity information to a file
	SaveIdentityW (see page 125)	Saves indentity information to a file
	SaveNonRepudiationCertificateA (see page 125)	Save Non Repudiation Certificate to a file
	SaveNonRepudiationCertificateExW (see page 126)	Save Non Repudiation Certificate to a file
	SaveNonRepudiationCertificateW (see page 126)	Save Non Repudiation Certificate to a file
	SavePersonCsvToStreamA (see page 126)	Read eID card and save the identity information to CSV memory buffer
	SavePersonCsvToStreamW (see page 127)	Read eID card and save the identity information to CSV memory buffer
	SavePersonToCsvA (see page 127)	Read eID card and save the identity information and address to CSV file
	SavePersonToCsvExA (see page 128)	Read eID card and save the identity information and address to CSV file

	SavePersonToCsvExW (see page 128)	Read eID card and save the identity information and address to CSV file
	SavePersonToCsvW (see page 128)	Read eID card and save the identity information and address to CSV file
	SavePhotoA (see page 129)	Save photo to a file
	SavePhotoAsBitmapA (see page 129)	Save the picture from the card to Windows Bitmap file Decription: Reads the photo from the Belgian eID card and writes it to the file as bitmap image. Reading the photo from the card is a time consuming operation. Do it only when needed.
	SavePhotoAsBitmapExA (see page 130)	Reads the picture from the card and saves it to Windows Bitmap file Decription: Reads the photo from the Belgian eID card and writes it to the file as bitmap image. Reading the photo from the card is a time consuming operation. Do it only when needed.
	SavePhotoAsBitmapExW (see page 130)	Reads the picture from the card and saves it to Windows Bitmap file Decription: Reads the photo from the Belgian eID card and writes it to the file as bitmap image. Reading the photo from the card is a time consuming operation. Do it only when needed.
	SavePhotoAsBitmapW (see page 130)	Save the picture from the card to Windows Bitmap file Decription: Reads the photo from the Belgian eID card and writes it to the file as bitmap image. Reading the photo from the card is a time consuming operation. Do it only when needed.
	SavePhotoAsJpegA (see page 131)	Save the picture from the card to JPG file Decription: Reads the photo from the Belgian eID card and writes it to the file as JPG image. Reading the photo from the card is a time consuming operation. Do it only when needed.
	SavePhotoAsJpegExA (see page 131)	Save the picture from the card to JPG file Decription: Reads the photo from the Belgian eID card and writes it to the file as JPG image. Reading the photo from the card is a time consuming operation. Do it only when needed.
	SavePhotoAsJpegExW (see page 131)	Save the picture from the card to JPG file Decription: Reads the photo from the Belgian eID card and writes it to the file as JPG image. Reading the photo from the card is a time consuming operation. Do it only when needed.
	SavePhotoAsJpegW (see page 132)	Save the picture from the card to JPG file Decription: Reads the photo from the Belgian eID card and writes it to the file as JPG image. Reading the photo from the card is a time consuming operation. Do it only when needed.
	SavePhotoW (see page 132)	Saves photo to a file
	SaveRootCaCertificateA (see page 133)	Save Root Ca Certificate to a file
	SaveRootCaCertificateExW (see page 133)	Save Root Ca Certificate to a file
	SaveRootCaCertificateW (see page 133)	Save Root Ca Certificate to a file
	SaveRrnCertificateA (see page 134)	Save RRN Certificate to a file
	SaveRrnCertificateExW (see page 134)	Save RRN Certificate to a file
	SaveRrnCertificateW (see page 134)	Save RRN Certificate to a file
	SelectReader (see page 135)	When more than 1 reader connected, select the reader with specified number
	SelectReaderByNameA (see page 135)	Select active smart card reader by providing the reader name
	SelectReaderByNameW (see page 136)	Select active smart card reader by providing the reader name
	SendAPDU (see page 136)	This is function SendAPDU.
	ServerAccessible (see page 136)	Checks if the server is accessible
	SetCallback (see page 137)	Activates callback procedure for card status change event

	SetContainerCanonization (see page 137)	Set XML canonization standard of the ASIC container
	SetContainerTimeServer (see page 137)	Set time server URI for digital signature of the ASIC container
	SetMWCompatibility (see page 138)	Set the compatibility mode with the old version of the oficial EID MiddleWare
	SetStartupA (see page 138)	Register application to run when Windows starts
	SetStartupW (see page 138)	Register application to run when Windows starts
	SetSupportSIS (see page 139)	Activates or deactivates SIS card support by engine
	SetTimeServer (see page 139)	Set time server URI for digital signature of the ASIC container
	ShellCopyFileA (see page 140)	Copies file to the new location
	ShellCopyFileW (see page 140)	Copies file to the new location
	ShutdownWindows (see page 140)	Logs off the interactive user, shuts down the system.
	SignPdfFile (see page 141)	Digitally sign PDF file using Belgian EID card
	SignPdfFileEx (see page 141)	Digitally sign PDF file using Belgian EID card
	StartEngine (see page 142)	Activates the Swelio Engine.
	StopEngine (see page 142)	Deactivates the Swelio Engine
	StretchNativeBitmap (see page 142)	Stretches a native bitmap to fit the specified dimensions
	StripFileNameA (see page 143)	Replaces environment variable names with values
	StripFileNameW (see page 143)	Replaces environment variable names with values
	SuspendWindows (see page 144)	Suspends Windows
	TurnMonitorOff (see page 144)	Turns the monitor off
	TurnMonitorOn (see page 144)	Turns the monitor on
	UpdateWindowPosition (see page 144)	Updated the window position
	VerifyContainer (see page 145)	Verify the signatures and the integrity of the ASIC container
	VerifyPinA (see page 145)	Verify PIN code
	VerifyPinExA (see page 145)	Verify PIN code
	VerifyPinExW (see page 146)	Verify PIN code
	VerifyPinW (see page 146)	Verify PIN code
	VerifySignature (see page 146)	Verifies the signature from the specified hash value.
	WriteBufferToFileA (see page 147)	Writes the memory buffer to file
	WriteBufferToFileW (see page 147)	Writes the memory buffer to file
	fpreset (see page 148)	Resets the floating-point unit (FPU) to its default state.

1.1.1 ActivateCard Function

Established communication between the card and the reader

C++

```
BOOL WINAPI ActivateCard();
```

File

File: Swelio.h (see page 183)

Returns

Returns TRUE if the card is activated, otherwise returns FALSE

Description

The ActivateCard function establishes a connection between the calling application and a smart card contained by a specific reader. If no card exists in the specified reader, an error is returned.

Example

```
if (IsCardPresent())
{
   ActivateCard();
}
```

1.1.2 ActivateCardEx Function

Established communication between the card and the reader

C++

```
BOOL WINAPI ActivateCardEx(int readerNumber);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
int readerNumber	The zero-based index of the card reader

Returns

Returns TRUE if the card is activated, otherwise returns FALSE

Description

The ActivateCard (see page 10) function establishes a connection between the calling application and a smart card contained by a specific reader. If no card exists in the specified reader, an error is returned.

1.1.3 AddFileToContainer Function

Add existing file to the container

C++

```
BOOL WINAPI AddFileToContainer(LPVOID container, LPSTR fileName);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPVOID container	Container handle, which is allocated by calling InitializeContainer (see page 83) function
LPSTR fileName	The name of the file which will be added to the container

Returns

Returns true if the operation is successful, otherwise returns false

Description

Call this function to include the file to the container. The file must exist.

1.1.4 AddRemoveMessageFilter Function

Adds or removes a message from the User Interface Privilege Isolation (UIPI) message filter.

C++

```
void WINAPI AddRemoveMessageFilter(UINT message, DWORD dwFlags);
```

File

File: System.h (see page 188)

Parameters

Parameters	Description
UINT message	Specifies the message to add to or remove from the filter.
DWORD dwFlags	Specifies the action to be performed. One of the following values.

Description

This function changes the message filter for Windows Vista or better. UIPI is a security feature that prevents messages from being received from a lower integrity level sender. All such messages with a value above WM_USER are blocked by default. The filter, somewhat contrary to intuition, is a list of messages that are allowed through. Therefore, adding a message to the filter allows that message to be received from a lower integrity sender, while removing a message blocks that message from being received.

Certain messages with a value less than WM_USER are required to pass through the filter regardless of the filter setting. You can call this function to remove one of those messages from the filter and it will return TRUE. However, the message will still be received by the calling process.

1.1.5 AllocateBuffer Function

Allocates the buffer in memory

C++

```
void* WINAPI AllocateBuffer(int bufferSize);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
int bufferSize	The size of the buffer

Returns

The pointer to the allocated memory block

Description

AllocateBuffer allocates a block of the given size on the heap, and returns the address of this memory.The bytes of the allocated buffer are not set to zero. To dispose of the buffer, use DeallocateBuffer (see page 31) function.

1.1.6 AllocateDefaultHWNDA Function

This function creates the invisible tool window

C++

```
HWND WINAPI AllocateDefaultHWNDA();
```

File

File: System.h (see page 188)

Returns

If the function succeeds, the return value is a handle to the new window. If the function fails, the return value is NULL.

Description

This function creates the invisible zero-size tool window that can be used for internal purposes, like processing the special Windows messages for synchronization, etc...

1.1.7 AllocateDefaultHWNDW Function

This function creates the invisible tool window

C++

```
HWND WINAPI AllocateDefaultHWNDW();
```

File

File: System.h (see page 188)

Returns

If the function succeeds, the return value is a handle to the new window. If the function fails, the return value is NULL.

Description

This function creates the invisible zero-size tool window that can be used for internal purposes, like processing the special Windows messages for synchronization, etc...

1.1.8 AllocateHWNDA Function

This function creates the invisible tool window using the provided window procedure

C++

```
HWND WINAPI AllocateHWNDA(LONG_PTR method);
```

File

File: System.h (see page 188)

Returns

If the function succeeds, the return value is a handle to the new window. If the function fails, the return value is NULL.

Description

This function creates the invisible zero-size tool window that can be used for internal purposes, like processing the special Windows messages for synchronization, etc...

1.1.9 AllocateHWNDW Function

This function creates the invisible tool window using the provided window procedure

C++

```
HWND WINAPI AllocateHWNDW(LONG_PTR method);
```

File

File: System.h (see page 188)

Returns

If the function succeeds, the return value is a handle to the new window. If the function fails, the return value is NULL.

Description

This function creates the invisible zero-size tool window that can be used for internal purposes, like processing the special Windows messages for synchronization, etc...

1.1.10 AllocateLayeredWindowA Function

This function creates the layered window using the provided window class name

C++

```
HWND WINAPI AllocateLayeredWindowA(LPCSTR className);
```

File

File: System.h (see page 188)

Returns

If the function succeeds, the return value is a handle to the new window. If the function fails, the return value is NULL.

Description

This function creates the layered window using the provided window class name

1.1.11 AllocateLayeredWindowW Function

This function creates the layered window using the provided window class name

C++

```
HWND WINAPI AllocateLayeredWindowW(LPCWSTR className);
```

File

File: System.h (see page 188)

Returns

If the function succeeds, the return value is a handle to the new window. If the function fails, the return value is NULL.

Description

This function creates the layered window using the provided window class name

1.1.12 AllocateWindowClassA Function

This function creates the standard window using the provided window class name

C++

```
HWND WINAPI AllocateWindowClassA(LPCSTR className);
```

File

File: System.h (see page 188)

Returns

If the function succeeds, the return value is a handle to the new window. If the function fails, the return value is NULL.

Description

This function creates the standard window using the provided window class name

1.1.13 AllocateWindowClassW Function

This function creates the standard window using the provided window class name

C++

```
HWND WINAPI AllocateWindowClassW(LPCWSTR className);
```

File

File: System.h (see page 188)

Returns

If the function succeeds, the return value is a handle to the new window. If the function fails, the return value is NULL.

Description

This function creates the standard window using the provided window class name

1.1.14 AlphaBlendBitmap Function

Performs alpha blending of a bitmap onto a device context

C++

```
void WINAPI AlphaBlendBitmap(HBITMAP src, HDC hdc, int left, int top, int width, int
height, int alpha);
```

File

File: Graphics.h (see page 182)

Parameters

Parameters	Description
HBITMAP src	Handle to the source bitmap
HDC hdc	Handle to the device context
int left	The left coordinate of the destination position
int top	The top coordinate of the destination position

int width	The width of the area to draw
int height	The height of the area to draw
int alpha	The alpha value for blending (0-255)
Returns	None

1.1.15 AlphaBlendNative Function

Performs alpha blending between two device contexts

C++

```
void WINAPI AlphaBlendNative(HDC hdcDest, int xoriginDest, int yoriginDest, int wDest, int
hDest, HDC hdcSrc, int xoriginSrc, int yoriginSrc, int wSrc, int hSrc, BYTE alpha);
```

File

File: Graphics.h (see page 182)

Parameters

Parameters	Description
HDC hdcDest	Handle to the destination device context
int xoriginDest	The x-coordinate of the upper-left corner of the destination rectangle
int yoriginDest	The y-coordinate of the upper-left corner of the destination rectangle
int wDest	The width of the destination rectangle
int hDest	The height of the destination rectangle
HDC hdcSrc	Handle to the source device context
int xoriginSrc	The x-coordinate of the upper-left corner of the source rectangle
int yoriginSrc	The y-coordinate of the upper-left corner of the source rectangle
int wSrc	The width of the source rectangle
int hSrc	The height of the source rectangle
BYTE alpha	The alpha value for blending (0-255)
Returns	None

1.1.16 BringWindowToFront Function

This function brings the specified window to the top of the z-order.

C++

```
void WINAPI BringWindowToFront(HWND window);
```

File

File: System.h (see page 188)

Parameters

Parameters	Description
HWND window	Handle to the window to bring to the top of the z-order.

1.1.17 CardDecryptFileA Function

Decrypt file using Belgian Id card

C++

```
BOOL WINAPI CardDecryptFileA(LPSTR szSource, LPSTR szDestination);
```

File

File: Encryption.h (see page 179)

Parameters

Parameters	Description
LPSTR szSource	The name of the encrypted file
LPSTR szDestination	The name of the decrypted file

Description

Decrypt file which was encrypted using CardEncryptFile function

1.1.18 CardDecryptFileW Function

Decrypt file using Belgian Id card

C++

```
BOOL WINAPI CardDecryptFileW(LPWSTR szSource, LPWSTR szDestination);
```

File

File: Encryption.h (see page 179)

Parameters

Parameters	Description
LPWSTR szSource	The name of the encrypted file
LPWSTR szDestination	The name of the decrypted file

Description

Decrypt file which was encrypted using CardEncryptFile function

1.1.19 CardEncryptFileA Function

Encrypt file using Belgian Id card

C++

```
BOOL WINAPI CardEncryptFileA(LPSTR szSource, LPSTR szDestination);
```

File

File: Encryption.h (see page 179)

Parameters

Parameters	Description
LPSTR szSource	The name of the source file

LPSTR szDestination	The name of the encrypted file

Description

Encrypt file using Belgian Id card. The card must be inserted in the reader

1.1.20 CardEncryptFileW Function

Encrypt file using Belgian Id card

C++

```
BOOL WINAPI CardEncryptFileW(LPWSTR szSource, LPWSTR szDestination);
```

File

File: Encryption.h (see page 179)

Parameters

Parameters	Description
LPWSTR szSource	The name of the source file
LPWSTR szDestination	The name of the encrypted file

Description

Encrypt file using Belgian Id card. The card must be inserted in the reader

1.1.21 CardSignCMS Function

Sign data with eID card according to CMS standard

C++

```
BOOL WINAPI CardSignCMS(int readerNumber, BYTE * data, UINT dataLen, BYTE * signature, UINT
* signatureLen);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
int readerNumber	The zero-based index of the card reader.
BYTE * data	the data to sign
UINT dataLen	the size of the data buffer
BYTE * signature	the signature buffer
UINT * signatureLen	the size of the signature buffer

Returns

Returns true if the operation is successful, otherwise returns false

Description

Create CMS signature for data buffer. Can be used for digital signature of PDF documents in combination with external PDF library

1.1.22 CardSignCMSEx Function

Sign data with eID card according to CMS standard

C++

```
BOOL WINAPI CardSignCMSEx(int readerNumber, BYTE* data, UINT dataLen, BYTE* signature,
UINT* signatureLen, ErrorInformation* Error);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
int readerNumber	The zero-based index of the card reader.
BYTE* data	the data to sign
UINT dataLen	the size of the data buffer
BYTE* signature	the signature buffer
UINT* signatureLen	the size of the signature buffer
error	error information

Returns

Returns true if the operation is successful, otherwise returns false

Description

Create CMS signature for data buffer. Can be used for digital signature of PDF documents in combination with external PDF library

1.1.23 CardSignCadesT Function

Sign data with eID card according to CADES-T standard

C++

```
BOOL WINAPI CardSignCadesT(int readerNumber, BYTE * data, UINT dataLen, BYTE * signature,
UINT * signatureLen);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
int readerNumber	The zero-based index of the card reader.
BYTE * data	the data to sign
UINT dataLen	the size of the data buffer
BYTE * signature	the signature buffer
UINT * signatureLen	the size of the signature buffer

Returns

Returns true if the operation is successful, otherwise returns false

Description

Create CADES-T signature for data buffer. Can be used for digital signature of PDF documents in combination with external

PDF library

1.1.24 CardSignCadesTEx Function

Sign data with eID card according to CADES-T standard

C++

```
BOOL WINAPI CardSignCadesTEx(int readerNumber, BYTE* data, UINT dataLen, BYTE* signature,
UINT* signatureLen, ErrorInformation* error);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
int readerNumber	The zero-based index of the card reader.
BYTE* data	the data to sign
UINT dataLen	the size of the data buffer
BYTE* signature	the signature buffer
UINT* signatureLen	the size of the signature buffer
ErrorInformation* error	error information

Returns

Returns true if the operation is successful, otherwise returns false

Description

Create CADES-T signature for data buffer. Can be used for digital signature of PDF documents in combination with external PDF library

1.1.25 CertSignCMS Function

Sign data with the certificate file according to CMS standard

C++

```
BOOL WINAPI CertSignCMS(LPWSTR certificate, LPWSTR password, BYTE * data, UINT dataLen,
BYTE * signature, UINT * signatureLen);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPWSTR certificate	The name of the certificate file
LPWSTR password	The private key password
BYTE * data	the data to sign
UINT dataLen	the size of the data buffer
BYTE * signature	the signature buffer
UINT * signatureLen	the size of the signature buffer

Returns

Returns true if the operation is successful, otherwise returns false

Description

Create CMS signature for data buffer. Can be used for digital signature of PDF documents in combination with external PDF library

1.1.26 CertSignCMSData Function

Sign data with the certificate according to CMS standard

C++

```
BOOL WINAPI CertSignCMSData(BYTE* certificate, DWORD certLen, LPWSTR password, BYTE* data,
UINT dataLen, BYTE* signature, UINT* signatureLen, ErrorInformation* error);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
BYTE* certificate	The byte array with the signing certificate content
DWORD certLen	The size of th signing certificate
LPWSTR password	The password of the signing certificate
BYTE* data	The data to be signed
UINT dataLen	The size of the data to be signed
BYTE* signature	The value of the signature
UINT* signatureLen	The size of the signature
ErrorInformation* error	The error information

Returns

Returns true if the operation is successful, otherwise returns false

1.1.27 CertSignCMSEx Function

Sign data with the certificate file according to CMS standard

C++

```
BOOL WINAPI CertSignCMSEx(LPWSTR certificate, LPWSTR password, BYTE* data, UINT dataLen,
BYTE* signature, UINT* signatureLen, ErrorInformation* error);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPWSTR certificate	The name of the certificate file
LPWSTR password	The private key password
BYTE* data	the data to sign
UINT dataLen	the size of the data buffer
BYTE* signature	the signature buffer
UINT* signatureLen	the size of the signature buffer
ErrorInformation* error	error information

Returns

Returns true if the operation is successful, otherwise returns false

Description

Create CMS signature for data buffer. Can be used for digital signature of PDF documents in combination with external PDF library

1.1.28 CertSignCadesT Function

Sign data with the certificate file according to CADES-T standard

C++

```
BOOL WINAPI CertSignCadesT(LPWSTR certificate, LPWSTR password, BYTE * data, UINT dataLen,
BYTE * signature, UINT * signatureLen);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPWSTR certificate	The name of the certificate file
LPWSTR password	The private key password
BYTE * data	the data to sign
UINT dataLen	the size of the data buffer
BYTE * signature	the signature buffer
UINT * signatureLen	the size of the signature buffer

Returns

Returns true if the operation is successful, otherwise returns false

Description

Create CADES-T signature for data buffer. Can be used for digital signature of PDF documents in combination with external PDF library

1.1.29 CertSignCadesTData Function

Sign data with the certificate according to CADES-T standard

C++

```
BOOL WINAPI CertSignCadesTData(BYTE* certificate, DWORD certLen, LPWSTR password, BYTE*
data, UINT dataLen, BYTE* signature, UINT* signatureLen, ErrorInformation* error);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
BYTE* certificate	The byte array with the signing certificate content
DWORD certLen	The size of th signing certificate
LPWSTR password	The password of the signing certificate

BYTE* data	The data to be signed
UINT dataLen	The size of the data to be signed
BYTE* signature	The value of the signature
UINT* signatureLen	The size of the signature
ErrorInformation* error	The error information

Returns

Returns true if the operation is successful, otherwise returns false

1.1.30 CertSignCadesTEx Function

Sign data with the certificate file according to CADES-T standard

C++

```
BOOL WINAPI CertSignCadesTEx(LPWSTR certificate, LPWSTR password, BYTE* data, UINT dataLen,
BYTE* signature, UINT* signatureLen, ErrorInformation* error);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPWSTR certificate	The name of the certificate file
LPWSTR password	The private key password
BYTE* data	the data to sign
UINT dataLen	the size of the data buffer
BYTE* signature	the signature buffer
UINT* signatureLen	the size of the signature buffer
ErrorInformation* error	error information

Returns

Returns true if the operation is successful, otherwise returns false

Description

Create CADES-T signature for data buffer. Can be used for digital signature of PDF documents in combination with external PDF library

1.1.31 CheckMD5 Function

Checks the MD5 hash value of the memory buffer

C++

```
BOOL WINAPI CheckMD5(BYTE* source, int sourceSize, BYTE* buffer, int bufferSize);
```

File

File: Encryption.h (see page 179)

Parameters

Parameters	Description
BYTE* source	The source bytes

int sourceSize	The size of the source buffer
BYTE* buffer	The hash value buffer
int bufferSize	The size of the hash value buffer

Returns

Returns TRUE if the hash value is correct, otherwise returns false

Description

This function checks if the provided value of the hash is valid

1.1.32 CheckSHA1 Function

Checks the SHA1 hash value of the memory buffer

C++

```
BOOL WINAPI CheckSHA1(BYTE* source, int sourceSize, BYTE* buffer, int bufferSize);
```

File

File: Encryption.h (see page 179)

Parameters

Parameters	Description
BYTE* source	The source bytes
int sourceSize	The size of the source buffer
BYTE* buffer	The hash value buffer
int bufferSize	The size of the hash value buffer

Returns

Returns TRUE if the hash value is correct, otherwise returns false

Description

This function checks if the provided value of the hash is valid

1.1.33 CheckSHA256 Function

Checks the SHA256 hash value of the memory buffer

C++

```
BOOL WINAPI CheckSHA256(BYTE* source, int sourceSize, BYTE* buffer, int bufferSize);
```

File

File: Encryption.h (see page 179)

Parameters

Parameters	Description
BYTE* source	The source bytes
int sourceSize	The size of the source buffer
BYTE* buffer	The hash value buffer
int bufferSize	The size of the hash value buffer

Returns

Returns TRUE if the hash value is correct, otherwise returns false

Description

This function checks if the provided value of the hash is valid

1.1.34 ClearFileAttributesA Function

This function sets the file attributes to normal.

C++

```
void WINAPI ClearFileAttributesA(LPSTR fileName);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
LPSTR fileName	The name of the file

Description

This function removed additional file attributes, like system, read-only and hidden.

1.1.35 ClearFileAttributesW Function

This function sets the file attributes to normal.

C++

```
void WINAPI ClearFileAttributesW(LPWSTR fileName);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
LPWSTR fileName	The name of the file

Description

This function removed additional file attributes, like system, read-only and hidden.

1.1.36 ClearUnusedMemory Function

Clears unused memory and minimized the application memory usage

C++

```
void WINAPI ClearUnusedMemory();
```

File

File: System.h (see page 188)

1.1.37 CloneFont Function

Clones an existing font

C++

```
HFONT WINAPI CloneFont(HFONT hFont);
```

File

File: Graphics.h (see page 182)

Parameters

Parameters	Description
HFONT hFont	Handle to the font to be cloned
Returns	A handle to the cloned font (HFONT)

1.1.38 ContainerCertificate Function

Assign certificate for signing ASIC container

C++

```
BOOL WINAPI ContainerCertificate(LPVOID container, LPWSTR fileName, LPWSTR password);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPVOID container	Container handle, which is allocated by calling InitializeContainer (see page 83) function

Returns

Returns true if the operation is successful, otherwise returns false

Description

Use this function to assign certificate which will be used to sign container at the moment when container will be saved to the file

1.1.39 ContainerEidCertificate Function

Select EID card certificate to sign ASIC container

C++

```
BOOL WINAPI ContainerEidCertificate(LPVOID container, int readerNumber);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPVOID container	Container handle, which is allocated by calling InitializeContainer (see page 83) function

Returns

Returns true if the operation is successful, otherwise returns false

Description

Call this function to select EID certificate to sign ASIC container

1.1.40 ContainerPickCertificate Function

Pick certificate to sign ASIC container

C++

```
BOOL WINAPI ContainerPickCertificate(LPVOID container);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPVOID container	Container handle, which is allocated by calling InitializeContainer (see page 83) function

Returns

Returns true if the operation is successful, otherwise returns false

Description

Use this function to let user pick the certificate to sign ASIC contaner The dialog to choose the certificate will be shown to the user

1.1.41 CopyNativeBitmap Function

Copies a native bitmap to a device context

C++

```
void WINAPI CopyNativeBitmap(HBITMAP src, HDC dstDC, int width, int height, int left, int
top);
```

File

File: Graphics.h (see page 182)

Parameters

Parameters	Description
HBITMAP src	Handle to the source bitmap
HDC dstDC	Handle to the destination device context
int width	The width of the area to copy
int height	The height of the area to copy
int left	The left coordinate of the destination position
int top	The top coordinate of the destination position
Returns	None

1.1.42 CreateCardBuffer Function

Creates XML buffer

C++

```
void* WINAPI CreateCardBuffer();
```

File

File: Swelio.h (see page 183)

Returns

The memory buffer to store information

Description

Use this function to create XML buffer

1.1.43 CreateNativeBitmap Function

Creates a new native bitmap

C++

```
HBITMAP WINAPI CreateNativeBitmap(INT width, INT height, __deref_opt_out void** ppvBits);
```

File

File: Graphics.h (see page 182)

Parameters

Parameters	Description
INT width	The width of the bitmap
INT height	The height of the bitmap
__deref_opt_out void** ppvBits	Pointer to a pointer that receives the bitmap bits (optional)
Returns	A handle to the created bitmap (HBITMAP) if successful; otherwise, NULL

1.1.44 CreateUnicodeFileA Function

Creates UNICODE file

C++

```
void WINAPI CreateUnicodeFileA(LPCSTR fileName);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
LPCSTR fileName	The name of the file

1.1.45 CreateUnicodeFileW Function

Creates UNICODE file

C++

```
void WINAPI CreateUnicodeFileW(LPCWSTR fileName);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
LPCWSTR fileName	The name of the file

1.1.46 CreateWindowsFont Function

Creates a Windows font with specified properties

C++

```
HFONT WINAPI CreateWindowsFont(LPCWSTR fontFamily, INT size, INT fontStyle, INT
fontQuality);
```

File

File: Graphics.h (see page 182)

Parameters

Parameters	Description
LPCWSTR fontFamily	The name of the font family
INT size	The size of the font in points
INT fontStyle	The style of the font (e.g., bold, italic)
INT fontQuality	The quality of the font rendering
Returns	A handle to the created font (HFONT)

1.1.47 CurrentIPAddressA Function

Returns the IP address

C++

```
void WINAPI CurrentIPAddressA(LPSTR address, UINT len);
```

File

File: SystemInfo.h (see page 189)

Returns

None.

Description

This function returns the IP address of the current system.

1.1.48 CurrentIPAddressW Function

Returns the IP address

C++

```
void WINAPI CurrentIPAddressW(LPWSTR address, UINT len);
```

File

File: SystemInfo.h (see page 189)

Returns

None.

Description

This function returns the IP address of the current system.

1.1.49 DeactivateCard Function

Terminates a cennection between a smart card and a reader

C++

```
void WINAPI DeactivateCard();
```

File

File: Swelio.h (see page 183)

Description

The DeactivateCard function terminates a connection previously opened between the calling application and a smart card in the target reader.

1.1.50 DeactivateCardEx Function

Terminates a connection between a smart card and a reader

C++

```
void WINAPI DeactivateCardEx(int readerNumber);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
int readerNumber	The zero-based index of the card reader

Description

The DeactivateCard (see page 30) function terminates a connection previously opened between the calling application and a smart card in the target reader.

1.1.51 DeallocateBuffer Function

Deallocates the memory buffer

C++

```
void WINAPI DeallocateBuffer(void* buffer);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
void* buffer	The pointer to the memory buffer

Description

DeallocateBuffer frees a memory block previously allocated with AllocateBuffer (see page 12). Use this procedure to dispose of a memory block obtained with AllocateBuffer (see page 12).

1.1.52 DeallocateHWNDA Function

This function destroys the specified window.

C++

```
BOOL WINAPI DeallocateHWNDA(HWND hwnd);
```

File

File: System.h (see page 188)

Parameters

Parameters	Description
HWND hwnd	Handle to the window to be destroyed

Description

This function restores the window default procedure and destroys the window

1.1.53 DeallocateHWNDW Function

This function destroys the specified window.

C++

```
BOOL WINAPI DeallocateHWNDW(HWND hwnd);
```

File

File: System.h (see page 188)

Parameters

Parameters	Description
HWND hwnd	Handle to the window to be destroyed

Description

This function restores the window default procedure and destroys the window

1.1.54 DecryptFileAESA Function

Decrypts file using AES algorithm.

C++

```
BOOL WINAPI DecryptFileAESA(LPSTR szSource, LPSTR szDestination, LPSTR szPassword);
```

File

File: Encryption.h (see page 179)

Parameters

Parameters	Description
LPSTR szSource	The source file name
LPSTR szDestination	The decrypted file name
LPSTR szPassword	The password

Returns

Returns TRUE if the file is successfully decrypted, otherwise returns FALSE.

Description

Use this function to decrypt the file using AES algorithm

1.1.55 DecryptFileAESW Function

Decrypts file using AES algorithm.

C++

```
BOOL WINAPI DecryptFileAESW(LPWSTR szSource, LPWSTR szDestination, LPWSTR szPassword);
```

File

File: Encryption.h (see page 179)

Parameters

Parameters	Description
LPWSTR szSource	The source file name
LPWSTR szDestination	The decrypted file name
LPWSTR szPassword	The password

Returns

Returns TRUE if the file is successfully decrypted, otherwise returns FALSE.

Description

Use this function to decrypt the file using AES algorithm

1.1.56 DeleteCardBuffer Function

Deletes XML buffer

C++

```
void WINAPI DeleteCardBuffer(void* buffer);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
void* buffer	The memory buffer to store information

Description

Use this function to delete the buffer

1.1.57 DeleteToRecycleBinA Function

Deletes file to the Windows Recycle Bin

C++

```
void WINAPI DeleteToRecycleBinA(LPSTR fileName, BOOL silent);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
LPSTR fileName	The name of the file
BOOL silent	Do not display a progress dialog box

Description

Use this function to delete the file to Windows Recycle Bin

1.1.58 DeleteToRecycleBinW Function

Deletes file to WIndows Recycle Bin

C++

```
void WINAPI DeleteToRecycleBinW(LPWSTR fileName, BOOL silent);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
LPWSTR fileName	The name of the file
BOOL silent	Do not display a progress dialog box

Description

Use this function to delete the file to Windows Recycle Bin

1.1.59 DestroyFont Function

Destroys a font and releases its resources

C++

```
void WINAPI DestroyFont(HFONT hFont);
```

File

File: Graphics.h (see page 182)

Parameters

Parameters	Description
HFONT hFont	Handle to the font to be destroyed
Returns	None

1.1.60 DestroyImageBuffer Function

Destroys the memory buffer

C++

```
void WINAPI DestroyImageBuffer(void* buffer);
```

File

File: quricol.h (see page 190)

Parameters

Parameters	Description
void* buffer	The memory buffer

Description

Destroys the memory buffer created to hold the image returned by GetPNGA (see page 74) (Ansi) or GetPNGW (see page 74) (Unicode) functions

1.1.61 DirectoryExistsA Function

Determines whether a specified directory exists.

C++

```
BOOL WINAPI DirectoryExistsA(LPSTR fileName);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
LPSTR fileName	The name of the directory

Returns

Returns TRUE if the directory exists, otherwise returns FALSE

Description

Call DirectoryExists to determine whether the directory specified by the Directory parameter exists. If the directory exists, the function returns True. If the directory does not exist, the function returns False. If a full path name is entered, DirectoryExists searches for the directory along the designated path. Otherwise, the Directory parameter is interpreted as a relative path name from the current directory.

1.1.62 DirectoryExistsW Function

Determines whether a specified directory exists.

C++

```
BOOL WINAPI DirectoryExistsW(LPWSTR fileName);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
LPWSTR fileName	The name of the directory

Returns

Returns TRUE if the directory exists, otherwise returns FALSE

Description

Call DirectoryExists to determine whether the directory specified by the Directory parameter exists. If the directory exists, the function returns True. If the directory does not exist, the function returns False. If a full path name is entered, DirectoryExists searches for the directory along the designated path. Otherwise, the Directory parameter is interpreted as a relative path name from the current directory.

1.1.63 DisplayCertificate Function

Displays the dialog window with certificate information

C++

```
void WINAPI DisplayCertificate(PEidCertificate certificate);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
PEidCertificate certificate	The certificate data

Description

Use this function to show the certificate for the user

1.1.64 DpiY Function

Gets the horizontal DPI of the display Returns: The horizontal DPI value

C++

```
int WINAPI DpiY();
```

File

File: Graphics.h (see page 182)

1.1.65 DrawAlphaText Function

Draws text with alpha blending onto a device context

C++

```
void WINAPI DrawAlphaText(HDC hDC, LPCWSTR s, HFONT hFont, COLORREF color, int width, int
height, UINT flags);
```

File

File: Graphics.h (see page 182)

Parameters

Parameters	Description
HDC hDC	Handle to the device context
LPCWSTR s	The text string to be drawn
HFONT hFont	The font to use for drawing
COLORREF color	The color of the text
int width	The width of the text area
int height	The height of the text area
UINT flags	Additional flags for drawing
Returns	None

1.1.66 DrawAlphaTextRect Function

Draws text with alpha blending within a specified rectangle on a device context

C++

```
void WINAPI DrawAlphaTextRect(HDC hDC, LPCWSTR s, HFONT hFont, COLORREF color, LPRECT
lpRect, UINT flags);
```

File

File: Graphics.h (see page 182)

Parameters

Parameters	Description
HDC hDC	Handle to the device context
LPCWSTR s	The text string to be drawn
HFONT hFont	The font to use for drawing

COLORREF color	The color of the text
LPRECT lpRect	A pointer to the rectangle that defines the text area
UINT flags	Additional flags for drawing
Returns	None

1.1.67 DrawLayeredWindow Function

Repaints the surface of the layered window

C++

```
void WINAPI DrawLayeredWindow(HWND handle, int left, int top, int width, int height, HDC buffer, COLORREF colorKey, byte alpha, BOOL redrawOnly);
```

File

File: System.h (see page 188)

Parameters

Parameters	Description
HWND handle	The handle of the window
int left	The horizontal coordinate of the window
int top	The vertical coordinate of the window
int width	The width of the window
int height	The height of the window
HDC buffer	Handle to a DC for the surface that defines the layered window
COLORREF colorKey	COLORREF structure that specifies the color key to be used when composing the layered window.
byte alpha	Specifies an alpha transparency value to be used on the entire source bitmap
BOOL redrawOnly	Only redraw and do not update the window position

1.1.68 DrawNativeBitmap Function

Draws a native bitmap onto another bitmap

C++

```
void WINAPI DrawNativeBitmap(HBITMAP src, HBITMAP dst, int width, int height);
```

File

File: Graphics.h (see page 182)

Parameters

Parameters	Description
HBITMAP src	Handle to the source bitmap
HBITMAP dst	Handle to the destination bitmap
int width	The width of the area to draw
int height	The height of the area to draw
Returns	None

1.1.69 DrawTextDirect Function

Draws text directly onto a device context

C++

```
void WINAPI DrawTextDirect(HDC hDC, LPCWSTR s, HFONT hFont, COLORREF color, int left, int top);
```

File

File: Graphics.h (see page 182)

Parameters

Parameters	Description
HDC hDC	Handle to the device context
LPCWSTR s	The text string to be drawn
HFONT hFont	The font to use for drawing
COLORREF color	The color of the text
int left	The left coordinate of the text position
int top	The top coordinate of the text position
Returns	None

1.1.70 DrawTextDirectEx Function

Draws text directly onto a device context with specified background color

C++

```
void WINAPI DrawTextDirectEx(HDC hDC, LPCWSTR s, HFONT hFont, COLORREF color, COLORREF background, int left, int top);
```

File

File: Graphics.h (see page 182)

Parameters

Parameters	Description
HDC hDC	Handle to the device context
LPCWSTR s	The text string to be drawn
HFONT hFont	The font to use for drawing
COLORREF color	The color of the text
COLORREF background	The background color
int left	The left coordinate of the text position
int top	The top coordinate of the text position
Returns	None

1.1.71 DrawTextGlow Function

Draws text with a glowing effect onto a device context

C++

```
void WINAPI DrawTextGlow(HDC hDC, LPCWSTR s, HFONT hFont, COLORREF foreColor, int left, int top);
```

File

File: Graphics.h (see page 182)

Parameters

Parameters	Description
HDC hDC	Handle to the device context
LPCWSTR s	The text string to be drawn
HFONT hFont	The font to use for drawing
COLORREF foreColor	The color of the glowing text
int left	The left coordinate of the text position
int top	The top coordinate of the text position
Returns	None

1.1.72 DrawTextLine Function

Draws a single line of text onto a device context

C++

```
void WINAPI DrawTextLine(HDC hDC, LPCWSTR s, HFONT hFont, COLORREF color, int left, int top);
```

File

File: Graphics.h (see page 182)

Parameters

Parameters	Description
HDC hDC	Handle to the device context
LPCWSTR s	The text string to be drawn
HFONT hFont	The font to use for drawing
COLORREF color	The color of the text
int left	The left coordinate of the text position
int top	The top coordinate of the text position
Returns	None

1.1.73 DrawTextOutline Function

Draws text with an outline effect onto a device context

C++

```
void WINAPI DrawTextOutline(HDC hDC, LPCWSTR s, HFONT hFont, COLORREF foreColor, COLORREF backColor, int left, int top);
```

File

File: Graphics.h (see page 182)

Parameters

Parameters	Description
HDC hDC	Handle to the device context
LPCWSTR s	The text string to be drawn
HFONT hFont	The font to use for drawing
COLORREF foreColor	The color of the text outline
COLORREF backColor	The background color
int left	The left coordinate of the text position
int top	The top coordinate of the text position
Returns	None

1.1.74 DrawTextRect Function

Draws text within a specified rectangle on a device context

C++

```
void WINAPI DrawTextRect(HDC hDC, LPCWSTR s, HFONT hFont, COLORREF color, COLORREF
background, LPRECT lpRect, UINT flags);
```

File

File: Graphics.h (see page 182)

Parameters

Parameters	Description
HDC hDC	Handle to the device context
LPCWSTR s	The text string to be drawn
HFONT hFont	The font to use for drawing
COLORREF color	The color of the text
COLORREF background	The background color
LPRECT lpRect	A pointer to the rectangle that defines the text area
UINT flags	Additional flags for drawing
Returns	None

1.1.75 EmToPixels Function

Converts em units to pixels

C++

```
int WINAPI EmToPixels(int em);
```

File

File: Graphics.h (see page 182)

Parameters

Parameters	Description
int em	The value in em units
Returns	The equivalent value in pixels

1.1.76 EmptyRecycleBin Function

Empties the recycle bin

C++

```
void WINAPI EmptyRecycleBin();
```

File

File: System.h (see page 188)

Description

Removes all files from the Windows recycle bin

1.1.77 EncodeCertificate Function

Performs Base64 encoding of the certificate

C++

```
int WINAPI EncodeCertificate(PEidCertificate certificate, BYTE* buffer, int bufferSize);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
PEidCertificate certificate	The certificate data
BYTE* buffer	The Base64 encoded certificate buffer
int bufferSize	The size of the buffer

Returns

Returns the size of the buffer needed to hold the encoded certificate

1.1.78 EncodePhoto Function

Performs Base64 encoding of the photo

C++

```
int WINAPI EncodePhoto(PeidPicture photo, BYTE* buffer, int bufferSize);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
PeidPicture photo	The pointer to EidPicture (see page 166) structure
BYTE* buffer	The Base64 encoded photo buffer
int bufferSize	The size of the buffer

Returns

Returns the size of the buffer needed to hold the encoded photo

Description

Use this function for Base64 encoding of the photo

1.1.79 EncryptFileAESA Function

Encrypts file using AES algorithm.

C++

```
BOOL WINAPI EncryptFileAESA(LPSTR szSource, LPSTR szDestination, LPSTR szPassword);
```

File

File: Encryption.h (see page 179)

Parameters

Parameters	Description
LPSTR szSource	The source file name
LPSTR szDestination	The encrypted file name
LPSTR szPassword	The password

Returns

Returns TRUE if the file is successfully encrypted, otherwise returns FALSE

Description

Use this function to encrypt the file using AES algorithm

1.1.80 EncryptFileAESW Function

Encrypts file using AES algorithm.

C++

```
BOOL WINAPI EncryptFileAESW(LPWSTR szSource, LPWSTR szDestination, LPWSTR szPassword);
```

File

File: Encryption.h (see page 179)

Parameters

Parameters	Description
LPWSTR szSource	The source file name
LPWSTR szDestination	The encrypted file name
LPWSTR szPassword	The password

Returns

Returns TRUE if the file is successfully encrypted, otherwise returns FALSE

Description

Use this function to encrypt the file using AES algorithm

1.1.81 FileCloseA Function

Concludes input/output (I/O) to a file opened using the FileCreateRewrite function.

C++

```
void WINAPI FileCloseA(HANDLE handle);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
HANDLE handle	The handle of the file

Description

Closes the file handle of the specified file when its not in use anymore

1.1.82 FileCloseW Function

Concludes input/output (I/O) to a file opened using the FileCreateRewrite function.

C++

```
void WINAPI FileCloseW(HANDLE handle);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
HANDLE handle	The handle of the file

Description

Closes the file handle of the specified file when its not in use anymore

1.1.83 FileCopyA Function

The CopyFile function copies an existing file to a new file.

C++

```
BOOL WINAPI FileCopyA(LPSTR oldName, LPSTR newName);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
LPSTR oldName	The name of the source file
LPSTR newName	The name of the destination file

Returns

The result of the function is TRUE when the file is successfully copied to the new location, otherwise the result is FALSE.

Description

This function makes a copy of the file with the new name or path.

1.1.84 FileCopyW Function

The CopyFile function copies an existing file to a new file.

C++

```
BOOL WINAPI FileCopyW(LPWSTR oldName, LPWSTR newName);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
LPWSTR oldName	The name of the source file
LPWSTR newName	The name of the destination file

Returns

The result of the function is TRUE when the file is successfully copied to the new location, otherwise the result is FALSE.

Description

This function makes a copy of the file with the new name or path.

1.1.85 FileCreateRewriteA Function

Creates new or overwrites existing file

C++

```
HANDLE WINAPI FileCreateRewriteA(LPCSTR fileName);
```

File

File: FileOperations.h (see page 180)

Returns

The result of the function is the handle of the file

Description

This function creates the new file with provided file name if the file with given name does not exists. If the file exists, it will be overwritten and the current content of the file will be lost

1.1.86 FileCreateRewriteW Function

Creates new or overwrites existing file

C++

```
HANDLE WINAPI FileCreateRewriteW(LPCWSTR fileName);
```

File

File: FileOperations.h (see page 180)

Returns

The result of the function is the handle of the file

Description

This function creates the new file with provided file name if the file with given name does not exists. If the file exists, it will be overwritten and the current content of the file will be lost

1.1.87 FileDeleteA Function

Deletes a file from disk.

C++

```
void WINAPI FileDeleteA(LPSTR fileName);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
LPSTR fileName	The name of the file

Description

DeleteFile deletes the file named by fileName from the disk.

1.1.88 FileDeleteW Function

Deletes a file from disk.

C++

```
void WINAPI FileDeleteW(LPWSTR fileName);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
LPWSTR fileName	The name of the file

Description

DeleteFile deletes the file named by fileName from the disk.

1.1.89 FileExistsA Function

Tests whether a specified file exists.

C++

```
BOOL WINAPI FileExistsA(LPSTR fileName);
```

File

File: FileOperations.h (see page 180)

Returns

FileExists returns TRUE if the file specified by FileName exists. If the file does not exist, FileExists returns FALSE.

Description

Use this function to check if the file with provided name exists.

1.1.90 FileExistsW Function

Tests whether a specified file exists.

C++

```
BOOL WINAPI FileExistsW(LPWSTR fileName);
```

File

File: FileOperations.h (see page 180)

Returns

FileExists returns TRUE if the file specified by FileName exists. If the file does not exist, FileExists returns FALSE.

Description

Use this function to check if the file with provided name exists.

1.1.91 FileExtensionIsA Function

Checks the file extension

C++

```
BOOL WINAPI FileExtensionIsA(LPCSTR fileName, LPCSTR ext);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
LPCSTR fileName	The name of the file
LPCSTR ext	The file name extension

Returns

Returns true if the file has a specified extension, otherwise returns false.

Description

This function checks if the file has a given extension

1.1.92 FileExtensionIsW Function

Checks the file extension

C++

```
BOOL WINAPI FileExtensionIsW(LPCWSTR fileName, LPCWSTR ext);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
LPCWSTR fileName	The name of the file
LPCWSTR ext	The file name extension

Returns

Returns true if the file has a specified extension, otherwise returns false.

Description

This function checks if the file has a given extension

1.1.93 FileGetSizeA Function

Retrieves the size of a specified file.

C++

```
DWORD WINAPI FileGetSizeA(LPCSTR fileName);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
LPCSTR fileName	The name of the file

Returns

The size of the file in bytes.

Description

This function determines the size of the file specified by the file name.

1.1.94 FileGetSizeW Function

Retrieves the size of a specified file.

C++

```
DWORD WINAPI FileGetSizeW(LPCWSTR fileName);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
LPCWSTR fileName	The name of the file

Returns

The size of the file in bytes.

Description

This function determines the size of the file specified by the name of the file

1.1.95 FileIsExeA Function

Checks if the file is a Windows executable

C++

```
BOOL WINAPI FileIsExeA(LPSTR fileName);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
LPSTR fileName	The name of the file

Returns

Returns TRUE if the file is a Windows executable, otherwise returns FALSE.

1.1.96 FileIsExeW Function

Checks if the file is a Windows executable

C++

```
BOOL WINAPI FileIsExeW(LPWSTR fileName);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
LPWSTR fileName	The name of the file

Returns

Returns TRUE if the file is a Windows executable, otherwise returns FALSE.

1.1.97 FileIsIconA Function

Checks if the file is a Windows icon (.ico) file

C++

```
BOOL WINAPI FileIsIconA(LPSTR fileName);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
LPSTR fileName	The name of the file

Returns

Returns TRUE if the file is a Windows icon (.ico) file, otherwise returns FALSE.

1.1.98 FileIsIconW Function

Checks if the file is a Windows icon (.ico) file

C++

```
BOOL WINAPI FileIsIconW(LPWSTR fileName);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
LPWSTR fileName	The name of the file

Returns

Returns TRUE if the file is a Windows icon (.ico) file, otherwise returns FALSE.

1.1.99 FileIsImageA Function

Checks if the file is an image file

C++

```
BOOL WINAPI FileIsImageA(LPCSTR fileName);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
LPCSTR fileName	The name of the file

Returns

Returns TRUE if the file is an image file, otherwise returns FALSE.

1.1.100 FileIsImageW Function

Checks if the file is an image file

C++

```
BOOL WINAPI FileIsImageW(LPCWSTR fileName);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
LPCWSTR fileName	The name of the file

Returns

Returns TRUE if the file is an image file, otherwise returns FALSE.

1.1.101 FileIsLink Function

Checks to see if the file specified by file name is a Microsoft Windows shortcut (.Lnk) file (and is neither a file nor a folder).

C++

```
BOOL WINAPI FileIsLink(LPCWSTR fileName);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
LPCWSTR fileName	The name of the file to check

Returns

Returns TRUE if the file is a Microsoft Windows shortcut, otherwise returns FALSE

Description

This function is used to check if the file with provided file name is a Windows shortcut or not

1.1.102 FileOrFolderExistsA Function

Checks if the file or folder with the given name exists

C++

```
BOOL WINAPI FileOrFolderExistsA(LPSTR fileName);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
LPSTR fileName	The file or folder name

Returns

Returns TRUE if the file or folder exists, otherwise returns FALSE.

1.1.103 FileOrFolderExistsW Function

Checks if the file or folder with the given name exists

C++

```
BOOL WINAPI FileOrFolderExistsW(LPWSTR fileName);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
LPWSTR fileName	The file or folder name

Returns

Returns TRUE if the file or folder exists, otherwise returns FALSE.

1.1.104 FileRenameA Function

Renames the file

C++

```
BOOL WINAPI FileRenameA(LPSTR oldName, LPSTR newName);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
LPSTR oldName	File to be renamed
LPSTR newName	New name of the file

Returns

Returns TRUE if the file was successfully renamed, otherwise returns FALSE

1.1.105 FileRenameW Function

Renames the file

C++

```
BOOL WINAPI FileRenameW(LPWSTR oldName, LPWSTR newName);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
LPWSTR oldName	File to be renamed
LPWSTR newName	New name of the file

Returns

Returns TRUE if the file was successfully renamed, otherwise returns FALSE

1.1.106 FileWriteA Function

Writes string to the file

C++

```
void WINAPI FileWriteA(HANDLE handle, LPSTR text);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
HANDLE handle	The handle of the file
LPSTR text	The text to write

1.1.107 FileWriteCharA Function

Writes one character to the file

C++

```
void WINAPI FileWriteCharA(HANDLE handle, CHAR text);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
HANDLE handle	The handle of the file
CHAR text	The character to write

1.1.108 FileWriteCharW Function

Writes one character to the file

C++

```
void WINAPI FileWriteCharW(HANDLE handle, WCHAR text);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
HANDLE handle	The handle of the file
WCHAR text	The character to write

1.1.109 FileWriteNewLineA Function

Writes new line sequence to the file

C++

```
void WINAPI FileWriteNewLineA(HANDLE handle);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
HANDLE handle	The handle of the file

1.1.110 FileWriteNewLineW Function

Writes new line sequence to the file

C++

```
void WINAPI FileWriteNewLineW(HANDLE handle);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
HANDLE handle	The handle of the file

1.1.111 FileWriteW Function

Writes string to the file

C++

```
void WINAPI FileWriteW(HANDLE handle, LPWSTR text);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
HANDLE handle	The handle of the file

LPWSTR text	The text to write

1.1.112 FreeContainer Function

Deallocates ASIC container

C++

```
void WINAPI FreeContainer(LPVOID container);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPVOID container	Container handle, which is allocated by calling InitializeContainer (see page 83) function

Returns

None

Description

Call this function to deallocate container memory and release container handle.

1.1.113 FullPathA Function

Gets the full path to the file based on file name

C++

```
BOOL WINAPI FullPathA(LPSTR fileName, LPSTR fullName);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
LPSTR fileName	The name of the file
LPSTR fullName	The full path to the file

1.1.114 FullPathW Function

Gets the full path to the file based on file name

C++

```
BOOL WINAPI FullPathW(LPWSTR fileName, LPWSTR fullName);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
LPWSTR fileName	The name of the file
LPWSTR fullName	The full path to the file

1.1.115 GenerateAuthenticationSignatureA Function

Generate authentication signature

C++

```
BOOL WINAPI GenerateAuthenticationSignatureA(LPSTR pinCode, BYTE* dataHash, int hashSize,
BYTE* signature, LPDWORD signatureSize);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPSTR pinCode	The card PIN code value
BYTE* dataHash	The hash value buffer
int hashSize	The size of the hash data buffer
BYTE* signature	The output buffer that contains the generated signature
LPDWORD signatureSize	The size of the signature buffer

Returns

Returns TRUE if the signature is successfully generated, otherwise returns FALSE

Description

Generate authentication signature using provided hash value

1.1.116 GenerateAuthenticationSignatureExA Function

Generate authentication signature

C++

```
BOOL WINAPI GenerateAuthenticationSignatureExA(int readerNumber, LPSTR pinCode, BYTE*
dataHash, int hashSize, BYTE* signature, LPDWORD signatureSize);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
int readerNumber	The zero-based index of the card reader.
LPSTR pinCode	The card PIN code value
BYTE* dataHash	The hash value buffer
int hashSize	The size of the hash data buffer
BYTE* signature	The output buffer that contains the generated signature
LPDWORD signatureSize	The size of the signature buffer

Returns

Returns TRUE if the signature is successfully generated, otherwise returns FALSE

Description

Generate authentication signature using provided hash value

1.1.117 GenerateAuthenticationSignatureExW Function

Generate authentication signature

C++

```
BOOL WINAPI GenerateAuthenticationSignatureExW(int readerNumber, LPWSTR pinCode, BYTE*
dataHash, int hashSize, BYTE* signature, LPDWORD signatureSize);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
int readerNumber	The zero-based index of the card reader.
LPWSTR pinCode	The card PIN code value
BYTE* dataHash	The hash value buffer
int hashSize	The size of the hash data buffer
BYTE* signature	The output buffer that contains the generated signature
LPDWORD signatureSize	The size of the signature buffer

Returns

Returns TRUE if the signature is successfully generated, otherwise returns FALSE

Description

Generate authentication signature using provided hash value

1.1.118 GenerateAuthenticationSignatureW Function

Generate authentication signature

C++

```
BOOL WINAPI GenerateAuthenticationSignatureW(LPWSTR pinCode, BYTE* dataHash, int hashSize,
BYTE* signature, LPDWORD signatureSize);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPWSTR pinCode	The card PIN code value
BYTE* dataHash	The hash value buffer
int hashSize	The size of the hash data buffer
BYTE* signature	The output buffer that contains the generated signature
LPDWORD signatureSize	The size of the signature buffer

Returns

Returns TRUE if the signature is successfully generated, otherwise returns FALSE

Description

Generate authentication signature using provided hash value

1.1.119 GenerateBMPA Function

Generates Windows Bitmap file with QR Code image

C++

```
void WINAPI GenerateBMPA(LPSTR fileName, LPSTR text, int margin, int size, int level);
```

File

File: quricol.h (see page 190)

Parameters

Parameters	Description
LPSTR fileName	The name of the file
LPSTR text	The text to encode
int margin	The margin from the border in points
int size	The size of the one point in pixels
int level	The error correction level

Description

Generate Windows Bitmap file with encoded text as QR Code image

1.1.120 GenerateBMPW Function

Generates Windows Bitmap file with QR Code image

C++

```
void WINAPI GenerateBMPW(LPWSTR fileName, LPWSTR text, int margin, int size, int level);
```

File

File: quricol.h (see page 190)

Parameters

Parameters	Description
LPWSTR fileName	The name of the file
LPWSTR text	The text to encode
int margin	The margin from the border in points
int size	The size of the one point in pixels
int level	The error correction level

Description

Generate Windows Bitmap file with encoded text as QR Code image

1.1.121 GenerateNonRepudiationSignatureA Function

Generate non repudiation signature

C++

```
BOOL WINAPI GenerateNonRepudiationSignatureA(LPSTR pinCode, BYTE* dataHash, int hashSize,
BYTE* signature, LPDWORD signatureSize);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPSTR pinCode	The card PIN code value
BYTE* dataHash	The hash value buffer
int hashSize	The size of the hash data buffer
BYTE* signature	The output buffer that contains the generated signature
LPDWORD signatureSize	The size of the signature buffer

Returns

Returns TRUE if the signature is successfully generated, otherwise returns FALSE

Description

Generate non repudiation signature using provided hash value

1.1.122 GenerateNonRepudiationSignatureExA Function

Generate non repudiation signature

C++

```
BOOL WINAPI GenerateNonRepudiationSignatureExA(int readerNumber, LPSTR pinCode, BYTE*
dataHash, int hashSize, BYTE* signature, LPDWORD signatureSize);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
int readerNumber	The zero-based index of the card reader.
LPSTR pinCode	The card PIN code value
BYTE* dataHash	The hash value buffer
int hashSize	The size of the hash data buffer
BYTE* signature	The output buffer that contains the generated signature
LPDWORD signatureSize	The size of the signature buffer

Returns

Returns TRUE if the signature is successfully generated, otherwise returns FALSE

Description

Generate non repudiation signature using provided hash value

1.1.123 GenerateNonRepudiationSignatureExW Function

Generate non repudiation signature

C++

```
BOOL WINAPI GenerateNonRepudiationSignatureExW(int readerNumber, LPWSTR pinCode, BYTE*
dataHash, int hashSize, BYTE* signature, LPDWORD signatureSize);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
int readerNumber	The zero-based index of the card reader.
LPWSTR pinCode	The card PIN code value
BYTE* dataHash	The hash value buffer
int hashSize	The size of the hash data buffer
BYTE* signature	The output buffer that contains the generated signature
LPDWORD signatureSize	The size of the signature buffer

Returns

Returns TRUE if the signature is successfully generated, otherwise returns FALSE

Description

Generate non repudiation signature using provided hash value

1.1.124 GenerateNonRepudiationSignatureW Function

Generate non repudiation signature

C++

```
BOOL WINAPI GenerateNonRepudiationSignatureW(LPWSTR pinCode, BYTE* dataHash, int hashSize,
BYTE* signature, LPDWORD signatureSize);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPWSTR pinCode	The card PIN code value
BYTE* dataHash	The hash value buffer
int hashSize	The size of the hash data buffer
BYTE* signature	The output buffer that contains the generated signature
LPDWORD signatureSize	The size of the signature buffer

Returns

Returns TRUE if the signature is successfully generated, otherwise returns FALSE

Description

Generate non repudiation signature using provided hash value

1.1.125 GeneratePNGA Function

Generates PNG file with QR Code image

C++

```
void WINAPI GeneratePNGA(LPSTR fileName, LPSTR text, int margin, int size, int level);
```

File

File: quricol.h (see page 190)

Parameters

Parameters	Description
LPSTR fileName	The name of the file
LPSTR text	The text to encode
int margin	The margin from the border in points
int size	The size of the one point in pixels
int level	The error correction level

Description

Generates PNG file with encoded text as QR Code image

1.1.126 GeneratePNGW Function

Generates PNG file with QR Code image

C++

```
void WINAPI GeneratePNGW(LPWSTR fileName, LPWSTR text, int margin, int size, int level);
```

File

File: quricol.h (see page 190)

Parameters

Parameters	Description
LPWSTR fileName	The name of the file
LPWSTR text	The text to encode
int margin	The margin from the border in points
int size	The size of the one point in pixels
int level	The error correction level

Description

Generates PNG file with encoded text as QR Code image

1.1.127 GenerateQRCodeA Function

Read eID card and save the identity information and address to PNG QR Code file

C++

```
BOOL WINAPI GenerateQRCodeA(LPSTR fileName);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPSTR fileName	File name to store information

Returns

Returns TRUE if the information is retrieved from the card, otherwise returns FALSE

Description

Use this function to read the information about the owner of the card and generate the QR Code PNG image

1.1.128 GenerateQRCodeExA Function

Read eID card and save the identity information and address to PNG QR Code file

C++

```
BOOL WINAPI GenerateQRCodeExA(int readerNumber, LPSTR fileName);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
int readerNumber	The zero-based index of the card reader.
LPSTR fileName	File name to store information

Returns

Returns TRUE if the information is retrieved from the card, otherwise returns FALSE

Description

Use this function to read the information about the owner of the card and generate the QR Code PNG image

1.1.129 GenerateQRCodeExW Function

Read eID card and save the identity information and address to PNG QR Code file

C++

```
BOOL WINAPI GenerateQRCodeExW(int readerNumber, LPWSTR fileName);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
int readerNumber	The zero-based index of the card reader.
LPWSTR fileName	File name to store information

Returns

Returns TRUE if the information is retrieved from the card, otherwise returns FALSE

Description

Use this function to read the information about the owner of the card and generate the QR Code PNG image

1.1.130 GenerateQRCodeW Function

Read eID card and save the identity information and address to PNG QR Code file

C++

```
BOOL WINAPI GenerateQRCodeW(LPWSTR fileName);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPWSTR fileName	File name to store information

Returns

Returns TRUE if the information is retrieved from the card, otherwise returns FALSE

Description

Use this function to read the information about the owner of the card and generate the QR Code PNG image

1.1.131 GetAllFiles Function

Returns the names of files in a specified directory.

C++

```
void WINAPI GetAllFiles(FOLDERENUMPROC lpEnumProc, LPWSTR folderName, LPWSTR searchMask,
LPARAM lParam);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
FOLDERENUMPROC lpEnumProc	Callback function
LPWSTR folderName	The name of the folder
LPWSTR searchMask	The search string to match against the names of files in path.
LPARAM lParam	Specifies an application-defined value to be passed to the callback function

Description

This function enumerates all files in the specified folder which names match the searchMask parameter and calls the callback function passing the name of the file to it.

1.1.132 GetCardBufferA Function

Gets XML or CSV information from the memory buffer

C++

```
void WINAPI GetCardBufferA(void* buffer, void* strDest, int count);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
void* buffer	Memory buffer with information
void* strDest	Destination buffer
int count	Destination buffer size

Returns

None

Description

Use this function to get the card information in CSV or XML format from the memory buffer

1.1.133 GetCardBufferSize Function

This function returns the size of the buffer needed to hold the information from the eID card in the XML or CSV format

C++

```
int WINAPI GetCardBufferSize(void* buffer);
```

File

File: Swelio.h (see page 183)

Returns

The size of the XML or CSV buffer

Description

Use this function to get the size of the XML buffer before allocating the buffer in memory

1.1.134 GetCardBufferW Function

Gets XML or CSV information from the memory buffer

C++

```
void WINAPI GetCardBufferW(void* buffer, void* strDest, int count);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
void* buffer	Memory buffer with information
void* strDest	Destination buffer
int count	Destination buffer size

Returns

None

Description

Use this function to get the card information in CSV or XML format from the memory buffer

1.1.135 GetCardSerialNumber Function

Get the serial number of EID card

C++

```
BOOL WINAPI GetCardSerialNumber(int readerNumber, BYTE* serialNumber, LPDWORD
serialNumberSize);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
int readerNumber	The zero-based index of the card reader.
BYTE* serialNumber	The memory buffer for getting the card serial number
LPDWORD serialNumberSize	the size of the memory buffer in bytes

Returns

Returns TRUE if the information is retrieved from the card, otherwise returns FALSE

Description

Use this function to get the serial number of EID card into the memory buffer

1.1.136 GetCardVersion Function

Get the applet version number for card in the reader with specified number

C++

```
int WINAPI GetCardVersion(int readerNumber);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
int readerNumber	The reader index, starting from 0

Returns

The card applet version number

Description

Get the version number of eid card applet using the zero-based reader index. The first reader has number 0, second - 1, etc... You can read the information only from one selected reader at once.

1.1.137 GetContainerError Function

Gets the value of the container validation error message based on the message index

C++

```
BOOL WINAPI GetContainerError(LPVOID container, int index, LPWSTR buffer, int bufferSize);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPVOID container	Container handle, which is allocated by calling InitializeContainer (see page 83) function
int index	error zero-based index
LPWSTR buffer	preallocated buffer for getting error message text
bufSize	the size of buffer in characters

Returns

Returns true if the operation is successful, otherwise returns false

Description

Call this function to retrieve the validation error message

1.1.138 GetContainerErrorsCount Function

Get the number of error messages availavle in the container validation report

C++

```
int WINAPI GetContainerErrorsCount(LPVOID container);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPVOID container	Container handle, which is allocated by calling InitializeContainer (see page 83) function

Returns

Returns the number of the container validation errors

Description

Call this function to get the number of the container validation errors

1.1.139 GetEncodedCertificateSize Function

Returns the size of the Base64 encoded certificate

C++

```
int WINAPI GetEncodedCertificateSize(PEidCertificate certificate);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
PEidCertificate certificate	The certificate data

Returns

Returns the size of the buffer needed to hold the encoded certificate

Description

Use this function to calculate the size of the buffer needed to encode the certificate

1.1.140 GetEncodedPhotoSize Function

Calculates buffer size for Base64 encoded photo

C++

```
int WINAPI GetEncodedPhotoSize(PeidPicture photo);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
PeidPicture photo	The pointer to EidPicture (see page 166) structure

Returns

The desired size of the buffer

Description

Use this function to calculate the size of the buffer needed for Base64 encoding of the photo This can be useful for including the photo data to the text document, for example to XML file

1.1.141 GetFileMD5A Function

Gets the MD5 hash value for the file

C++

```
BOOL WINAPI GetFileMD5A(LPSTR fileName, BYTE* buffer, int bufferSize);
```

File

File: Encryption.h (see page 179)

Parameters

Parameters	Description
LPSTR fileName	The name of the file
BYTE* buffer	The buffer to store the hash value

int bufferSize	The size of the buffer

Returns

The result of the function is equal to TRUE if operation is completed successfully, otherwise the result is FALSE

Description

Calculates MD5 hash value for the given file

1.1.142 GetFileMD5W Function

Gets the MD5 hash value for the file

C++

```
BOOL WINAPI GetFileMD5W(LPWSTR fileName, BYTE* buffer, int bufferSize);
```

File

File: Encryption.h (see page 179)

Parameters

Parameters	Description
LPWSTR fileName	The name of the file
BYTE* buffer	The buffer to store the hash value
int bufferSize	The size of the buffer

Returns

The result of the function is equal to TRUE if operation is completed successfully, otherwise the result is FALSE

Description

Calculates MD5 hash value for the given file

1.1.143 GetFileSHA1A Function

Gets the SHA1 hash value for the file

C++

```
BOOL WINAPI GetFileSHA1A(LPSTR fileName, BYTE* buffer, int bufferSize);
```

File

File: Encryption.h (see page 179)

Parameters

Parameters	Description
LPSTR fileName	The name of the file
BYTE* buffer	The buffer to store the hash value
int bufferSize	The size of the buffer

Returns

The result of the function is equal to TRUE if operation is completed successfully, otherwise the result is FALSE

Description

Calculates SHA1 hash value for the given file

1.1.144 GetFileSHA1W Function

Gets the SHA1 hash value for the file

C++

```
BOOL WINAPI GetFileSHA1W(LPWSTR fileName, BYTE* buffer, int bufferSize);
```

File

File: Encryption.h (see page 179)

Parameters

Parameters	Description
LPWSTR fileName	The name of the file
BYTE* buffer	The buffer to store the hash value
int bufferSize	The size of the buffer

Returns

The result of the function is equal to TRUE if operation is completed successfully, otherwise the result is FALSE

Description

Calculates SHA1 hash value for the given file

1.1.145 GetFileSHA256A Function

Gets the SHA256 hash value for the file

C++

```
BOOL WINAPI GetFileSHA256A(LPSTR fileName, BYTE* buffer, int bufferSize);
```

File

File: Encryption.h (see page 179)

Parameters

Parameters	Description
LPSTR fileName	The name of the file
BYTE* buffer	The buffer to store the hash value
int bufferSize	The size of the buffer

Returns

The result of the function is equal to TRUE if operation is completed successfully, otherwise the result is FALSE

Description

Calculates SHA256 hash value for the given file

1.1.146 GetFileSHA256W Function

Gets the SHA256 hash value for the file

C++

```
BOOL WINAPI GetFileSHA256W(LPWSTR fileName, BYTE* buffer, int bufferSize);
```

File

File: Encryption.h (see page 179)

Parameters

Parameters	Description
LPWSTR fileName	The name of the file
BYTE* buffer	The buffer to store the hash value
int bufferSize	The size of the buffer

Returns

The result of the function is equal to TRUE if operation is completed successfully, otherwise the result is FALSE

Description

Calculates SHA256 hash value for the given file

1.1.147 GetFileSHA384A Function

Gets the SHA384 hash value for the file

C++

```
BOOL WINAPI GetFileSHA384A(LPSTR fileName, BYTE* buffer, int bufferSize);
```

File

File: Encryption.h (see page 179)

Parameters

Parameters	Description
LPSTR fileName	The name of the file
BYTE* buffer	The buffer to store the hash value
int bufferSize	The size of the buffer

Returns

The result of the function is equal to TRUE if operation is completed successfully, otherwise the result is FALSE

Description

Calculates SHA384 hash value for the given file

1.1.148 GetFileSHA384W Function

Gets the SHA384 hash value for the file

C++

```
BOOL WINAPI GetFileSHA384W(LPWSTR fileName, BYTE* buffer, int bufferSize);
```

File

File: Encryption.h (see page 179)

Parameters

Parameters	Description
LPWSTR fileName	The name of the file
BYTE* buffer	The buffer to store the hash value
int bufferSize	The size of the buffer

Returns

The result of the function is equal to TRUE if operation is completed successfully, otherwise the result is FALSE

Description

Calculates SHA384 hash value for the given file

1.1.149 GetFileSHA512A Function

Gets the SHA512 hash value for the file

C++

```
BOOL WINAPI GetFileSHA512A(LPSTR fileName, BYTE* buffer, int bufferSize);
```

File

File: Encryption.h (see page 179)

Parameters

Parameters	Description
LPSTR fileName	The name of the file
BYTE* buffer	The buffer to store the hash value
int bufferSize	The size of the buffer

Returns

The result of the function is equal to TRUE if operation is completed successfully, otherwise the result is FALSE

Description

Calculates SHA512 hash value for the given file

1.1.150 GetFileSHA512W Function

Gets the SHA512 hash value for the file

C++

```
BOOL WINAPI GetFileSHA512W(LPWSTR fileName, BYTE* buffer, int bufferSize);
```

File

File: Encryption.h (see page 179)

Parameters

Parameters	Description
LPWSTR fileName	The name of the file
BYTE* buffer	The buffer to store the hash value
int bufferSize	The size of the buffer

Returns

The result of the function is equal to TRUE if operation is completed successfully, otherwise the result is FALSE

Description

Calculates SHA512 hash value for the given file

1.1.151 GetFilesCountA Function

Calculates the number of files in the given folder

C++

```
int WINAPI GetFilesCountA(LPSTR folderName);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
LPSTR folderName	The name of the folder

Returns

The number of files in the given folder

1.1.152 GetFilesCountW Function

Calculates the number of files in the given folder

C++

```
int WINAPI GetFilesCountW(LPWSTR folderName);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
LPWSTR folderName	The name of the folder

Returns

The number of files in the given folder

1.1.153 GetHBitmapA Function

Generates Windows Bitmap in memory with QR Code image

C++

```
HBITMAP WINAPI GetHBitmapA(LPSTR text, int margin, int size, int level);
```

File

File: quricol.h (see page 190)

Parameters

Parameters	Description
LPSTR text	The text to encode
int margin	The margin from the border in points
int size	The size of the one point in pixels
int level	The error correction level

Returns

The result of the function is HBITMAP handle. You have to destroy it by yourself when its not needed anymore.

Description

Generates Windows Bitmap in memory file with encoded text as QR Code image

1.1.154 GetHBitmapW Function

Generates Windows Bitmap in memory with QR Code image

C++

```
HBITMAP WINAPI GetHBitmapW(LPWSTR text, int margin, int size, int level);
```

File

File: quricol.h (see page 190)

Parameters

Parameters	Description
LPWSTR text	The text to encode
int margin	The margin from the border in points
int size	The size of the one point in pixels
int level	The error correction level

Returns

The result of the function is HBITMAP handle. You have to destroy it by yourself when its not needed anymore.

Description

Generates Windows Bitmap in memory file with encoded text as QR Code image

1.1.155 GetISOCodeA Function

Returns the country ISO code based on the nationality string

C++

```
BOOL WINAPI GetISOCodeA(LPCSTR nationality, LPSTR iso, int bufferSize);
```

File

File: NationalityConverter.h (see page 183)

Parameters

Parameters	Description
LPCSTR nationality	The nationality string
LPSTR iso	The ISO code memory buffer

int bufferSize	The size if the memory buffer

Returns

Returns TRUE if the ISO code is successfully obtained; FALSE otherwise

Description

This function converts the nationality string stored on ID card to the country ISO code

1.1.156 GetISOCodeW Function

Returns the country ISO code based on the nationality string

C++

```
BOOL WINAPI GetISOCodeW(LPCWSTR nationality, LPWSTR iso, int bufferSize);
```

File

File: NationalityConverter.h (see page 183)

Parameters

Parameters	Description
LPCWSTR nationality	The nationality string
LPWSTR iso	The ISO code memory buffer
int bufferSize	The size if the memory buffer

Returns

Returns TRUE if the ISO code is successfully obtained; FALSE otherwise

Description

This function converts the nationality string stored on ID card to the country ISO code

1.1.157 GetMD5 Function

Gets the MD5 hash value for the content of the memory buffer

C++

```
BOOL WINAPI GetMD5(BYTE* source, int sourceSize, BYTE* buffer, int bufferSize);
```

File

File: Encryption.h (see page 179)

Parameters

Parameters	Description
BYTE* source	The source memory block
int sourceSize	The size of the source memory block
BYTE* buffer	The buffer for the hash value
int bufferSize	The size of the destination buffer

Returns

The result of the function is equal to TRUE if operation is completed successfully, otherwise the result is FALSE

Description

Calculates MD5 hash value for the given memory buffer

1.1.158 GetPNGA Function

Writes PNG image to the memory buffer.

C++

```
void WINAPI GetPNGA(LPSTR text, int margin, int size, int level, LPINT bufSize,
__deref_opt_out void ** ppvBits);
```

File

File: quricol.h (see page 190)

Parameters

Parameters	Description
LPSTR text	The text to encode
int margin	The margin from the image border in points
int size	The size of the point in pixels
int level	The error correction level
LPINT bufSize	The size of the output buffer
__deref_opt_out void ** ppvBits	The buffer when the resulting image is stored

Description

Writes PNG image to the memory buffer. Can be useful for web development.

1.1.159 GetPNGW Function

Writes PNG image to the memory buffer.

C++

```
void WINAPI GetPNGW(LPWSTR text, int margin, int size, int level, LPINT bufSize,
__deref_opt_out void ** ppvBits);
```

File

File: quricol.h (see page 190)

Parameters

Parameters	Description
LPWSTR text	The text to encode
int margin	The margin from the image border in points
int size	The size of the point in pixels
int level	The error correction level
LPINT bufSize	The size of the output buffer
__deref_opt_out void ** ppvBits	The buffer when the resulting image is stored

Description

Writes PNG image to the memory buffer. Can be useful for web development.

1.1.160 GetReaderIndexA Function

Returns the zero-based reader index with specified name

C++

```
int WINAPI GetReaderIndexA(LPSTR readerName);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPSTR readerName	The name of the reader

Returns

The zero-based reader index

Description

Use this function to get the zero-based index of the card reader with specified name

1.1.161 GetReaderIndexW Function

Returns the zero-based reader index with specified name

C++

```
int WINAPI GetReaderIndexW(LPWSTR readerName);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPWSTR readerName	The name of the reader

Returns

The zero-based reader index

Description

Use this function to get the zero-based index of the card reader with specified name

1.1.162 GetReaderNameA Function

Returns the name of the card reader

C++

```
int WINAPI GetReaderNameA(int readerNumber, LPSTR strDest, int count);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
int readerNumber	The zero-based index of the card reader
LPSTR strDest	Destination string
int count	The number of characters to be copied

Returns

Returns the reader name length

Description

Returns the name of the card reader with the specified zero-based index

1.1.163 GetReaderNameLenA Function

Returns the length of the reader name

C++

```
int WINAPI GetReaderNameLenA(int readerNumber);
```

File

File: Swelio.h (see page 183)

Returns

The length of the reader name

Description

Returns the length of the reader name for the smart card reader with specified zero-based index

1.1.164 GetReaderNameLenW Function

Returns the length of the reader name

C++

```
int WINAPI GetReaderNameLenW(int readerNumber);
```

File

File: Swelio.h (see page 183)

Returns

The length of the reader name

Description

Returns the length of the reader name for the smart card reader with specified zero-based index

1.1.165 GetReaderNameW Function

Returns the name of the card reader

C++

```
int WINAPI GetReaderNameW(int readerNumber, LPWSTR strDest, int count);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
int readerNumber	The zero-based index of the card reader
LPWSTR strDest	Destination string
int count	Number of characters to be copied

Returns

The character count of the reader name

Description

Returns the name of the card reader with the specified zero-based index

1.1.166 GetReadersCount Function

Get number of card readers connected to PC

C++

```
int WINAPI GetReadersCount(VOID);
```

File

File: Swelio.h (see page 183)

Returns

The number of the connected smart card readers

Description

Checks how many smart card readers are connected to PC. If there is no readers connected then the usage of the Swelio Engine is not possible. The engine can control the change of the number of the card readers and can raise an event when the reader is connected or disconnected from PC

1.1.167 GetSHA1 Function

Gets the SHA1 hash value for the content of the memory buffer

C++

```
BOOL WINAPI GetSHA1(BYTE* source, int sourceSize, BYTE* buffer, int bufferSize);
```

File

File: Encryption.h (see page 179)

Parameters

Parameters	Description
BYTE* source	The source memory block
int sourceSize	The size of the source memory block
BYTE* buffer	The buffer for the hash value

int bufferSize	The size of the destination buffer

Returns

The result of the function is equal to TRUE if operation is completed successfully, otherwise the result is FALSE

Description

Calculates SHA1 hash value for the given memory buffer

1.1.168 GetSHA256 Function

Gets the SHA256 hash value for the content of the memory buffer

C++

```
BOOL WINAPI GetSHA256(BYTE* source, int sourceSize, BYTE* buffer, int bufferSize);
```

File

File: Encryption.h (see page 179)

Parameters

Parameters	Description
BYTE* source	The source memory block
int sourceSize	The size of the source memory block
BYTE* buffer	The buffer for the hash value
int bufferSize	The size of the destination buffer

Returns

The result of the function is equal to TRUE if operation is completed successfully, otherwise the result is FALSE

Description

Calculates SHA256 hash value for the given memory buffer

1.1.169 GetSHA384 Function

Gets the SHA384 hash value for the content of the memory buffer

C++

```
BOOL WINAPI GetSHA384(BYTE* source, int sourceSize, BYTE* buffer, int bufferSize);
```

File

File: Encryption.h (see page 179)

Parameters

Parameters	Description
BYTE* source	The source memory block
int sourceSize	The size of the source memory block
BYTE* buffer	The buffer for the hash value
int bufferSize	The size of the destination buffer

Returns

The result of the function is equal to TRUE if operation is completed successfully, otherwise the result is FALSE

Description

Calculates SHA384 hash value for the given memory buffer

1.1.170 GetSHA512 Function

Gets the SHA512 hash value for the content of the memory buffer

C++

```
BOOL WINAPI GetSHA512(BYTE* source, int sourceSize, BYTE* buffer, int bufferSize);
```

File

File: Encryption.h (see page 179)

Parameters

Parameters	Description
BYTE* source	The source memory block
int sourceSize	The size of the source memory block
BYTE* buffer	The buffer for the hash value
int bufferSize	The size of the destination buffer

Returns

The result of the function is equal to TRUE if operation is completed successfully, otherwise the result is FALSE

Description

Calculates SHA512 hash value for the given memory buffer

1.1.171 GetSelectedReaderIndex Function

Returns the index of the active smart card reader

C++

```
int WINAPI GetSelectedReaderIndex();
```

File

File: Swelio.h (see page 183)

Returns

The index of the selected card reader. The first reader has index 0.

Description

The zero-based index of the selected card reader. If there is only one reader is connected to PC then this reader has the index 0 and it's a default (selected) reader.

1.1.172 GetStartupA Function

Checks if the application is registered to run when Windows starts

C++

```
BOOL WINAPI GetStartupA(LPCSTR appName);
```

File

File: System.h (see page 188)

Parameters

Parameters	Description
LPCSTR appName	The name of the application

1.1.173 GetStartupW Function

Checks if the application is registered to run when Windows starts

C++

```
BOOL WINAPI GetStartupW(LPCWSTR appName);
```

File

File: System.h (see page 188)

Parameters

Parameters	Description
LPCWSTR appName	The name of the application

1.1.174 GetSupportSIS Function

Checks if the SIS cards are supported by the engine

C++

```
BOOL WINAPI GetSupportSIS();
```

File

File: Swelio.h (see page 183)

Returns

Returns TRUE if SIS card support is activated, otherwise returns FALSE

Description

The SIS card reading operation takes more time than the reading of the eID card. By default when the card is inserted in the reader the engine will try to detect the card type and the card insertion event will be raised for eID cards only. If you want to support the SIS cards in your application then you have to activate it using SetSupportSIS (see page 139) function. Use GetSupportSIS function to check if the SIS card support is activated.

1.1.175 GetTextLineSize Function

Calculates the size of a single line of text in pixels

C++

```
SIZE WINAPI GetTextLineSize(LPCWSTR s, HFONT hFont);
```

File

File: Graphics.h (see page 182)

Parameters

Parameters	Description
LPCWSTR s	The text string to be measured
HFONT hFont	The font to use for measurement
Returns	A SIZE structure containing the width and height of the text line in pixels

1.1.176 GetTextSize Function

Calculates the size of the text in pixels

C++

```
SIZE WINAPI GetTextSize(LPCWSTR s, HFONT hFont, UINT flags);
```

File

File: Graphics.h (see page 182)

Parameters

Parameters	Description
LPCWSTR s	The text string to be measured
HFONT hFont	The font to use for measurement
UINT flags	Additional flags for measurement
Returns	A SIZE structure containing the width and height of the text in pixels

1.1.177 GetTextSizeEx Function

Gets the size of text when rendered with a specific font

C++

```
SIZE WINAPI GetTextSizeEx(LPCWSTR s, HFONT hFont, UINT proposedWidth, UINT flags, BOOL
margins);
```

File

File: Graphics.h (see page 182)

Parameters

Parameters	Description
LPCWSTR s	The text string
HFONT hFont	Handle to the font
UINT proposedWidth	The proposed width of the text area
UINT flags	Flags for text measurement
BOOL margins	Whether to include margins in the size calculation
Returns	The size of the text (SIZE)

1.1.178 GetTimeServer Function

Get the time server URI for digital signature of the ASIC container

C++

```
BOOL WINAPI GetTimeServer(LPWSTR buffer, DWORD size);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPWSTR buffer	The buffer to store the time server URI
DWORD size	The size of the buffer

Returns

Returns true if the operation is successful, otherwise returns false

Description

Call this function to get the timeserver URI value

1.1.179 HibernateWindows Function

Hibernates Windows

C++

```
BOOL WINAPI HibernateWindows();
```

File

File: System.h (see page 188)

1.1.180 IgnoreHardwareEvents Function

Ignore USB reader insert / remove events

C++

```
void WINAPI IgnoreHardwareEvents(BOOL value);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
BOOL value	true - to ignore reader remove / insert events

Returns

None

1.1.181 IgnoreServiceEvents Function

Ignore smartcard service stop events when reporting readers list change

C++

```
void WINAPI IgnoreServiceEvents(BOOL value);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
BOOL value	To ignore or not the service event

Returns

None

1.1.182 IgnoreUnknownCards Function

Ignore Unknown cards insert / remove events

C++

```
void WINAPI IgnoreUnknownCards(BOOL value);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
BOOL value	true - to ignore unknown cards remove / insert events

Returns

None

1.1.183 InitializeContainer Function

Initializes ASIC container

C++

```
LPVOID WINAPI InitializeContainer();
```

File

File: Swelio.h (see page 183)

Returns

Retrns container handle pointer

Description

This functions initializes container handle needed for all container operations. Must be called first prior to other container-related calls

1.1.184 IsAnimatedGIFA Function

Checks if the file is an animated GIF image file

C++

```
BOOL WINAPI IsAnimatedGIFA(LPSTR fileName);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
LPSTR fileName	The name of the file

Returns

Returns TRUE if the file is an animated GIF image file, otherwise returns FALSE.

1.1.185 IsAnimatedGIFW Function

Checks if the file is an animated GIF image file

C++

```
BOOL WINAPI IsAnimatedGIFW(LPWSTR fileName);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
LPWSTR fileName	The name of the file

Returns

Returns TRUE if the file is an animated GIF image file, otherwise returns FALSE.

1.1.186 IsCardActivated Function

Checks the connection between a smart card and a reader

C++

```
BOOL WINAPI IsCardActivated();
```

File

File: Swelio.h (see page 183)

Description

This function checks the connection between the calling application and a smart card in the target reader.

1.1.187 IsCardActivatedEx Function

Checks the connection between a smart card and a reader

C++

```
BOOL WINAPI IsCardActivatedEx(int readerNumber);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
int readerNumber	The zero-based index of the card reader

Description

This function checks the connection between the calling application and a smart card in the target reader.

1.1.188 IsCardPresent Function

Checks if the card is present in the card reader

C++

```
BOOL WINAPI IsCardPresent();
```

File

File: Swelio.h (see page 183)

Returns

Returns TRUE if the card is inserted in the reader, otherwise returns FALSE

Description

Use IsCardPresent function to check if the card is inserted in the card reader or not

1.1.189 IsCardPresentEx Function

Checks if the card is present in the card reader

C++

```
BOOL WINAPI IsCardPresentEx(int readerNumber);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
int readerNumber	The zero-based index of the card reader

Returns

Returns TRUE if the card is inserted in the reader, otherwise returns FALSE

Description

Use isCardPresent function to check if the card is inserted in the card reader or not

1.1.190 IsCardStillInserted Function

Checks if the card is still inserted in the card reader

C++

```
BOOL WINAPI IsCardStillInserted();
```

File

File: Swelio.h (see page 183)

Description

This function checks if the card is still present in the card reader

1.1.191 IsCardStillInsertedEx Function

Checks if the card is still inserted in the card reader

C++

```
BOOL WINAPI IsCardStillInsertedEx(int readerNumber);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
int readerNumber	The zero-based index of the card reader

Description

This function checks if the card is still present in the card reader

1.1.192 IsCitrixSession Function

Checks if application is running in Citrix session

C++

```
BOOL WINAPI IsCitrixSession();
```

File

File: SystemInfo.h (see page 189)

Returns

Returns TRUE if the application is running in a Citrix session, otherwise returns FALSE.

Description

This function checks if the application is running in a Citrix session.

1.1.193 IsConnectedToInternet Function

Checks if PC is connected to Internet

C++

```
BOOL WINAPI IsConnectedToInternet();
```

File

File: SystemInfo.h (see page 189)

Returns

Returns TRUE if the PC is connected to the Internet, otherwise returns FALSE.

Description

This function checks if the PC is connected to the Internet.

1.1.194 IsDirectoryA Function

Verifies that a path is a valid directory.

C++

```
BOOL WINAPI IsDirectoryA(LPSTR folderName);
```

File

File: FileOperations.h (see page 180)

Returns

Returns TRUE if the path is a valid directory, or FALSE otherwise.

Description

This function verifies if provided value is the name of the folder

1.1.195 IsDirectoryW Function

Verifies that a path is a valid directory.

C++

```
BOOL WINAPI IsDirectoryW(LPWSTR folderName);
```

File

File: FileOperations.h (see page 180)

Returns

Returns TRUE if the path is a valid directory, or FALSE otherwise.

Description

This function verifies if provided value is the name of the folder

1.1.196 IsEIDCard Function

Check if Belgian EID card is inserted into card reader

C++

```
BOOL WINAPI IsEIDCard();
```

File

File: Swelio.h (see page 183)

Returns

Returns TRUE, if Belgian eID card is inserted in the reader. If there is no card in the reader or the card of other type is inserted, returns FALSE

Description

If the card is inserted in the reader, this function performs the card type check.

1.1.197 IsEIDCardEx Function

Check if Belgian EID card is inserted into card reader

C++

```
BOOL WINAPI IsEIDCardEx(int readerNumber);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
int readerNumber	The zero-based index of the card reader.

Returns

Returns TRUE, if Belgian eID card is inserted in the reader. If there is no card in the reader or the card of other type is inserted, returns FALSE

Description

If the card is inserted in the reader, this function performs the card type check.

1.1.198 IsEngineActive Function

Checks if the Swelio Engine is activated

C++

```
BOOL WINAPI IsEngineActive();
```

File

File: Swelio.h (see page 183)

Returns

Returns TRUE if the Swelio Engine is active, otherwise returns FALSE.

Description

This function checks if the Engine already activated using the StartEngine (see page 142) function.

1.1.199 IsFemaleA Function

Checks if the card owner is female

C++

```
BOOL WINAPI IsFemaleA(PEidIdentityA identity);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
PEidIdentityA identity	The person identity information structure

Returns

Returns TRUE if the card owner is female, otherwise returns FALSE

Description

Use this function to check the gender of the card owner

1.1.200 IsFemaleW Function

Checks if the card owner is female

C++

```
BOOL WINAPI IsFemaleW(PEidIdentityW identity);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
PEidIdentityW identity	The person identity information structure

Returns

Returns TRUE if the card owner is female, otherwise returns FALSE

Description

Use this function to check the gender of the card owner

1.1.201 IsMaleA Function

Checks if the card owner is male

C++

```
BOOL WINAPI IsMaleA(PEidIdentityA identity);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
PEidIdentityA identity	The person identity information structure

Returns

Returns TRUE if the card owner is male, otherwise returns FALSE

Description

Use this function to check the gender of the card owner

1.1.202 IsMaleW Function

Checks if the card owner is male

C++

```
BOOL WINAPI IsMaleW(PEidIdentityW identity);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
PEidIdentityW identity	The person identity information structure

Returns

Returns TRUE if the card owner is male, otherwise returns FALSE

Description

Use this function to check the gender of the card owner

1.1.203 IsMediaCenter Function

Checks if the Media Center version of Windows is installed

C++

```
BOOL WINAPI IsMediaCenter();
```

File

File: SystemInfo.h (see page 189)

Returns

Returns TRUE if the Media Center version of Windows is installed, otherwise returns FALSE.

Description

This function checks if the Media Center version of Windows is installed.

1.1.204 IsMetroActive Function

Checks if metro interface is active

C++

```
BOOL WINAPI IsMetroActive();
```

File

File: SystemInfo.h (see page 189)

Returns

Returns TRUE if the metro interface is active, otherwise returns FALSE.

Description

This function checks if the metro interface is active.

1.1.205 IsMultiTouchReady Function

Checks if the system is multi touch ready

C++

```
BOOL WINAPI IsMultiTouchReady();
```

File

File: SystemInfo.h (see page 189)

Returns

Returns TRUE if the system is multi touch ready, otherwise returns FALSE.

Description

This function checks if the system is multi touch ready.

1.1.206 IsNativeWin64 Function

Checks if the application is native 64 bit executable

C++

```
BOOL WINAPI IsNativeWin64();
```

File

File: SystemInfo.h (see page 189)

Returns

Returns TRUE if the application is a native 64 bit executable, otherwise returns FALSE.

Description

This function checks if the application is a native 64 bit executable.

1.1.207 IsRemoteSession Function

Checks if application is running in RDP session

C++

```
BOOL WINAPI IsRemoteSession();
```

File

File: SystemInfo.h (see page 189)

Returns

Returns TRUE if the application is running in an RDP session, otherwise returns FALSE.

Description

This function checks if the application is running in an RDP session.

1.1.208 IsSISCard Function

Check if Belgian SIS card is inserted into card reader

C++

```
BOOL WINAPI IsSISCard();
```

File

File: Swelio.h (see page 183)

Returns

Returns TRUE, if Belgian SIS card is inserted in the reader. If there is no card in the reader or the card of other type is inserted, returns FALSE

Description

If the card is inserted in the reader, this function performs the card type check.

1.1.209 IsSISCardEx Function

Check if Belgian SIS card is inserted into card reader

C++

```
BOOL WINAPI IsSISCardEx(int readerNumber);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
int readerNumber	The zero-based index of the card reader.

Returns

Returns TRUE, if Belgian SIS card is inserted in the reader. If there is no card in the reader or the card of other type is inserted, returns FALSE

Description

If the card is inserted in the reader, this function performs the card type check.

1.1.210 IsTabletPC Function

Checks if the application is running on the Tablet PC

C++

```
BOOL WINAPI IsTabletPC();
```

File

File: SystemInfo.h (see page 189)

Returns

Returns TRUE if the application is running on the Tablet PC, otherwise returns FALSE.

Description

This function checks if the application is running on the Tablet PC.

1.1.211 IsUnicodeFileA Function

Checks if the file is UNICODE file

C++

```
BOOL WINAPI IsUnicodeFileA(LPCSTR fileName);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
LPCSTR fileName	The name of the file

Returns

Returns TRUE if file is stored in UNICODE format, otherwise returns FALSE.

Description

This function checks the file encoding based on BOM (Byte Order Mark).

1.1.212 IsUnicodeFileW Function

Checks if the file is UNICODE file

C++

```
BOOL WINAPI IsUnicodeFileW(LPCWSTR fileName);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
LPCWSTR fileName	The name of the file

Returns

Returns TRUE if file is stored in UNICODE format, otherwise returns FALSE.

Description

This function checks the file encoding based on BOM (Byte Order Mark).

1.1.213 IsValidFileNameA Function

Checks if provided string is a valid file name

C++

```
BOOL WINAPI IsValidFileNameA(LPSTR fileName);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
LPSTR fileName	The name of the file

Returns

Returns TRUE if provided string is valid file name, otherwise returns FALSE

Description

Checks if provided string is a valid file name and does not contain any illegal characters

1.1.214 IsValidFileNameW Function

Checks if provided string is a valid file name

C++

```
BOOL WINAPI IsValidFileNameW(LPWSTR fileName);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
LPWSTR fileName	The name of the file

Returns

Returns TRUE if provided string is valid file name, otherwise returns FALSE

Description

Checks if provided string is a valid file name and does not contain any illegal characters

1.1.215 IsValidPathNameA Function

Checks if provided string is a valid file path

C++

```
BOOL WINAPI IsValidPathNameA(LPSTR fileName);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
LPSTR fileName	The file path to check

Returns

Returns TRUE if provided string is valid file path, otherwise returns FALSE

Description

Checks if provided string is a valid file path and does not contain any illegal characters

1.1.216 IsValidPathNameW Function

Checks if provided string is a valid file path

C++

```
BOOL WINAPI IsValidPathNameW(LPWSTR fileName);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
LPWSTR fileName	The file path to check

Returns

Returns TRUE if provided string is valid file path, otherwise returns FALSE

Description

Checks if provided string is a valid file path and does not contain any illegal characters

1.1.217 IsWindows10 Function

Checks if PC is running Windows 10 or better

C++

```
BOOL WINAPI IsWindows10();
```

File

File: SystemInfo.h (see page 189)

Returns

Returns TRUE if the PC is running Windows 10 or better, otherwise returns FALSE.

Description

This function checks if the PC is running Windows 10 or better.

1.1.218 IsWindows11 Function

Checks if PC is running Windows 11 or better

C++

```
BOOL WINAPI IsWindows11();
```

File

File: SystemInfo.h (see page 189)

Returns

Returns TRUE if the PC is running Windows 11 or better, otherwise returns FALSE.

Description

This function checks if the PC is running Windows 11 or better. It can be used to check if the application can use the new features of Windows 11.

1.1.219 IsWindows7 Function

Checks if PC is running Windows 7 or better

C++

```
BOOL WINAPI IsWindows7();
```

File

File: SystemInfo.h (see page 189)

Returns

Returns TRUE if the PC is running Windows 7 or better, otherwise returns FALSE.

Description

This function checks if the PC is running Windows 7 or better.

1.1.220 IsWindows8 Function

Checks if PC is Running Windows 8 or better

C++

```
BOOL WINAPI IsWindows8();
```

File

File: SystemInfo.h (see page 189)

Returns

Returns TRUE if the PC is running Windows 8 or better, otherwise returns FALSE.

Description

This function checks if the PC is running Windows 8 or better.

1.1.221 IsWindowsVista Function

Checks if PC is running Windows Vista or better

C++

```
BOOL WINAPI IsWindowsVista();
```

File

File: SystemInfo.h (see page 189)

Returns

Returns TRUE if the PC is running Windows Vista or better, otherwise returns FALSE.

Description

This function checks if the PC is running Windows Vista or better.

1.1.222 IsWindowsXP Function

Checks if PC is running Windows XP

C++

```
BOOL WINAPI IsWindowsXP();
```

File

File: SystemInfo.h (see page 189)

Returns

Returns TRUE if the PC is running Windows XP, otherwise returns FALSE.

Description

This function checks if the PC is running Windows XP.

1.1.223 IsWindowsXPSP2 Function

Checks if PC is running Windows XP with Service Pack 2 installed

C++

```
BOOL WINAPI IsWindowsXPSP2();
```

File

File: SystemInfo.h (see page 189)

Returns

Returns TRUE if the PC is running Windows XP with Service Pack 2 installed, otherwise returns FALSE.

Description

This function checks if the PC is running Windows XP with Service Pack 2 installed.

1.1.224 IsWow64 Function

Checks if the 32 bit application runs on 64 bit Windows

C++

```
BOOL WINAPI IsWow64();
```

File

File: SystemInfo.h (see page 189)

Returns

Returns TRUE if the 32 bit application is running on 64 bit Windows, otherwise returns FALSE.

Description

This function checks if the 32 bit application is running on 64 bit Windows.

1.1.225 LayeredWndProcA Function

The default window procedure for the layered window

C++

```
LRESULT CALLBACK LayeredWndProcA(HWND hWnd, UINT message, WPARAM wParam, LPARAM lParam);
```

File

File: System.h (see page 188)

1.1.226 LayeredWndProcW Function

The default window procedure for the layered window

C++

```
LRESULT CALLBACK LayeredWndProcW(HWND hWnd, UINT message, WPARAM wParam, LPARAM lParam);
```

File

File: System.h (see page 188)

1.1.227 LoadAddressA Function

Reads the raw address information from a file

C++

```
void WINAPI LoadAddressA(LPSTR fileName, PEidAddressA address);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPSTR fileName	The name of the source file
PEidAddressA address	The pointer to EidAddressA (see page 159) structure

Description

Use this function to read back the address information stored to the file using SaveAddressA (see page 119) function

1.1.228 LoadAddressW Function

Reads the raw address information from a file

C++

```
void WINAPI LoadAddressW(LPWSTR fileName, PEidAddressW address);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPWSTR fileName	The name of the source file
PEidAddressW address	The pointer to EidAddressW (see page 160) structure

Description

Use this function to read back the address information stored to the file using SaveAddressW (see page 119) function

1.1.229 LoadBitmapJPG Function

Loads a JPEG bitmap from a file

C++

```
HBITMAP WINAPI LoadBitmapJPG(LPCWSTR szFile, LPINT lpiWidth, LPINT lpiHeight,
__deref_opt_out void** ppvBits);
```

File

File: Graphics.h (see page 182)

Parameters

Parameters	Description
LPCWSTR szFile	The path to the JPEG file
LPINT lpiWidth	Pointer to an integer that receives the width of the bitmap
LPINT lpiHeight	Pointer to an integer that receives the height of the bitmap
__deref_opt_out void** ppvBits	Pointer to a pointer that receives the bitmap bits (optional)
Returns	A handle to the loaded bitmap (HBITMAP) if successful; otherwise, NULL

1.1.230 LoadBitmapPNG Function

Loads a PNG bitmap from a file

C++

```
HBITMAP WINAPI LoadBitmapPNG(LPCWSTR szFile, LPINT lpiWidth, LPINT lpiHeight,
__deref_opt_out void** ppvBits);
```

File

File: Graphics.h (see page 182)

Parameters

Parameters	Description
LPCWSTR szFile	The path to the PNG file
LPINT lpiWidth	Pointer to an integer that receives the width of the bitmap
LPINT lpiHeight	Pointer to an integer that receives the height of the bitmap
__deref_opt_out void** ppvBits	Pointer to a pointer that receives the bitmap bits (optional)
Returns	A handle to the loaded bitmap (HBITMAP) if successful; otherwise, NULL

1.1.231 LoadCertificateA Function

Reads the certificate from a file

C++

```
void WINAPI LoadCertificateA(LPSTR fileName, PEidCertificate certificate);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPSTR fileName	The source file name
PEidCertificate certificate	The pointer to EidCertificate (see page 160) structure

Description

Use this function to read the certificate from the file

1.1.232 LoadCertificateW Function

Reads the certificate from a file

C++

```
void WINAPI LoadCertificateW(LPWSTR fileName, PEidCertificate certificate);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPWSTR fileName	The source file name
PEidCertificate certificate	The pointer to EidCertificate (see page 160) structure

Description

Use this function to read the certificate from the file

1.1.233 LoadIdentityA Function

Reads the raw identity information from a file

C++

```
void WINAPI LoadIdentityA(LPSTR fileName, PEidIdentityA identity);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPSTR fileName	The name of the source file
PEidIdentityA identity	The pointer to EidIdentityA (see page 160) structure

Description

Use this function to read back the identity information stored to the file using SaveIdentityA (see page 124) function

1.1.234 LoadIdentityW Function

Reads the raw identity information from a file

C++

```
void WINAPI LoadIdentityW(LPWSTR fileName, PEidIdentityW identity);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPWSTR fileName	The name of the source file
PEidIdentityW identity	The pointer to EidIdentityW (see page 165) structure

Description

Use this function to read back the identity information stored to the file using SaveIdentityW (see page 125) function

1.1.235 LoadPNGResource Function

Loads a PNG bitmap from a resource

C++

```
HBITMAP WINAPI LoadPNGResource(HMODULE handle, LPCWSTR szName, LPINT lpiWidth, LPINT
lpiHeight, __deref_opt_out void** ppvBits);
```

File

File: Graphics.h (see page 182)

Parameters

Parameters	Description
HMODULE handle	Handle to the module containing the resource
LPCWSTR szName	The name of the PNG resource
LPINT lpiWidth	Pointer to an integer that receives the width of the bitmap
LPINT lpiHeight	Pointer to an integer that receives the height of the bitmap
__deref_opt_out void** ppvBits	Pointer to a pointer that receives the bitmap bits (optional)
Returns	A handle to the loaded bitmap (HBITMAP) if successful; otherwise, NULL

1.1.236 LoadPhotoA Function

Loads photo from a file

C++

```
void WINAPI LoadPhotoA(PeidPicture photo, LPSTR fileName);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
PeidPicture photo	The pointer to EidPicture (see page 166) structure
LPSTR fileName	Destination file name

Description

Loads raw picture data from a file

1.1.237 LoadPhotoW Function

Loads photo from a file

C++

```
void WINAPI LoadPhotoW(PeidPicture photo, LPWSTR fileName);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
PeidPicture photo	The pointer to EidPicture (see page 166) structure
LPWSTR fileName	Destination file name

Description

Loads raw picture data from a file

1.1.238 MakeCompatibleBitmap Function

Creates a bitmap compatible with the specified source bitmap

C++

```
HBITMAP WINAPI MakeCompatibleBitmap(HBITMAP source, int width, int height);
```

File

File: Graphics.h (see page 182)

Parameters

Parameters	Description
HBITMAP source	Handle to the source bitmap
int width	The width of the new bitmap
int height	The height of the new bitmap
Returns	A handle to the created compatible bitmap (HBITMAP)

1.1.239 MakeSoundFromFileA Function

Plays the wave sound from the file

C++

```
void WINAPI MakeSoundFromFileA(LPCSTR soundName);
```

File

File: System.h (see page 188)

Parameters

Parameters	Description
LPCSTR soundName	The name of the file

Description

This function plays a sound specified by the given file name.

1.1.240 MakeSoundFromFileW Function

Plays the wave sound from the file

C++

```
void WINAPI MakeSoundFromFileW(LPCWSTR soundName);
```

File

File: System.h (see page 188)

Parameters

Parameters	Description
LPCWSTR soundName	The name of the file

Description

This function plays a sound specified by the given file name.

1.1.241 MakeSoundFromResourceA Function

Plays the wave sound from the resource

C++

```
void WINAPI MakeSoundFromResourceA(HMODULE hModule, LPCSTR soundName);
```

File

File: System.h (see page 188)

Parameters

Parameters	Description
HMODULE hModule	Handle to the executable file that contains the resource to be loaded.
LPCSTR soundName	A string that specifies the sound to play.

Description

This function plays a sound specified by the given resource name.

1.1.242 MakeSoundFromResourceW Function

Plays the wave sound from the resource

C++

```
void WINAPI MakeSoundFromResourceW(HMODULE hModule, LPCWSTR soundName);
```

File

File: System.h (see page 188)

Parameters

Parameters	Description
HMODULE hModule	Handle to the executable file that contains the resource to be loaded.
LPCWSTR soundName	A string that specifies the sound to play.

Description

This function plays a sound specified by the given resource name.

1.1.243 NonRepudiationSignatureAlgoA Function

This is function NonRepudiationSignatureAlgoA.

C++

```
BOOL WINAPI NonRepudiationSignatureAlgoA(int readerNumber, LPSTR pinCode, BYTE* dataHash,
int hashSize, BYTE* signature, LPDWORD signatureSize, BYTE algorithm);
```

File

File: Swelio.h (see page 183)

1.1.244 NonRepudiationSignatureAlgoW Function

This is function NonRepudiationSignatureAlgoW.

C++

```
BOOL WINAPI NonRepudiationSignatureAlgoW(int readerNumber, LPWSTR pinCode, BYTE* dataHash,
int hashSize, BYTE* signature, LPDWORD signatureSize, BYTE algorithm);
```

File

File: Swelio.h (see page 183)

1.1.245 PointsToPixels Function

Converts points to pixels

C++

```
int WINAPI PointsToPixels(int points);
```

File

File: Graphics.h (see page 182)

Parameters

Parameters	Description
int points	The value in points
Returns	The equivalent value in pixels

1.1.246 PortAvailable Function

Checks if the port with specified number is available

C++

```
BOOL WINAPI PortAvailable(int port);
```

File

File: SystemInfo.h (see page 189)

Returns

Returns TRUE if the port is available, otherwise returns FALSE.

Description

This function checks if the port with the specified number is available.

1.1.247 ReadAddressA Function

Read address information from Belgian eID card

C++

```
BOOL WINAPI ReadAddressA(PEidAddressA address);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
PEidAddressA address	The pointer to the address information structure

Returns

TRUE when information is successfully received from the card; otherwise returns FALSE

1.1.248 ReadAddressExA Function

Read address information from Belgian eID card

C++

```
BOOL WINAPI ReadAddressExA(int readerNumber, PEidAddressA address);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
int readerNumber	The zero-based index of the card reader.
PEidAddressA address	The pointer to the address information structure

Returns

TRUE when information is successfully received from the card; otherwise returns FALSE

1.1.249 ReadAddressExW Function

Read address information from Belgian eID card

C++

```
BOOL WINAPI ReadAddressExW(int readerNumber, PEidAddressW address);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
int readerNumber	The zero-based index of the card reader.
PEidAddressW address	The pointer to the address information structure

Returns

TRUE when information is successfully received from the card; otherwise returns FALSE

1.1.250 ReadAddressW Function

Read address information from Belgian eID card

C++

```
BOOL WINAPI ReadAddressW(PEidAddressW address);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
PEidAddressW address	the pointer to the address information structure

Returns

TRUE when information is successfully received from the card; otherwise returns FALSE

1.1.251 ReadAuthenticationCertificate Function

Read Authentication Certificate to memory

C++

```
BOOL WINAPI ReadAuthenticationCertificate(PEidCertificate certificate);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
cerificate	The pointer to EidCertificate (see page 160) structure

Returns

TRUE when certificate is successfully received from the card; otherwise returns FALSE

Description

Read Authentication Certificate from the card to EidCertificate (see page 160) structure

1.1.252 ReadAuthenticationCertificateEx Function

Read Authentication Certificate to memory

C++

```
BOOL WINAPI ReadAuthenticationCertificateEx(int readerNumber, PEidCertificate certificate);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
int readerNumber	The zero-based index of the card reader
cerificate	The pointer to EidCertificate (see page 160) structure

Returns

TRUE when certificate is successfully received from the card; otherwise returns FALSE

Description

Read Authentication Certificate from the card to EidCertificate (see page 160) structure

1.1.253 ReadBufferFromFileA Function

Reads the content of the file to the memory buffer

C++

```
void WINAPI ReadBufferFromFileA(LPSTR fileName, BYTE* buffer, int bufferSize);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
LPSTR fileName	The name of the file
BYTE* buffer	The address of the memory block
int bufferSize	The size of the memory block

Description

Use this function to retrieve the content of the file to the memory block

1.1.254 ReadBufferFromFileW Function

Reads the content of the file to the memory buffer

C++

```
void WINAPI ReadBufferFromFileW(LPWSTR fileName, BYTE* buffer, int bufferSize);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
LPWSTR fileName	The name of the file
BYTE* buffer	The address of the memory block
int bufferSize	The size of the memory block

Description

Use this function to retrieve the content of the file to the memory block

1.1.255 ReadCaCertificate Function

Read Ca Certificate to memory

C++

```
BOOL WINAPI ReadCaCertificate(PEidCertificate certificate);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
cerificate	The pointer to EidCertificate (see page 160) structure

Returns

TRUE when certificate is successfully received from the card; otherwise returns FALSE

Description

Read Ca Certificate to EidCertificate (see page 160) structure

1.1.256 ReadCaCertificateEx Function

Read Ca Certificate to memory

C++

```
BOOL WINAPI ReadCaCertificateEx(int readerNumber, PEidCertificate certificate);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
int readerNumber	The zero-based index of the card reader
cerificate	The pointer to EidCertificate (see page 160) structure

Returns

TRUE when certificate is successfully received from the card; otherwise returns FALSE

Description

Read Ca Certificate to EidCertificate (see page 160) structure

1.1.257 ReadIdentityA Function

Read identity information from Belgian eID card

C++

```
BOOL WINAPI ReadIdentityA(PEidIdentityA identity);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
PEidIdentityA identity	The pointer to the identity information structure

Returns

TRUE when information is successfully received from the card; otherwise returns FALSE

1.1.258 ReadIdentityExA Function

Read identity information from Belgian eID card

C++

```
BOOL WINAPI ReadIdentityExA(int readerNumber, PEidIdentityA identity);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
int readerNumber	The zero-based index of the card reader.
PEidIdentityA identity	The pointer to the identity information structure

Returns

Returns TRUE when information is successfully received from the card; otherwise returns FALSE

1.1.259 ReadIdentityExW Function

Read identity information from Belgian eID card

C++

```
BOOL WINAPI ReadIdentityExW(int readerNumber, PEidIdentityW identity);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
int readerNumber	The zero-based index of the card reader.
PEidIdentityW identity	The pointer to the identity information structure

Returns

TRUE when information is successfully received from the card; otherwise returns FALSE

1.1.260 ReadIdentityW Function

Read identity information from Belgian eID card

C++

```
BOOL WINAPI ReadIdentityW(PEidIdentityW identity);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
PEidIdentityW identity	The pointer to the identity information structure

Returns

TRUE when information is successfully received from the card; otherwise returns FALSE

1.1.261 ReadNonRepudiationCertificate Function

Read Non Repudiation Certificate to memory

C++

```
BOOL WINAPI ReadNonRepudiationCertificate(PEidCertificate certificate);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
cerificate	The pointer to EidCertificate (see page 160) structure

Returns

TRUE when certificate is successfully received from the card; otherwise returns FALSE

Description

Read Non Repudiation Certificate to EidCertificate (see page 160) structure

1.1.262 ReadNonRepudiationCertificateEx Function

Read Non Repudiation Certificate to memory

C++

```
BOOL WINAPI ReadNonRepudiationCertificateEx(int readerNumber, PEidCertificate certificate);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
int readerNumber	The zero-based index of the card reader
cerificate	The pointer to EidCertificate (see page 160) structure

Returns

TRUE when certificate is successfully received from the card; otherwise returns FALSE

Description

Read Non Repudiation Certificate to EidCertificate (see page 160) structure

1.1.263 ReadPhoto Function

Reads a photo from a card

C++

```
BOOL WINAPI ReadPhoto(PeidPicture photo);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
PeidPicture photo	The pointer to EidPicture (see page 166) structure

Returns

Returns TRUE if the photo is retrieved from the card, otherwise return FALSE

Description

Reads a photo from Belgian eID card to EidPicture (see page 166) structure. This structure holds the raw image bytes and the length of the image bytes array

1.1.264 ReadPhotoAsBitmap Function

Reads the picture from the card, converts it to bitmap and returns the bitmap handle Decription: Reads the photo from the Belgian eID card and returns the bitmap handle Reading the photo from the card is a time consuming operation. Do it only when needed.

C++

```
HBITMAP WINAPI ReadPhotoAsBitmap();
```

File

File: Swelio.h (see page 183)

Returns

A handle to a bitmap indicates success. NULL indicates failure.

1.1.265 ReadPhotoAsBitmapEx Function

Reads the picture from the card, converts it to bitmap and returns the bitmap handle Decription: Reads the photo from the Belgian eID card and returns the Windows bitmap handle Reading the photo from the card is a time consuming operation. Do it only when needed.

C++

```
HBITMAP WINAPI ReadPhotoAsBitmapEx(int readerNumber);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
int readerNumber	The zero-based index of the card reader.

Returns

A handle to a bitmap indicates success. NULL indicates failure.

1.1.266 ReadPhotoEx Function

Reads a photo from a card

C++

```
BOOL WINAPI ReadPhotoEx(int readerNumber, PeidPicture photo);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
int readerNumber	The zero-based index of the card reader.
PeidPicture photo	The pointer to EidPicture (see page 166) structure

Returns

Returns TRUE if the photo is retrieved from the card, otherwise return FALSE

Description

Reads a photo from Belgian eID card to EidPicture (see page 166) structure

1.1.267 ReadRootCaCertificate Function

Read Root Ca Certificate to memory

C++

```
BOOL WINAPI ReadRootCaCertificate(PEidCertificate certificate);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
cerificate	The pointer to EidCertificate (see page 160) structure

Returns

TRUE when certificate is successfully received from the card; otherwise returns FALSE

Description

Read Root Ca Certificate to EidCertificate (see page 160) structure

1.1.268 ReadRootCaCertificateEx Function

Read Root Ca Certificate to memory

C++

```
BOOL WINAPI ReadRootCaCertificateEx(int readerNumber, PEidCertificate certificate);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
int readerNumber	The zero-based index of the card reader
cerificate	The pointer to EidCertificate (see page 160) structure

Returns

TRUE when certificate is successfully received from the card; otherwise returns FALSE

Description

Read Root Ca Certificate to EidCertificate (see page 160) structure

1.1.269 ReadRrnCertificate Function

Read Rrn Certificate to memory

C++

```
BOOL WINAPI ReadRrnCertificate(PEidCertificate certificate);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
cerificate	The pointer to EidCertificate (see page 160) structure

Returns

TRUE when certificate is successfully received from the card; otherwise returns FALSE

Description

Read Rrn Certificate to EidCertificate (see page 160) structure

1.1.270 ReadRrnCertificateEx Function

Read Rrn Certificate to memory

C++

```
BOOL WINAPI ReadRrnCertificateEx(int readerNumber, PEidCertificate certificate);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
int readerNumber	The zero-based index of the card reader
cerificate	The pointer to EidCertificate (see page 160) structure

Returns

TRUE when certificate is successfully received from the card; otherwise returns FALSE

Description

Read Rrn Certificate to EidCertificate (see page 160) structure

1.1.271 ReadSISCardA Function

Read Belgian SIS card.

C++

```
BOOL WINAPI ReadSISCardA(PSISRecordA identity);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
PSISRecordA	The pointer to SISRecordA (see page 176) structure

Returns

TRUE when information is successfully received from the card; otherwise returns FALSE

Description

Read the public information from the Belgian SIS card. The SIS card is the social security card of each Belgian resident (Belgian or foreigner)

1.1.272 ReadSISCardExA Function

Read Belgian SIS card.

C++

```
BOOL WINAPI ReadSISCardExA(int readerNumber, PSISRecordA identity);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
int readerNumber	The zero-based index of the card reader.
PSISRecordA	The pointer to SISRecordA (see page 176) structure

Returns

TRUE when information is successfully received from the card; otherwise returns FALSE

Description

Read the public information from the Belgian SIS card. The SIS card is the social security card of each Belgian resident (Belgian or foreigner)

1.1.273 ReadSISCardExW Function

Read Belgian SIS card.

C++

```
BOOL WINAPI ReadSISCardExW(int readerNumber, PSISRecordW identity);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
int readerNumber	The zero-based index of the card reader.
PSISRecordW	The pointer to SISRecordW (see page 177) structure

Returns

TRUE when information is successfully received from the card; otherwise returns FALSE

Description

Read the public information from the Belgian SIS card. The SIS card is the social security card of each Belgian resident (Belgian or foreigner)

1.1.274 ReadSISCardW Function

Read Belgian SIS card.

C++

```
BOOL WINAPI ReadSISCardW(PSISRecordW identity);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
PSISRecordW	The pointer to SISRecordW (see page 177) structure

Returns

TRUE when information is successfully received from the card; otherwise returns FALSE

Description

Read the public information from the Belgian SIS card. The SIS card is the social security card of each Belgian resident (Belgian or foreigner)

1.1.275 RecycleBinEmpty Function

Checks if Windows Recycle Bin is empty

C++

```
BOOL WINAPI RecycleBinEmpty();
```

File

File: SystemInfo.h (see page 189)

Returns

Returns TRUE if the Recycle Bin is empty, otherwise returns FALSE.

Description

This function checks if the Windows Recycle Bin is empty.

1.1.276 ReloadReadersList Function

Reloads the list of the available card readers

C++

```
void WINAPI ReloadReadersList();
```

File

File: Swelio.h (see page 183)

Description

When the card reader is inserted or removed you may need to reload the list of the available card readers

1.1.277 RemoveCallback Function

Remove callback procedure for card events

C++

```
void WINAPI RemoveCallback();
```

File

File: Swelio.h (see page 183)

Description

Use this function to deactivate card events callback procedure

1.1.278 RemoveStartupA Function

Removes the application from the list of the automatically started applications

C++

```
void WINAPI RemoveStartupA(LPCSTR appName);
```

File

File: System.h (see page 188)

Parameters

Parameters	Description
LPCSTR appName	The name of the application

Description

For application that starts automatically when Windows starts removes it from the automatically launching applications list

1.1.279 RemoveStartupW Function

Removes the application from the list of the automatically started applications

C++

```
void WINAPI RemoveStartupW(LPCWSTR appName);
```

File

File: System.h (see page 188)

Parameters

Parameters	Description
LPCWSTR appName	The name of the application

Description

For application that starts automatically when Windows starts removes it from the automatically launching applications list

1.1.280 RestoreWindowSubclassA Function

Restores window standard procedure

C++

```
void WINAPI RestoreWindowSubclassA(HWND hwnd);
```

File

File: System.h (see page 188)

Parameters

Parameters	Description
HWND hwnd	The window handle

1.1.281 RestoreWindowSubclassW Function

Restores window standard procedure

C++

```
void WINAPI RestoreWindowSubclassW(HWND hwnd);
```

File

File: System.h (see page 188)

Parameters

Parameters	Description
HWND hwnd	The window handle

1.1.282 SaveAddressA Function

Saves address information to a file

C++

```
void WINAPI SaveAddressA(LPSTR fileName);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPSTR fileName	The name of the destination file

Description

Use this function to store the raw address information from the Belgian eID card to a file. You can use LoadAddressA (see page 98) to read this information from the file to EidAddressA (see page 159) structure

1.1.283 SaveAddressW Function

Saves address information to a file

C++

```
void WINAPI SaveAddressW(LPWSTR fileName);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPWSTR fileName	The name of the destination file

Description

Use this function to store the raw address information from the Belgian eID card to a file. You can use LoadAddressW (see page 99) to read this information from the file to EidAddressW (see page 160) structure

1.1.284 SaveAuthenticationCertificateA Function

Save Authentication Certificate to a file

C++

```
void WINAPI SaveAuthenticationCertificateA(LPSTR fileName);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPSTR fileName	File name to store the certificate

Description

Read Authentication Certificate from the card and save it to a file.

1.1.285 SaveAuthenticationCertificateExW Function

Save Authentication Certificate to a file

C++

```
void WINAPI SaveAuthenticationCertificateExW(LPWSTR fileName, int readerNumber);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPWSTR fileName	File name to store the certificate
int readerNumber	The reader index, starting from 0

Description

Read Authentication Certificate from the card and save it to a file.

1.1.286 SaveAuthenticationCertificateW Function

Save Authentication Certificate to a file

C++

```
void WINAPI SaveAuthenticationCertificateW(LPWSTR fileName);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPWSTR fileName	File name to store the certificate

Description

Read Authentication Certificate from the card and save it to a file.

1.1.287 SaveCaCertificateA Function

Save Ca Certificate to a file

C++

```
void WINAPI SaveCaCertificateA(LPSTR fileName);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPSTR fileName	File name to store the certificate

Description

Read Ca Certificate from the card and save it to a file

1.1.288 SaveCaCertificateExW Function

Save Ca Certificate to a file

C++

```
void WINAPI SaveCaCertificateExW(LPWSTR fileName, int readerNumber);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPWSTR fileName	File name to store the certificate
int readerNumber	The reader index, starting from 0

Description

Read Ca Certificate from the card and save it to a file

1.1.289 SaveCaCertificateW Function

Save Ca Certificate to a file

C++

```
void WINAPI SaveCaCertificateW(LPWSTR fileName);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPWSTR fileName	File name to store the certificate

Description

Read Ca Certificate from the card and save it to a file

1.1.290 SaveCardToToXMLStreamExA Function

Read eID card and save the information to XML buffer

C++

```
BOOL WINAPI SaveCardToToXMLStreamExA(int readerNumber, void* buffer);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
int readerNumber	The zero-based index of the card reader.
void* buffer	The memory buffer to store information

Returns

Returns TRUE if the information is retrieved from the card, otherwise returns FALSE

Description

Use this function to read the information about the owner of the card and save it to XML buffer in the memory.

1.1.291 SaveCardToToXMLStreamExW Function

Read eID card and save the information to XML buffer

C++

```
BOOL WINAPI SaveCardToToXMLStreamExW(int readerNumber, void* buffer);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
int readerNumber	The zero-based index of the card reader.
void* buffer	The memory buffer to store information

Returns

Returns TRUE if the information is retrieved from the card, otherwise returns FALSE

Description

Use this function to read the information about the owner of the card and save it to XML buffer in the memory.

1.1.292 SaveCardToXmlA Function

Read eID card and save the information to XML file

C++

```
BOOL WINAPI SaveCardToXmlA(LPSTR fileName);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPSTR fileName	File name to store information

Returns

Returns TRUE if the information is retrieved from the card, otherwise returns FALSE

Description

Use this function to read the information about the owner of the card and save it to XML file.

1.1.293 SaveCardToXmlExA Function

Read eID card and save the information to XML file

C++

```
BOOL WINAPI SaveCardToXmlExA(int readerNumber, LPSTR fileName);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
int readerNumber	The zero-based index of the card reader.
LPSTR fileName	File name to store information

Returns

Returns TRUE if the information is retrieved from the card, otherwise returns FALSE

Description

Use this function to read the information about the owner of the card and save it to XML file.

1.1.294 SaveCardToXmlExW Function

Read eID card and save the information to XML file

C++

```
BOOL WINAPI SaveCardToXmlExW(int readerNumber, LPWSTR fileName);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
int readerNumber	The zero-based index of the card reader.
LPWSTR fileName	File name to store information

Returns

Returns TRUE if the information is retrieved from the card, otherwise returns FALSE

Description

Use this function to read the information about the owner of the card and save it to XML file.

1.1.295 SaveCardToXmlW Function

Read eID card and save the information to XML file

C++

```
BOOL WINAPI SaveCardToXmlW(LPWSTR fileName);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPWSTR fileName	File name to store information

Returns

Returns TRUE if the information is retrieved from the card, otherwise returns FALSE

Description

Use this function to read the information about the owner of the card and save it to XML file.

1.1.296 SaveContainer Function

Save container to the file

C++

```
BOOL WINAPI SaveContainer(LPVOID container, LPWSTR fileName);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPVOID container	Container handle, which is allocated by calling InitializeContainer (see page 83) function
LPWSTR fileName	Desired name of the container file

Returns

Returns true if the operation is successful, otherwise returns false

Description

After adding all necessary files to the container call this function to save container to the file

1.1.297 SaveIdentityA Function

Saves indentity information to a file

C++

```
void WINAPI SaveIdentityA(LPSTR fileName);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPSTR fileName	The name of the destination file

Description

Use this function to store the raw identity information from the Belgian eID card to a file. You can use LoadIdentityA (see page 101) to read this information from the file to EidIdentityA (see page 160) structure

1.1.298 SaveIdentityW Function

Saves indentity information to a file

C++

```
void WINAPI SaveIdentityW(LPWSTR fileName);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPWSTR fileName	The name of the destination file

Description

Use this function to store the raw identity information from the Belgian eID card to a file. You can use LoadIdentityW (see page 101) to read this information from the file to EidIdentityW (see page 165) structure

1.1.299 SaveNonRepudiationCertificateA Function

Save Non Repudiation Certificate to a file

C++

```
void WINAPI SaveNonRepudiationCertificateA(LPSTR fileName);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPSTR fileName	File name to store the certificate

Description

Read Non Repudiation Certificate from the card and save it to a file

1.1.300 SaveNonRepudiationCertificateExW Function

Save Non Repudiation Certificate to a file

C++

```
void WINAPI SaveNonRepudiationCertificateExW(LPWSTR fileName, int readerNumber);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPWSTR fileName	File name to store the certificate
int readerNumber	The reader index, starting from 0

Description

Read Non Repudiation Certificate from the card and save it to a file

1.1.301 SaveNonRepudiationCertificateW Function

Save Non Repudiation Certificate to a file

C++

```
void WINAPI SaveNonRepudiationCertificateW(LPWSTR fileName);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPWSTR fileName	File name to store the certificate

Description

Read Non Repudiation Certificate from the card and save it to a file

1.1.302 SavePersonCsvToStreamA Function

Read eID card and save the identity information to CSV memory buffer

C++

```
BOOL WINAPI SavePersonCsvToStreamA(int readerNumber, void* buffer);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
int readerNumber	the card reader index
void* buffer	Memory buffer

Returns

Returns TRUE if the information is retrieved from the card, otherwise returns FALSE

Description

Use this function to read the information about the owner of the card and save it to CSV memory buffer.

1.1.303 SavePersonCsvToStreamW Function

Read eID card and save the identity information to CSV memory buffer

C++

```
BOOL WINAPI SavePersonCsvToStreamW(int readerNumber, void* buffer);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
int readerNumber	the card reader index
void* buffer	Memory buffer

Returns

Returns TRUE if the information is retrieved from the card, otherwise returns FALSE

Description

Use this function to read the information about the owner of the card and save it to CSV memory buffer.

1.1.304 SavePersonToCsvA Function

Read eID card and save the identity information and address to CSV file

C++

```
BOOL WINAPI SavePersonToCsvA(LPSTR fileName);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPSTR fileName	File name to store information

Returns

Returns TRUE if the information is retrieved from the card, otherwise returns FALSE

Description

Use this function to read the information about the owner of the card and save it to CSV file.

1.1.305 SavePersonToCsvExA Function

Read eID card and save the identity information and address to CSV file

C++

```
BOOL WINAPI SavePersonToCsvExA(int readerNumber, LPSTR fileName);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
int readerNumber	The zero-based index of the card reader.
LPSTR fileName	File name to store information

Returns

Returns TRUE if the information is retrieved from the card, otherwise returns FALSE

Description

Use this function to read the information about the owner of the card and save it to CSV file.

1.1.306 SavePersonToCsvExW Function

Read eID card and save the identity information and address to CSV file

C++

```
BOOL WINAPI SavePersonToCsvExW(int readerNumber, LPWSTR fileName);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
int readerNumber	The zero-based index of the card reader.
LPWSTR fileName	File name to store information

Returns

Returns TRUE if the information is retrieved from the card, otherwise returns FALSE

Description

Use this function to read the information about the owner of the card and save it to CSV file.

1.1.307 SavePersonToCsvW Function

Read eID card and save the identity information and address to CSV file

C++

```
BOOL WINAPI SavePersonToCsvW(LPWSTR fileName);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPWSTR fileName	File name to store information

Returns

Returns TRUE if the information is retrieved from the card, otherwise returns FALSE

Description

Use this function to read the information about the owner of the card and save it to CSV file.

1.1.308 SavePhotoA Function

Save photo to a file

C++

```
void WINAPI SavePhotoA(PeidPicture photo, LPSTR fileName);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
PeidPicture photo	The pointer to EidPicture (see page 166) structure
LPSTR fileName	Destination file name

Description

Save the raw picture data to a file

1.1.309 SavePhotoAsBitmapA Function

Save the picture from the card to Windows Bitmap file Decription: Reads the photo from the Belgian eID card and writes it to the file as bitmap image. Reading the photo from the card is a time consuming operation. Do it only when needed.

C++

```
BOOL WINAPI SavePhotoAsBitmapA(LPSTR fileName);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPSTR fileName	File name to store the photo

Returns

Returns TRUE if the photo is retrieved from the card, otherwise return FALSE

1.1.310 SavePhotoAsBitmapExA Function

Reads the picture from the card and saves it to Windows Bitmap file Decription: Reads the photo from the Belgian eID card and writes it to the file as bitmap image. Reading the photo from the card is a time consuming operation. Do it only when needed.

C++

```
BOOL WINAPI SavePhotoAsBitmapExA(int readerNumber, LPSTR fileName);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
int readerNumber	The zero-based index of the card reader.
LPSTR fileName	File name to store the photo

Returns

Returns TRUE if the photo is retrieved from the card, otherwise return FALSE

1.1.311 SavePhotoAsBitmapExW Function

Reads the picture from the card and saves it to Windows Bitmap file Decription: Reads the photo from the Belgian eID card and writes it to the file as bitmap image. Reading the photo from the card is a time consuming operation. Do it only when needed.

C++

```
BOOL WINAPI SavePhotoAsBitmapExW(int readerNumber, LPWSTR fileName);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
int readerNumber	the zero-based index of the card reader.
LPWSTR fileName	File name to store the photo

Returns

Returns TRUE if the photo is retrieved from the card, otherwise return FALSE

1.1.312 SavePhotoAsBitmapW Function

Save the picture from the card to Windows Bitmap file Decription: Reads the photo from the Belgian eID card and writes it to the file as bitmap image. Reading the photo from the card is a time consuming operation. Do it only when needed.

C++

```
BOOL WINAPI SavePhotoAsBitmapW(LPWSTR fileName);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPWSTR fileName	File name to store the photo

Returns

Returns TRUE if the photo is retrieved from the card, otherwise return FALSE

1.1.313 SavePhotoAsJpegA Function

Save the picture from the card to JPG file Decription: Reads the photo from the Belgian eID card and writes it to the file as JPG image. Reading the photo from the card is a time consuming operation. Do it only when needed.

C++

```
BOOL WINAPI SavePhotoAsJpegA(LPSTR fileName);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPSTR fileName	File name to store the photo

Returns

Returns TRUE if the photo is retrieved from the card, otherwise return FALSE

1.1.314 SavePhotoAsJpegExA Function

Save the picture from the card to JPG file Decription: Reads the photo from the Belgian eID card and writes it to the file as JPG image. Reading the photo from the card is a time consuming operation. Do it only when needed.

C++

```
BOOL WINAPI SavePhotoAsJpegExA(int readerNumber, LPSTR fileName);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
int readerNumber	the zero-based index of the card reader.
LPSTR fileName	File name to store the photo

Returns

Returns TRUE if the photo is retrieved from the card, otherwise return FALSE

1.1.315 SavePhotoAsJpegExW Function

Save the picture from the card to JPG file Decription: Reads the photo from the Belgian eID card and writes it to the file as JPG image. Reading the photo from the card is a time consuming operation. Do it only when needed.

C++

```
BOOL WINAPI SavePhotoAsJpegExW(int readerNumber, LPWSTR fileName);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
int readerNumber	The zero-based index of the card reader.
LPWSTR fileName	File name to store the photo

Returns

Returns TRUE if the photo is retrieved from the card, otherwise return FALSE

1.1.316 SavePhotoAsJpegW Function

Save the picture from the card to JPG file Decription: Reads the photo from the Belgian eID card and writes it to the file as JPG image. Reading the photo from the card is a time consuming operation. Do it only when needed.

C++

```
BOOL WINAPI SavePhotoAsJpegW(LPWSTR fileName);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPWSTR fileName	File name to store the photo

Returns

Returns TRUE if the photo is retrieved from the card, otherwise return FALSE

1.1.317 SavePhotoW Function

Saves photo to a file

C++

```
void WINAPI SavePhotoW(PeidPicture photo, LPWSTR fileName);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
PeidPicture photo	The pointer to EidPicture (see page 166) structure
LPWSTR fileName	Destination file name

Description

Saves the raw picture data to a file

1.1.318 SaveRootCaCertificateA Function

Save Root Ca Certificate to a file

C++

```
void WINAPI SaveRootCaCertificateA(LPSTR fileName);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPSTR fileName	File name to store the certificate

Description

Read Root CA certificate from the card and save it to a file

1.1.319 SaveRootCaCertificateExW Function

Save Root Ca Certificate to a file

C++

```
void WINAPI SaveRootCaCertificateExW(LPWSTR fileName, int readerNumber);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPWSTR fileName	File name to store the certificate
int readerNumber	The reader index, starting from 0

Description

Read Root CA certificate from the card and save it to a file

1.1.320 SaveRootCaCertificateW Function

Save Root Ca Certificate to a file

C++

```
void WINAPI SaveRootCaCertificateW(LPWSTR fileName);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPWSTR fileName	File name to store the certificate

Description

Read Root CA certificate from the card and save it to a file

1.1.321 SaveRrnCertificateA Function

Save RRN Certificate to a file

C++

```
void WINAPI SaveRrnCertificateA(LPSTR fileName);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPSTR fileName	File name to store the certificate

Description

Read RRN certificate from the card and save it to a file

1.1.322 SaveRrnCertificateExW Function

Save RRN Certificate to a file

C++

```
void WINAPI SaveRrnCertificateExW(LPWSTR fileName, int readerNumber);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPWSTR fileName	File name to store the certificate
int readerNumber	The reader index, starting from 0

Description

Read RRN certificate from the card and save it to a file

1.1.323 SaveRrnCertificateW Function

Save RRN Certificate to a file

C++

```
void WINAPI SaveRrnCertificateW(LPWSTR fileName);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPWSTR fileName	File name to store the certificate

Description

Read RRN certificate from the card and save it to a file

1.1.324 SelectReader Function

When more than 1 reader connected, select the reader with specified number

C++

```
BOOL WINAPI SelectReader(int readerNumber);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
int readerNumber	The reader index, starting from 0

Returns

TRUE if the reader is selected, FALSE if the reader with specified number does not exist

Description

Selects the default card reader using the zero-based reader index. The first reader has number 0, second - 1, etc... You can read the information only from one selected reader at once.

1.1.325 SelectReaderByNameA Function

Select active smart card reader by providing the reader name

C++

```
BOOL WINAPI SelectReaderByNameA(LPSTR readerName);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPSTR readerName	The name of the card reader

Returns

TRUE if the reader is selected. If the reader with specified name is not found - returns FALSE

Description

Activates the reader with specified name

1.1.326 SelectReaderByNameW Function

Select active smart card reader by providing the reader name

C++

```
BOOL WINAPI SelectReaderByNameW(LPWSTR readerName);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPWSTR readerName	The name of the card reader

Returns

Returns TRUE if the reader is selected. If the reader with specified name is not found - returns FALSE

Description

Activates the reader with specified name

1.1.327 SendAPDU Function

This is function SendAPDU.

C++

```
BOOL WINAPI SendAPDU(int readerNumber, LPCBYTE apdu, DWORD apduLen, PCHAR result, LPDWORD
len);
```

File

File: Swelio.h (see page 183)

1.1.328 ServerAccessible Function

Checks if the server is accessible

C++

```
BOOL WINAPI ServerAccessible(LPWSTR name);
```

File

File: Network.h (see page 183)

Parameters

Parameters	Description
LPWSTR name	The server name or URL to check

Returns

Returns TRUE if the server is accessible, otherwise returns FALSE.

Description

This function checks if the server is accessible. It can be used to check if the server is reachable before trying to download the

certificate revocation list (CRL) or other resources from the server.

1.1.329 SetCallback Function

Activates callback procedure for card status change event

C++

```
void WINAPI SetCallback(CALLBACK_HANDLER callback, LPVOID userContext);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
CALLBACK_HANDLER callback	The pointer to callback procedure
LPVOID userContext	The user defined value passed to the callback procedure

Description

Your application can be notified about insertion or removal of the card from the card reader and the changes of the available card readers list (the reader is connected or disconnected from PC) Use this function to install the callback procedure

1.1.330 SetContainerCanonization Function

Set XML canonization standard of the ASIC container

C++

```
BOOL WINAPI SetContainerCanonization(LPVOID container, LPWSTR canonization);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPVOID container	Container handle, which is allocated by calling InitializeContainer (see page 83) function
LPWSTR canonization	XML canonization URI string

Returns

Returns true if the operation is successful, otherwise returns false

Description

Call this function to set the canonization string value of the ASIC container

1.1.331 SetContainerTimeServer Function

Set time server URI for digital signature of the ASIC container

C++

```
BOOL WINAPI SetContainerTimeServer(LPVOID container, LPWSTR timeServer);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPVOID container	Container handle, which is allocated by calling InitializeContainer (see page 83) function
canonization	the address of the time server

Returns

Returns true if the operation is successful, otherwise returns false

Description

Call this function to set the timeserver URI value

1.1.332 SetMWCompatibility Function

Set the compatibility mode with the old version of the oficial EID MiddleWare

C++

```
void WINAPI SetMWCompatibility();
```

File

File: Swelio.h (see page 183)

Description

The compatibility mode can be useful when the MiddleWare version 1.x or 2.x is installed on the target PC. Usually the more recent MiddleWare is used and this function is provided for backward compatibility only

1.1.333 SetStartupA Function

Register application to run when Windows starts

C++

```
void WINAPI SetStartupA(LPCSTR appName, LPCWSTR appPath);
```

File

File: System.h (see page 188)

Parameters

Parameters	Description
LPCSTR appName	The name of the application
LPCWSTR appPath	The path to the application executable

1.1.334 SetStartupW Function

Register application to run when Windows starts

C++

```
void WINAPI SetStartupW(LPCWSTR appName, LPCWSTR appPath);
```

File

File: System.h (see page 188)

Parameters

Parameters	Description
LPCWSTR appName	The name of the application
LPCWSTR appPath	The path to the application executable

1.1.335 SetSupportSIS Function

Activates or deactivates SIS card support by engine

C++

```
void WINAPI SetSupportSIS(BOOL value);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
BOOL value	The SIS card support status

Description

Use SetSupportSIS to activate or deactivate the SIS card detection and reading. Even if SIS card support is activated it can be used only with ACR38U card readers Other card readers are not supported.

1.1.336 SetTimeServer Function

Set time server URI for digital signature of the ASIC container

C++

```
BOOL WINAPI SetTimeServer(LPWSTR timeServer);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPWSTR timeServer	the address of the time server

Returns

Returns true if the operation is successful, otherwise returns false

Description

Call this function to set the timeserver URI value

1.1.337 ShellCopyFileA Function

Copies file to the new location

C++

```
void WINAPI ShellCopyFileA(LPSTR oldName, LPSTR newName);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
LPSTR oldName	The source file name
LPSTR newName	The destination file name

Description

Copies file to the new location using Windows shell copy routine

1.1.338 ShellCopyFileW Function

Copies file to the new location

C++

```
void WINAPI ShellCopyFileW(LPWSTR oldName, LPWSTR newName);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
LPWSTR oldName	The source file name
LPWSTR newName	The destination file name

Description

Copies file to the new location using Windows shell copy routine

1.1.339 ShutdownWindows Function

Logs off the interactive user, shuts down the system.

C++

```
BOOL WINAPI ShutdownWindows(UINT flags);
```

File

File: System.h (see page 188)

Returns

If the function succeeds returns TRUE, otherwise returns FALSE

Description

Logs off the interactive user, shuts down the system, or shuts down and restarts the system. It sends the WM_QUERYENDSESSION message to all applications to determine if they can be terminated.

This function accepts the following parameter:

flags : The shutdown type. This parameter must include one of the following values:

Value	Meaning
EWX_LOGOFF	Shuts down all processes running in the logon session of the process that called the ExitWindowsEx function. Then it logs the user off.
EWX_POWEROFF	Shuts down the system and turns off the power. The system must support the power-off feature.
EWX_REBOOT	Shuts down the system and then restarts the system.
EWX_RESTARTAPPS	Shuts down the system and then restarts it
EWX_SHUTDOWN	Shuts down the system to a point at which it is safe to turn off the power.

1.1.340 SignPdfFile Function

Digitally sign PDF file using Belgian EID card

C++

```
BOOL WINAPI SignPdfFile(int readerNumber, LPWSTR fileName);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPWSTR fileName	The name of the PDF file

Returns

Returns true if the operation is successful, otherwise returns false

Description

Sign PDF document using ID card. The card must be inserted in the reader before calling this function

1.1.341 SignPdfFileEx Function

Digitally sign PDF file using Belgian EID card

C++

```
BOOL WINAPI SignPdfFileEx(LPVOID container, LPWSTR fileName);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPVOID container	Pointer to container structrure which is allocated by calling InitializeContainer (see page 83) function
LPWSTR fileName	The name of the PDF file

Returns

Returns true if the operation is successful, otherwise returns false

Description

Sign PDF document using ID card. The card must be inserted in the reader before calling this function

1.1.342 StartEngine Function

Activates the Swelio Engine.

C++

```
BOOL WINAPI StartEngine();
```

File

File: Swelio.h (see page 183)

Returns

Returns TRUE if the Swelio Engine is successfully started; otherwize returns FALSE

Description

This procedure must be called first before any other functions from Swelio library can be used.

1.1.343 StopEngine Function

Deactivates the Swelio Engine

C++

```
void WINAPI StopEngine();
```

File

File: Swelio.h (see page 183)

Description

Deactivates the Swelio Engine and clean up the used memory. Call this procedure at the end of you application once to finalize the usage of the Swelio Engine.

1.1.344 StretchNativeBitmap Function

Stretches a native bitmap to fit the specified dimensions

C++

```
BOOL WINAPI StretchNativeBitmap(HBITMAP src, HBITMAP dst, int srcWidth, int srcHeight, int
dstWidth, int dstHeight);
```

File

File: Graphics.h (see page 182)

Parameters

Parameters	Description
HBITMAP src	Handle to the source bitmap
HBITMAP dst	Handle to the destination bitmap
int srcWidth	The width of the source bitmap
int srcHeight	The height of the source bitmap
int dstWidth	The width of the destination bitmap
int dstHeight	The height of the destination bitmap
Returns	TRUE if the operation was successful; otherwise, FALSE

1.1.345 StripFileNameA Function

Replaces environment variable names with values

C++

```
void WINAPI StripFileNameA(LPCSTR fileName, LPSTR fullName);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
LPCSTR fileName	The source file name
LPSTR fullName	The expanded file name

Description

This function expands environment-variable strings and replaces them with their defined values in the file name.

1.1.346 StripFileNameW Function

Replaces environment variable names with values

C++

```
void WINAPI StripFileNameW(LPCWSTR fileName, LPWSTR fullName);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
LPCWSTR fileName	The source file name
LPWSTR fullName	The expanded file name

Description

This function expands environment-variable strings and replaces them with their defined values in the file name.

1.1.347 SuspendWindows Function

Suspends Windows

C++

```
BOOL WINAPI SuspendWindows();
```

File

File: System.h (see page 188)

1.1.348 TurnMonitorOff Function

Turns the monitor off

C++

```
void WINAPI TurnMonitorOff();
```

File

File: System.h (see page 188)

1.1.349 TurnMonitorOn Function

Turns the monitor on

C++

```
void WINAPI TurnMonitorOn();
```

File

File: System.h (see page 188)

1.1.350 UpdateWindowPosition Function

Updated the window position

C++

```
void WINAPI UpdateWindowPosition(HWND handle, int x, int y);
```

File

File: System.h (see page 188)

Parameters

Parameters	Description
HWND handle	The handle of the window
int x	New horizontal coordinate
int y	New vertical coordinate

1.1.351 VerifyContainer Function

Verify the signatures and the integrity of the ASIC container

C++

```
BOOL WINAPI VerifyContainer(LPVOID container, LPWSTR fileName);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPVOID container	Container handle, which is allocated by calling InitializeContainer (see page 83) function
LPWSTR fileName	Container file name

Returns

Returns true if the operation is successful, otherwise returns false

Description

Call this function to verify the integrity of the ASIC container

1.1.352 VerifyPinA Function

Verify PIN code

C++

```
BOOL WINAPI VerifyPinA(LPSTR value);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPSTR value	PIN code to verify

Returns

TRUE when the correct PIN code is provided; otherwise returns FALSE

1.1.353 VerifyPinExA Function

Verify PIN code

C++

```
BOOL WINAPI VerifyPinExA(int readerNumber, LPSTR value);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
int readerNumber	The zero-based index of the card reader.
LPSTR value	PIN code to verify

Returns

TRUE when the correct PIN code is provided; otherwise returns FALSE

1.1.354 VerifyPinExW Function

Verify PIN code

C++

```
BOOL WINAPI VerifyPinExW(int readerNumber, LPWSTR value);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
int readerNumber	The zero-based index of the card reader.
LPWSTR value	PIN code to verify

Returns

TRUE when the correct PIN code is provided; otherwise returns FALSE

1.1.355 VerifyPinW Function

Verify PIN code

C++

```
BOOL WINAPI VerifyPinW(LPWSTR value);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
LPWSTR value	PIN code to verify

Returns

TRUE when the correct PIN code is provided; otherwise returns FALSE

1.1.356 VerifySignature Function

Verifies the signature from the specified hash value.

C++

```
BOOL WINAPI VerifySignature(PEidCertificate certificate, BYTE* buffer, int bufferSize,
```

```
BYTE* signature, DWORD signatureSize);
```

File

File: Swelio.h (see page 183)

Parameters

Parameters	Description
PEidCertificate certificate	The public certificate
BYTE* buffer	The hash buffer
int bufferSize	The size of the hash buffer
BYTE* signature	The signature to be verified.
DWORD signatureSize	The size of the signature buffer

Returns

Returns TRUE if the signature is valid for the hash; otherwise, FALSE.

Description

Verify the signature using the public certificate of the signer

1.1.357 WriteBufferToFileA Function

Writes the memory buffer to file

C++

```
void WINAPI WriteBufferToFileA(LPSTR fileName, BYTE* buffer, int bufferSize);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
LPSTR fileName	The name of the file
BYTE* buffer	The address of the memory block
int bufferSize	The size of the memory block

Description

This function stores the content of the memory buffer to the file.

1.1.358 WriteBufferToFileW Function

Writes the memory buffer to file

C++

```
void WINAPI WriteBufferToFileW(LPWSTR fileName, BYTE* buffer, int bufferSize);
```

File

File: FileOperations.h (see page 180)

Parameters

Parameters	Description
LPWSTR fileName	The name of the file

BYTE* buffer	The address of the memory block
int bufferSize	The size of the memory block

Description

This function stores the content of the memory buffer to the file.

1.1.359 fpreset Function

Resets the floating-point unit (FPU) to its default state.

C++

```
void fpreset();
```

File

File: System.h (see page 188)

Returns

This function does not return a value. It performs the reset operation on the FPU.

Description

This function resets the floating-point unit (FPU) to its default state, clearing any exceptions and restoring the control word to its default settings. This is useful in scenarios where floating-point operations may have altered the FPU state, and you want to ensure a clean state for subsequent calculations.

Notes

This function is platform-specific and may not be available on all systems. It is typically used in low-level programming or when interfacing with hardware that requires precise control over floating-point operations.

1.2 Structs, Records, Enums

The following table lists structs, records, enums in this documentation.

Enumerations

	Name	Description
	tagCardEventType (see page 150)	The type of the reader event
	CardEventType (see page 159)	The type of the reader event

Structures

	Name	Description
	structErrorInformation (see page 149)	Information about error when operation with the card or certificate is performed
	tagEidAddressA (see page 150)	EID address information, stored on the card - ANSI version
	tagEidAddressW (see page 150)	EID address information, stored on the card - UNICODE version
	tagEidCertificate (see page 151)	Certificate, stored on EID card
	tagEidIdentityA (see page 151)	Identity information stored on EID card - ANSI version
	tagEidIdentityExA (see page 153)	Identity information stored on EID card - ANSI version
	tagEidIdentityExW (see page 154)	Identity information stored on EID card - UNICODE version
	tagEidIdentityW (see page 156)	Identity information stored on EID card - UNICODE version

	tagEidPicture (see page 157)	Raw picture data from EID card
	tagSISRecordA (see page 157)	Public information stored on Belgian SIS card - ANSI version. The SIS card is the social security card of each Belgian resident (Belgian or foreigner)
	tagSISRecordW (see page 158)	Public information stored on Belgian SIS card - UNICODE version. The SIS card is the social security card of each Belgian resident (Belgian or foreigner)
	EidAddressA (see page 159)	EID address information, stored on the card - ANSI version
	EidAddressW (see page 160)	EID address information, stored on the card - UNICODE version
	EidCertificate (see page 160)	Certificate, stored on EID card
	EidIdentityA (see page 160)	Identity information stored on EID card - ANSI version
	EidIdentityExA (see page 162)	Identity information stored on EID card - ANSI version
	EidIdentityExW (see page 163)	Identity information stored on EID card - UNICODE version
	EidIdentityW (see page 165)	Identity information stored on EID card - UNICODE version
	EidPicture (see page 166)	Raw picture data from EID card
	ErrorInformation (see page 167)	Information about error when operation with the card or certificate is performed
	PEidAddressA (see page 167)	EID address information, stored on the card - ANSI version
	PEidAddressW (see page 167)	EID address information, stored on the card - UNICODE version
	PEidCertificate (see page 168)	Certificate, stored on EID card
	PEidIdentityA (see page 168)	Identity information stored on EID card - ANSI version
	PEidIdentityExA (see page 170)	Identity information stored on EID card - ANSI version
	PEidIdentityExW (see page 171)	Identity information stored on EID card - UNICODE version
	PEidIdentityW (see page 173)	Identity information stored on EID card - UNICODE version
	PErrorInformation (see page 174)	Information about error when operation with the card or certificate is performed
	PSISRecordA (see page 174)	Public information stored on Belgian SIS card - ANSI version. The SIS card is the social security card of each Belgian resident (Belgian or foreigner)
	PSISRecordW (see page 175)	Public information stored on Belgian SIS card - UNICODE version. The SIS card is the social security card of each Belgian resident (Belgian or foreigner)
	PeidPicture (see page 176)	Raw picture data from EID card
	SISRecordA (see page 176)	Public information stored on Belgian SIS card - ANSI version. The SIS card is the social security card of each Belgian resident (Belgian or foreigner)
	SISRecordW (see page 177)	Public information stored on Belgian SIS card - UNICODE version. The SIS card is the social security card of each Belgian resident (Belgian or foreigner)

1.2.1 structErrorInformation Structure

Information about error when operation with the card or certificate is performed

C++

```
struct structErrorInformation {
  int Code;
  WCHAR Description[ERROR_MAX_DESCRIPTION + 1];
};
```

File

File: CardStructures.h (see page 178)

1.2.2 tagCardEventType Enumeration

The type of the reader event

C++

```
enum tagCardEventType {
  ewtUnknownEvent,
  ewtCardInsert,
  ewtCardRemove,
  ewtReadersChange
};
```

File

File: CardEvents.h (see page 178)

Members

Members	Description
ewtUnknownEvent	Unknown event
ewtCardInsert	The card was inserted in the reader
ewtCardRemove	The card was removed from the reader
ewtReadersChange	The readers list changed

1.2.3 tagEidAddressA Structure

EID address information, stored on the card - ANSI version

C++

```
struct tagEidAddressA {
  char street[EID_MAX_STREET_LEN+1];
  char zip[EID_MAX_ZIP_LEN+1];
  char municipality[EID_MAX_MUNICIPALITY_LEN+1];
};
```

File

File: CardStructures.h (see page 178)

Members

Members	Description
char street[EID_MAX_STREET_LEN+1];	Street name
char zip[EID_MAX_ZIP_LEN+1];	ZIP code
char municipality[EID_MAX_MUNICIPALITY_LEN+1];	Municipality

1.2.4 tagEidAddressW Structure

EID address information, stored on the card - UNICODE version

C++

```
struct tagEidAddressW {
  WCHAR street[EID_MAX_STREET_LEN+1];
  WCHAR zip[EID_MAX_ZIP_LEN+1];
  WCHAR municipality[EID_MAX_MUNICIPALITY_LEN+1];
};
```

File

File: CardStructures.h (see page 178)

Members

Members	Description
WCHAR street[EID_MAX_STREET_LEN+1];	Street name
WCHAR zip[EID_MAX_ZIP_LEN+1];	ZIP code
WCHAR municipality[EID_MAX_MUNICIPALITY_LEN+1];	Municipality

1.2.5 tagEidCertificate Structure

Certificate, stored on EID card

C++

```
struct tagEidCertificate {
  BYTE certificate[EID_MAX_CERT_LEN+1];
  int certificateLength;
};
```

File

File: CardStructures.h (see page 178)

Members

Members	Description
BYTE certificate[EID_MAX_CERT_LEN+1];	Certificate raw data buffer
int certificateLength;	Certificate data length

1.2.6 tagEidIdentityA Structure

Identity information stored on EID card - ANSI version

C++

```
struct tagEidIdentityA {
  char cardNumber[EID_MAX_CARD_NUMBER_LEN+1];
  char chipNumber[EID_MAX_CHIP_NUMBER_LEN+1];
  char validityDateBegin[EID_MAX_DATE_BEGIN_LEN+1];
  char validityDateEnd[EID_MAX_DATE_END_LEN +1];
  char municipality[EID_MAX_DELIVERY_MUNICIPALITY_LEN+1];
  char nationalNumber[EID_MAX_NATIONAL_NUMBER_LEN+1];
  char name[EID_MAX_NAME_LEN+1];
  char firstName1[EID_MAX_FIRST_NAME1_LEN+1];
  char firstName2[EID_MAX_FIRST_NAME2_LEN+1];
  char nationality[EID_MAX_NATIONALITY_LEN+1];
  char birthLocation[EID_MAX_BIRTHPLACE_LEN+1];
  char birthDate[EID_MAX_BIRTHDATE_LEN+1];
  char sex[EID_MAX_SEX_LEN+1];
  char nobleCondition[EID_MAX_NOBLE_CONDITION_LEN+1];
  int documentType;
  BOOL whiteCane;
  BOOL yellowCane;
  BOOL extendedMinority;
  char duplicate[EID_MAX_DUPLICATE_LEN + 1];
  char specialOrganization[EID_MAX_SPECIALORGANIZATION_LEN + 1];
  BOOL memberOfFamily;
  char dateAndCountryOfProtection[EID_MAX_DATEANDCOUNTRYOFPROTECTION_LEN + 1];
  char workPermitType[EID_MAX_WORKPERMITTYPE_LEN + 1];
  char vat1[EID_MAX_VAT1_LEN + 1];
```

```
    char vat2[EID_MAX_VAT2_LEN + 1];
    char regionalFileNumber[EID_MAX_REGIONALFILENUMBER_LEN + 1];
    char brexitMention1[EID_MAX_BREXITMENTION1_LEN + 1];
    char brexitMention2[EID_MAX_BREXITMENTION2_LEN + 1];
  };
```

File

File: CardStructures.h (see page 178)

Members

Members	Description
char cardNumber[EID_MAX_CARD_NUMBER_LEN+1];	Electronic ID card number
char chipNumber[EID_MAX_CHIP_NUMBER_LEN+1];	Electronic ID card physical chip number
char validityDateBegin[EID_MAX_DATE_BEGIN_LEN+1];	Card validity start date
char validityDateEnd[EID_MAX_DATE_END_LEN +1];	Card validity end date
char municipality[EID_MAX_DELIVERY_MUNICIPALITY_LEN+1];	Card delivery municipality
char nationalNumber[EID_MAX_NATIONAL_NUMBER_LEN+1];	National number
char name[EID_MAX_NAME_LEN+1];	Surname
char firstName1[EID_MAX_FIRST_NAME1_LEN+1];	First name (2 first given names)
char firstName2[EID_MAX_FIRST_NAME2_LEN+1];	First name part 2 (first letter of the 3rd given name).
char nationality[EID_MAX_NATIONALITY_LEN+1];	Nationality
char birthLocation[EID_MAX_BIRTHPLACE_LEN+1];	Birth location
char birthDate[EID_MAX_BIRTHDATE_LEN+1];	Birth date
char sex[EID_MAX_SEX_LEN+1];	Sex
char nobleCondition[EID_MAX_NOBLE_CONDITION_LEN+1];	Noble condition
int documentType;	Document type code (Belgian citizen card, Kids Card, Foreigner card)
BOOL whiteCane;	White cane (blind people)
BOOL yellowCane;	Yellow cane (partially sighted people)
BOOL extendedMinority;	Extended minority
char duplicate[EID_MAX_DUPLICATE_LEN + 1];	Duplicata
char specialOrganization[EID_MAX_SPECIALORGANIZATION_LEN + 1];	Special Organization
BOOL memberOfFamily;	Member of family
char dateAndCountryOfProtection[EID_MAX_DATEANDCOUNTRYOFPROTECTION_LEN + 1];	Date and country of protection
char workPermitType[EID_MAX_WORKPERMITTYPE_LEN + 1];	Work Permit type
char vat1[EID_MAX_VAT1_LEN + 1];	Employer VAT1
char vat2[EID_MAX_VAT2_LEN + 1];	Employer VAT2
char regionalFileNumber[EID_MAX_REGIONALFILENUMBER_LEN + 1];	Regional file number
char brexitMention1[EID_MAX_BREXITMENTION1_LEN + 1];	BREXIT
char brexitMention2[EID_MAX_BREXITMENTION2_LEN + 1];	BREXIT

1.2.7 tagEidIdentityExA Structure

Identity information stored on EID card - ANSI version

C++

```
struct tagEidIdentityExA {
  WORD nSize;
  WORD nVersion;
  char cardNumber[EID_MAX_CARD_NUMBER_LEN + 1];
  char chipNumber[EID_MAX_CHIP_NUMBER_LEN + 1];
  char validityDateBegin[EID_MAX_DATE_BEGIN_LEN + 1];
  char validityDateEnd[EID_MAX_DATE_END_LEN + 1];
  char municipality[EID_MAX_DELIVERY_MUNICIPALITY_LEN + 1];
  char nationalNumber[EID_MAX_NATIONAL_NUMBER_LEN + 1];
  char name[EID_MAX_NAME_LEN + 1];
  char firstName1[EID_MAX_FIRST_NAME1_LEN + 1];
  char firstName2[EID_MAX_FIRST_NAME2_LEN + 1];
  char nationality[EID_MAX_NATIONALITY_LEN + 1];
  char birthLocation[EID_MAX_BIRTHPLACE_LEN + 1];
  char birthDate[EID_MAX_BIRTHDATE_LEN + 1];
  char sex[EID_MAX_SEX_LEN + 1];
  char nobleCondition[EID_MAX_NOBLE_CONDITION_LEN + 1];
  int documentType;
  BOOL whiteCane;
  BOOL yellowCane;
  BOOL extendedMinority;
  char duplicate[EID_MAX_DUPLICATE_LEN + 1];
  char specialOrganization[EID_MAX_SPECIALORGANIZATION_LEN + 1];
  BOOL memberOfFamily;
  char dateAndCountryOfProtection[EID_MAX_DATEANDCOUNTRYOFPROTECTION_LEN + 1];
  char workPermitType[EID_MAX_WORKPERMITTYPE_LEN + 1];
  char vat1[EID_MAX_VAT1_LEN + 1];
  char vat2[EID_MAX_VAT2_LEN + 1];
  char regionalFileNumber[EID_MAX_REGIONALFILENUMBER_LEN + 1];
  char brexitMention1[EID_MAX_BREXITMENTION1_LEN + 1];
  char brexitMention2[EID_MAX_BREXITMENTION2_LEN + 1];
};
```

File

File: CardStructures.h (see page 178)

Members

Members	Description
WORD nSize;	The size of the structure
WORD nVersion;	The version of the structure
char cardNumber[EID_MAX_CARD_NUMBER_LEN + 1];	Electronic ID card number
char chipNumber[EID_MAX_CHIP_NUMBER_LEN + 1];	Electronic ID card physical chip number
char validityDateBegin[EID_MAX_DATE_BEGIN_LEN + 1];	Card validity start date
char validityDateEnd[EID_MAX_DATE_END_LEN + 1];	Card validity end date
char municipality[EID_MAX_DELIVERY_MUNICIPALITY_LEN + 1];	Card delivery municipality
char nationalNumber[EID_MAX_NATIONAL_NUMBER_LEN + 1];	National number
char name[EID_MAX_NAME_LEN + 1];	Surname
char firstName1[EID_MAX_FIRST_NAME1_LEN + 1];	First name (2 first given names)
char firstName2[EID_MAX_FIRST_NAME2_LEN + 1];	First name part 2 (first letter of the 3rd given name).

char nationality[EID_MAX_NATIONALITY_LEN + 1];	Nationality
char birthLocation[EID_MAX_BIRTHPLACE_LEN + 1];	Birth location
char birthDate[EID_MAX_BIRTHDATE_LEN + 1];	Birth date
char sex[EID_MAX_SEX_LEN + 1];	Sex
char nobleCondition[EID_MAX_NOBLE_CONDITION_LEN + 1];	Noble condition
int documentType;	Document type code (Belgian citizen card, Kids Card, Foreigner card)
BOOL whiteCane;	White cane (blind people)
BOOL yellowCane;	Yellow cane (partially sighted people)
BOOL extendedMinority;	Extended minority
char duplicate[EID_MAX_DUPLICATE_LEN + 1];	Duplicata
char specialOrganization[EID_MAX_SPECIALORGANIZATION_LEN + 1];	Special Organization
BOOL memberOfFamily;	Member of family
char dateAndCountryOfProtection[EID_MAX_DATEANDCOUNTRYOFPROTECTION_LEN + 1];	Date and country of protection
char workPermitType[EID_MAX_WORKPERMITTYPE_LEN + 1];	Work Permit type
char vat1[EID_MAX_VAT1_LEN + 1];	Employer VAT1
char vat2[EID_MAX_VAT2_LEN + 1];	Employer VAT2
char regionalFileNumber[EID_MAX_REGIONALFILENUMBER_LEN + 1];	Regional file number
char brexitMention1[EID_MAX_BREXITMENTION1_LEN + 1];	BREXIT
char brexitMention2[EID_MAX_BREXITMENTION2_LEN + 1];	BREXIT

1.2.8 tagEidIdentityExW Structure

Identity information stored on EID card - UNICODE version

C++

```
struct tagEidIdentityExW {
  WORD nSize;
  WORD nVersion;
  WCHAR cardNumber[EID_MAX_CARD_NUMBER_LEN + 1];
  WCHAR chipNumber[EID_MAX_CHIP_NUMBER_LEN + 1];
  WCHAR validityDateBegin[EID_MAX_DATE_BEGIN_LEN + 1];
  WCHAR validityDateEnd[EID_MAX_DATE_END_LEN + 1];
  WCHAR municipality[EID_MAX_DELIVERY_MUNICIPALITY_LEN + 1];
  WCHAR nationalNumber[EID_MAX_NATIONAL_NUMBER_LEN + 1];
  WCHAR name[EID_MAX_NAME_LEN + 1];
  WCHAR firstName1[EID_MAX_FIRST_NAME1_LEN + 1];
  WCHAR firstName2[EID_MAX_FIRST_NAME2_LEN + 1];
  WCHAR nationality[EID_MAX_NATIONALITY_LEN + 1];
  WCHAR birthLocation[EID_MAX_BIRTHPLACE_LEN + 1];
  WCHAR birthDate[EID_MAX_BIRTHDATE_LEN + 1];
  WCHAR sex[EID_MAX_SEX_LEN + 1];
  WCHAR nobleCondition[EID_MAX_NOBLE_CONDITION_LEN + 1];
  int documentType;
  BOOL whiteCane;
  BOOL yellowCane;
  BOOL extendedMinority;
  WCHAR duplicate[EID_MAX_DUPLICATE_LEN + 1];
  WCHAR specialOrganization[EID_MAX_SPECIALORGANIZATION_LEN + 1];
  BOOL memberOfFamily;
```

```
    WCHAR dateAndCountryOfProtection[EID_MAX_DATEANDCOUNTRYOFPROTECTION_LEN + 1];
    WCHAR workPermitType[EID_MAX_WORKPERMITTYPE_LEN + 1];
    WCHAR vat1[EID_MAX_VAT1_LEN + 1];
    WCHAR vat2[EID_MAX_VAT2_LEN + 1];
    WCHAR regionalFileNumber[EID_MAX_REGIONALFILENUMBER_LEN + 1];
    WCHAR brexitMention1[EID_MAX_BREXITMENTION1_LEN + 1];
    WCHAR brexitMention2[EID_MAX_BREXITMENTION2_LEN + 1];
  };
```

File

File: CardStructures.h (see page 178)

Members

Members	Description
WORD nSize;	The size of the structure
WORD nVersion;	The version of the structure
WCHAR cardNumber[EID_MAX_CARD_NUMBER_LEN + 1];	Electronic ID card number
WCHAR chipNumber[EID_MAX_CHIP_NUMBER_LEN + 1];	Electronic ID card physical chip number
WCHAR validityDateBegin[EID_MAX_DATE_BEGIN_LEN + 1];	Card validity start date
WCHAR validityDateEnd[EID_MAX_DATE_END_LEN + 1];	Card validity end date
WCHAR municipality[EID_MAX_DELIVERY_MUNICIPALITY_LEN + 1];	Card delivery municipality
WCHAR nationalNumber[EID_MAX_NATIONAL_NUMBER_LEN + 1];	National number
WCHAR name[EID_MAX_NAME_LEN + 1];	Surname
WCHAR firstName1[EID_MAX_FIRST_NAME1_LEN + 1];	First name (2 first given names)
WCHAR firstName2[EID_MAX_FIRST_NAME2_LEN + 1];	First name part 2 (first letter of the 3rd given name).
WCHAR nationality[EID_MAX_NATIONALITY_LEN + 1];	Nationality
WCHAR birthLocation[EID_MAX_BIRTHPLACE_LEN + 1];	Birth location
WCHAR birthDate[EID_MAX_BIRTHDATE_LEN + 1];	Birth date
WCHAR sex[EID_MAX_SEX_LEN + 1];	Sex
WCHAR nobleCondition[EID_MAX_NOBLE_CONDITION_LEN + 1];	Noble condition
int documentType;	Document type code (Belgian citizen card, Kids Card, Foreigner card)
BOOL whiteCane;	White cane (blind people)
BOOL yellowCane;	Yellow cane (partially sighted people)
BOOL extendedMinority;	Extended minority
WCHAR duplicate[EID_MAX_DUPLICATE_LEN + 1];	Duplicata
WCHAR specialOrganization[EID_MAX_SPECIALORGANIZATION_LEN + 1];	Special Organization
BOOL memberOfFamily;	Member of family
WCHAR dateAndCountryOfProtection[EID_MAX_DATEANDCOUNTRYOFPROTECTION_LEN + 1];	Date and country of protection
WCHAR workPermitType[EID_MAX_WORKPERMITTYPE_LEN + 1];	Work permit type
WCHAR vat1[EID_MAX_VAT1_LEN + 1];	Employer VAT1
WCHAR vat2[EID_MAX_VAT2_LEN + 1];	Employer VAT2
WCHAR regionalFileNumber[EID_MAX_REGIONALFILENUMBER_LEN + 1];	Regional file number

WCHAR brexitMention1[EID_MAX_BREXITMENTION1_LEN + 1];	BREXIT
WCHAR brexitMention2[EID_MAX_BREXITMENTION2_LEN + 1];	BREXIT

1.2.9 tagEidIdentityW Structure

Identity information stored on EID card - UNICODE version

C++

```
struct tagEidIdentityW {
  WCHAR cardNumber[EID_MAX_CARD_NUMBER_LEN+1];
  WCHAR chipNumber[EID_MAX_CHIP_NUMBER_LEN+1];
  WCHAR validityDateBegin[EID_MAX_DATE_BEGIN_LEN+1];
  WCHAR validityDateEnd[EID_MAX_DATE_END_LEN +1];
  WCHAR municipality[EID_MAX_DELIVERY_MUNICIPALITY_LEN+1];
  WCHAR nationalNumber[EID_MAX_NATIONAL_NUMBER_LEN+1];
  WCHAR name[EID_MAX_NAME_LEN+1];
  WCHAR firstName1[EID_MAX_FIRST_NAME1_LEN+1];
  WCHAR firstName2[EID_MAX_FIRST_NAME2_LEN+1];
  WCHAR nationality[EID_MAX_NATIONALITY_LEN+1];
  WCHAR birthLocation[EID_MAX_BIRTHPLACE_LEN+1];
  WCHAR birthDate[EID_MAX_BIRTHDATE_LEN+1];
  WCHAR sex[EID_MAX_SEX_LEN+1];
  WCHAR nobleCondition[EID_MAX_NOBLE_CONDITION_LEN+1];
  int documentType;
  BOOL whiteCane;
  BOOL yellowCane;
  BOOL extendedMinority;
  WCHAR duplicate[EID_MAX_DUPLICATE_LEN + 1];
  WCHAR specialOrganization[EID_MAX_SPECIALORGANIZATION_LEN + 1];
  BOOL memberOfFamily;
  WCHAR dateAndCountryOfProtection[EID_MAX_DATEANDCOUNTRYOFPROTECTION_LEN + 1];
  WCHAR workPermitType[EID_MAX_WORKPERMITTYPE_LEN + 1];
  WCHAR vat1[EID_MAX_VAT1_LEN + 1];
  WCHAR vat2[EID_MAX_VAT2_LEN + 1];
  WCHAR regionalFileNumber[EID_MAX_REGIONALFILENUMBER_LEN + 1];
  WCHAR brexitMention1[EID_MAX_BREXITMENTION1_LEN + 1];
  WCHAR brexitMention2[EID_MAX_BREXITMENTION2_LEN + 1];
};
```

File

File: CardStructures.h (see page 178)

Members

Members	Description
WCHAR cardNumber[EID_MAX_CARD_NUMBER_LEN+1];	Electronic ID card number
WCHAR chipNumber[EID_MAX_CHIP_NUMBER_LEN+1];	Electronic ID card physical chip number
WCHAR validityDateBegin[EID_MAX_DATE_BEGIN_LEN+1];	Card validity start date
WCHAR validityDateEnd[EID_MAX_DATE_END_LEN +1];	Card validity end date
WCHAR municipality[EID_MAX_DELIVERY_MUNICIPALITY_LEN+1];	Card delivery municipality
WCHAR nationalNumber[EID_MAX_NATIONAL_NUMBER_LEN+1];	National number
WCHAR name[EID_MAX_NAME_LEN+1];	Surname
WCHAR firstName1[EID_MAX_FIRST_NAME1_LEN+1];	First name (2 first given names)
WCHAR firstName2[EID_MAX_FIRST_NAME2_LEN+1];	First name part 2 (first letter of the 3rd given name).
WCHAR nationality[EID_MAX_NATIONALITY_LEN+1];	Nationality

WCHAR birthLocation[EID_MAX_BIRTHPLACE_LEN+1];	Birth location
WCHAR birthDate[EID_MAX_BIRTHDATE_LEN+1];	Birth date
WCHAR sex[EID_MAX_SEX_LEN+1];	Sex
WCHAR nobleCondition[EID_MAX_NOBLE_CONDITION_LEN+1];	Noble condition
int documentType;	Document type code (Belgian citizen card, Kids Card, Foreigner card)
BOOL whiteCane;	White cane (blind people)
BOOL yellowCane;	Yellow cane (partially sighted people)
BOOL extendedMinority;	Extended minority
WCHAR duplicate[EID_MAX_DUPLICATE_LEN + 1];	Duplicata
WCHAR specialOrganization[EID_MAX_SPECIALORGANIZATION_LEN + 1];	Special Organization
BOOL memberOfFamily;	Member of family
WCHAR dateAndCountryOfProtection[EID_MAX_DATEANDCOUNTRYOFPROTECTION_LEN + 1];	Date and country of protection
WCHAR workPermitType[EID_MAX_WORKPERMITTYPE_LEN + 1];	Work permit type
WCHAR vat1[EID_MAX_VAT1_LEN + 1];	Employer VAT1
WCHAR vat2[EID_MAX_VAT2_LEN + 1];	Employer VAT2
WCHAR regionalFileNumber[EID_MAX_REGIONALFILENUMBER_LEN + 1];	Regional file number
WCHAR brexitMention1[EID_MAX_BREXITMENTION1_LEN + 1];	BREXIT
WCHAR brexitMention2[EID_MAX_BREXITMENTION2_LEN + 1];	BREXIT

1.2.10 tagEidPicture Structure

Raw picture data from EID card

C++

```
struct tagEidPicture {
  BYTE picture[EID_MAX_PICTURE_LEN+1];
  int pictureLength;
};
```

File

File: CardStructures.h (see page 178)

Members

Members	Description
BYTE picture[EID_MAX_PICTURE_LEN+1];	Picture raw data buffer
int pictureLength;	Picture raw data buffer length

1.2.11 tagSISRecordA Structure

Public information stored on Belgian SIS card - ANSI version. The SIS card is the social security card of each Belgian resident (Belgian or foreigner)

C++

```
struct tagSISRecordA {
  char Name[SIS_MAX_NAME_LEN + 1];
  char FirstName[SIS_MAX_FIRSTNAMES_LEN + 1];
  char Initial[SIS_MAX_INITIAL_LEN+ 1];
  char Sex[SIS_MAX_SEX_LEN + 1];
  char BirthDate[SIS_FIELD_MAX_BIRTHDATE_LEN + 1];
  char SocialSecurityNumber[SIS_FIELD_MAX_SOCIAL_SECURITY_NUMBER_LEN + 1];
  char CaptureDate[SIS_FIELD_MAX_CAPTUREDATE_LEN + 1];
  char ValidityDateBegin[SIS_FIELD_MAX_VALIDBEGIN_LEN + 1];
  char ValidityDateEnd[SIS_FIELD_MAX_VALIDEND_LEN + 1];
  char CardNumber[SIS_FIELD_MAX_CARDNUMBER_LEN + 1];
  char CardName[SIS_MAX_CARDNAME_LEN +1 ];
};
```

File

File: CardStructures.h (see page 178)

Members

Members	Description
char Name[SIS_MAX_NAME_LEN + 1];	Family name of the card owner
char FirstName[SIS_MAX_FIRSTNAMES_LEN + 1];	First name of the card owner
char Initial[SIS_MAX_INITIAL_LEN+ 1];	Initial of the card owner
char Sex[SIS_MAX_SEX_LEN + 1];	Sex of the card owner
char BirthDate[SIS_FIELD_MAX_BIRTHDATE_LEN + 1];	Birth date of the card owner
char SocialSecurityNumber[SIS_FIELD_MAX_SOCIAL_SECURITY_NUMBER_LEN + 1];	Social security number of the card owner
char CaptureDate[SIS_FIELD_MAX_CAPTUREDATE_LEN + 1];	Date of issue
char ValidityDateBegin[SIS_FIELD_MAX_VALIDBEGIN_LEN + 1];	Card validity begin date
char ValidityDateEnd[SIS_FIELD_MAX_VALIDEND_LEN + 1];	Card validity end date
char CardNumber[SIS_FIELD_MAX_CARDNUMBER_LEN + 1];	Card number
char CardName[SIS_MAX_CARDNAME_LEN +1];	Name of the card

1.2.12 tagSISRecordW Structure

Public information stored on Belgian SIS card - UNICODE version. The SIS card is the social security card of each Belgian resident (Belgian or foreigner)

C++

```
struct tagSISRecordW {
  WCHAR Name[SIS_MAX_NAME_LEN + 1];
  WCHAR FirstName[SIS_MAX_FIRSTNAMES_LEN + 1];
  WCHAR Initial[SIS_MAX_INITIAL_LEN+ 1];
  WCHAR Sex[SIS_MAX_SEX_LEN + 1];
  WCHAR BirthDate[SIS_FIELD_MAX_BIRTHDATE_LEN + 1];
  WCHAR SocialSecurityNumber[SIS_FIELD_MAX_SOCIAL_SECURITY_NUMBER_LEN + 1];
  WCHAR CaptureDate[SIS_FIELD_MAX_CAPTUREDATE_LEN + 1];
  WCHAR ValidityDateBegin[SIS_FIELD_MAX_VALIDBEGIN_LEN + 1];
  WCHAR ValidityDateEnd[SIS_FIELD_MAX_VALIDEND_LEN + 1];
  WCHAR CardNumber[SIS_FIELD_MAX_CARDNUMBER_LEN + 1];
  WCHAR CardName[SIS_MAX_CARDNAME_LEN +1 ];
};
```

File

File: CardStructures.h (see page 178)

Members

Members	Description
WCHAR Name[SIS_MAX_NAME_LEN + 1];	Family name of the card owner
WCHAR FirstName[SIS_MAX_FIRSTNAMES_LEN + 1];	First name of the card owner
WCHAR Initial[SIS_MAX_INITIAL_LEN+ 1];	Initial of the card owner
WCHAR Sex[SIS_MAX_SEX_LEN + 1];	Sex of the card owner
WCHAR BirthDate[SIS_FIELD_MAX_BIRTHDATE_LEN + 1];	Birth date of the card owner
WCHAR SocialSecurityNumber[SIS_FIELD_MAX_SOCIAL_SECURITY_NUMBER_LEN + 1];	Social security number of the card owner
WCHAR CaptureDate[SIS_FIELD_MAX_CAPTUREDATE_LEN + 1];	Date of issue
WCHAR ValidityDateBegin[SIS_FIELD_MAX_VALIDBEGIN_LEN + 1];	Card validity begin date
WCHAR ValidityDateEnd[SIS_FIELD_MAX_VALIDEND_LEN + 1];	Card validity end date
WCHAR CardNumber[SIS_FIELD_MAX_CARDNUMBER_LEN + 1];	Card number
WCHAR CardName[SIS_MAX_CARDNAME_LEN +1];	Name of the card

1.2.13 CardEventType Enumeration

The type of the reader event

C++

```
typedef enum tagCardEventType {
  ewtUnknownEvent,
  ewtCardInsert,
  ewtCardRemove,
  ewtReadersChange
} CardEventType;
```

File

File: CardEvents.h (see page 178)

Members

Members	Description
ewtUnknownEvent	Unknown event
ewtCardInsert	The card was inserted in the reader
ewtCardRemove	The card was removed from the reader
ewtReadersChange	The readers list changed

1.2.14 EidAddressA Structure

EID address information, stored on the card - ANSI version

C++

```
typedef struct tagEidAddressA {
  char street[EID_MAX_STREET_LEN+1];
  char zip[EID_MAX_ZIP_LEN+1];
  char municipality[EID_MAX_MUNICIPALITY_LEN+1];
} EidAddressA, * PEidAddressA;
```

File

File: CardStructures.h (see page 178)

Members

Members	Description
char street[EID_MAX_STREET_LEN+1];	Street name
char zip[EID_MAX_ZIP_LEN+1];	ZIP code
char municipality[EID_MAX_MUNICIPALITY_LEN+1];	Municipality

1.2.15 EidAddressW Structure

EID address information, stored on the card - UNICODE version

C++

```
typedef struct tagEidAddressW {
  WCHAR street[EID_MAX_STREET_LEN+1];
  WCHAR zip[EID_MAX_ZIP_LEN+1];
  WCHAR municipality[EID_MAX_MUNICIPALITY_LEN+1];
} EidAddressW, * PEidAddressW;
```

File

File: CardStructures.h (see page 178)

Members

Members	Description
WCHAR street[EID_MAX_STREET_LEN+1];	Street name
WCHAR zip[EID_MAX_ZIP_LEN+1];	ZIP code
WCHAR municipality[EID_MAX_MUNICIPALITY_LEN+1];	Municipality

1.2.16 EidCertificate Structure

Certificate, stored on EID card

C++

```
typedef struct tagEidCertificate {
  BYTE certificate[EID_MAX_CERT_LEN+1];
  int certificateLength;
} EidCertificate, * PEidCertificate;
```

File

File: CardStructures.h (see page 178)

Members

Members	Description
BYTE certificate[EID_MAX_CERT_LEN+1];	Certificate raw data buffer
int certificateLength;	Certificate data length

1.2.17 EidIdentityA Structure

Identity information stored on EID card - ANSI version

C++

```
typedef struct tagEidIdentityA {
```

```
    char cardNumber[EID_MAX_CARD_NUMBER_LEN+1];
    char chipNumber[EID_MAX_CHIP_NUMBER_LEN+1];
    char validityDateBegin[EID_MAX_DATE_BEGIN_LEN+1];
    char validityDateEnd[EID_MAX_DATE_END_LEN +1];
    char municipality[EID_MAX_DELIVERY_MUNICIPALITY_LEN+1];
    char nationalNumber[EID_MAX_NATIONAL_NUMBER_LEN+1];
    char name[EID_MAX_NAME_LEN+1];
    char firstName1[EID_MAX_FIRST_NAME1_LEN+1];
    char firstName2[EID_MAX_FIRST_NAME2_LEN+1];
    char nationality[EID_MAX_NATIONALITY_LEN+1];
    char birthLocation[EID_MAX_BIRTHPLACE_LEN+1];
    char birthDate[EID_MAX_BIRTHDATE_LEN+1];
    char sex[EID_MAX_SEX_LEN+1];
    char nobleCondition[EID_MAX_NOBLE_CONDITION_LEN+1];
    int documentType;
    BOOL whiteCane;
    BOOL yellowCane;
    BOOL extendedMinority;
    char duplicate[EID_MAX_DUPLICATE_LEN + 1];
    char specialOrganization[EID_MAX_SPECIALORGANIZATION_LEN + 1];
    BOOL memberOfFamily;
    char dateAndCountryOfProtection[EID_MAX_DATEANDCOUNTRYOFPROTECTION_LEN + 1];
    char workPermitType[EID_MAX_WORKPERMITTYPE_LEN + 1];
    char vat1[EID_MAX_VAT1_LEN + 1];
    char vat2[EID_MAX_VAT2_LEN + 1];
    char regionalFileNumber[EID_MAX_REGIONALFILENUMBER_LEN + 1];
    char brexitMention1[EID_MAX_BREXITMENTION1_LEN + 1];
    char brexitMention2[EID_MAX_BREXITMENTION2_LEN + 1];
} EidIdentityA, * PEidIdentityA;
```

File

File: CardStructures.h (see page 178)

Members

Members	Description
char cardNumber[EID_MAX_CARD_NUMBER_LEN+1];	Electronic ID card number
char chipNumber[EID_MAX_CHIP_NUMBER_LEN+1];	Electronic ID card physical chip number
char validityDateBegin[EID_MAX_DATE_BEGIN_LEN+1];	Card validity start date
char validityDateEnd[EID_MAX_DATE_END_LEN +1];	Card validity end date
char municipality[EID_MAX_DELIVERY_MUNICIPALITY_LEN+1];	Card delivery municipality
char nationalNumber[EID_MAX_NATIONAL_NUMBER_LEN+1];	National number
char name[EID_MAX_NAME_LEN+1];	Surname
char firstName1[EID_MAX_FIRST_NAME1_LEN+1];	First name (2 first given names)
char firstName2[EID_MAX_FIRST_NAME2_LEN+1];	First name part 2 (first letter of the 3rd given name).
char nationality[EID_MAX_NATIONALITY_LEN+1];	Nationality
char birthLocation[EID_MAX_BIRTHPLACE_LEN+1];	Birth location
char birthDate[EID_MAX_BIRTHDATE_LEN+1];	Birth date
char sex[EID_MAX_SEX_LEN+1];	Sex
char nobleCondition[EID_MAX_NOBLE_CONDITION_LEN+1];	Noble condition
int documentType;	Document type code (Belgian citizen card, Kids Card, Foreigner card)
BOOL whiteCane;	White cane (blind people)
BOOL yellowCane;	Yellow cane (partially sighted people)
BOOL extendedMinority;	Extended minority

char duplicate[EID_MAX_DUPLICATE_LEN + 1];	Duplicata
char specialOrganization[EID_MAX_SPECIALORGANIZATION_LEN + 1];	Special Organization
BOOL memberOfFamily;	Member of family
char dateAndCountryOfProtection[EID_MAX_DATEANDCOUNTRYOFPROTECTION_LEN + 1];	Date and country of protection
char workPermitType[EID_MAX_WORKPERMITTYPE_LEN + 1];	Work Permit type
char vat1[EID_MAX_VAT1_LEN + 1];	Employer VAT1
char vat2[EID_MAX_VAT2_LEN + 1];	Employer VAT2
char regionalFileNumber[EID_MAX_REGIONALFILENUMBER_LEN + 1];	Regional file number
char brexitMention1[EID_MAX_BREXITMENTION1_LEN + 1];	BREXIT
char brexitMention2[EID_MAX_BREXITMENTION2_LEN + 1];	BREXIT

1.2.18 EidIdentityExA Structure

Identity information stored on EID card - ANSI version

C++

```
typedef struct tagEidIdentityExA {
  WORD nSize;
  WORD nVersion;
  char cardNumber[EID_MAX_CARD_NUMBER_LEN + 1];
  char chipNumber[EID_MAX_CHIP_NUMBER_LEN + 1];
  char validityDateBegin[EID_MAX_DATE_BEGIN_LEN + 1];
  char validityDateEnd[EID_MAX_DATE_END_LEN + 1];
  char municipality[EID_MAX_DELIVERY_MUNICIPALITY_LEN + 1];
  char nationalNumber[EID_MAX_NATIONAL_NUMBER_LEN + 1];
  char name[EID_MAX_NAME_LEN + 1];
  char firstName1[EID_MAX_FIRST_NAME1_LEN + 1];
  char firstName2[EID_MAX_FIRST_NAME2_LEN + 1];
  char nationality[EID_MAX_NATIONALITY_LEN + 1];
  char birthLocation[EID_MAX_BIRTHPLACE_LEN + 1];
  char birthDate[EID_MAX_BIRTHDATE_LEN + 1];
  char sex[EID_MAX_SEX_LEN + 1];
  char nobleCondition[EID_MAX_NOBLE_CONDITION_LEN + 1];
  int documentType;
  BOOL whiteCane;
  BOOL yellowCane;
  BOOL extendedMinority;
  char duplicate[EID_MAX_DUPLICATE_LEN + 1];
  char specialOrganization[EID_MAX_SPECIALORGANIZATION_LEN + 1];
  BOOL memberOfFamily;
  char dateAndCountryOfProtection[EID_MAX_DATEANDCOUNTRYOFPROTECTION_LEN + 1];
  char workPermitType[EID_MAX_WORKPERMITTYPE_LEN + 1];
  char vat1[EID_MAX_VAT1_LEN + 1];
  char vat2[EID_MAX_VAT2_LEN + 1];
  char regionalFileNumber[EID_MAX_REGIONALFILENUMBER_LEN + 1];
  char brexitMention1[EID_MAX_BREXITMENTION1_LEN + 1];
  char brexitMention2[EID_MAX_BREXITMENTION2_LEN + 1];
} EidIdentityExA, * PEidIdentityExA;
```

File

File: CardStructures.h (see page 178)

Members

Members	Description
WORD nSize;	The size of the structure
WORD nVersion;	The version of the structure

char cardNumber[EID_MAX_CARD_NUMBER_LEN + 1];	Electronic ID card number
char chipNumber[EID_MAX_CHIP_NUMBER_LEN + 1];	Electronic ID card physical chip number
char validityDateBegin[EID_MAX_DATE_BEGIN_LEN + 1];	Card validity start date
char validityDateEnd[EID_MAX_DATE_END_LEN + 1];	Card validity end date
char municipality[EID_MAX_DELIVERY_MUNICIPALITY_LEN + 1];	Card delivery municipality
char nationalNumber[EID_MAX_NATIONAL_NUMBER_LEN + 1];	National number
char name[EID_MAX_NAME_LEN + 1];	Surname
char firstName1[EID_MAX_FIRST_NAME1_LEN + 1];	First name (2 first given names)
char firstName2[EID_MAX_FIRST_NAME2_LEN + 1];	First name part 2 (first letter of the 3rd given name).
char nationality[EID_MAX_NATIONALITY_LEN + 1];	Nationality
char birthLocation[EID_MAX_BIRTHPLACE_LEN + 1];	Birth location
char birthDate[EID_MAX_BIRTHDATE_LEN + 1];	Birth date
char sex[EID_MAX_SEX_LEN + 1];	Sex
char nobleCondition[EID_MAX_NOBLE_CONDITION_LEN + 1];	Noble condition
int documentType;	Document type code (Belgian citizen card, Kids Card, Foreigner card)
BOOL whiteCane;	White cane (blind people)
BOOL yellowCane;	Yellow cane (partially sighted people)
BOOL extendedMinority;	Extended minority
char duplicate[EID_MAX_DUPLICATE_LEN + 1];	Duplicata
char specialOrganization[EID_MAX_SPECIALORGANIZATION_LEN + 1];	Special Organization
BOOL memberOfFamily;	Member of family
char dateAndCountryOfProtection[EID_MAX_DATEANDCOUNTRYOFPROTECTION_LEN + 1];	Date and country of protection
char workPermitType[EID_MAX_WORKPERMITTYPE_LEN + 1];	Work Permit type
char vat1[EID_MAX_VAT1_LEN + 1];	Employer VAT1
char vat2[EID_MAX_VAT2_LEN + 1];	Employer VAT2
char regionalFileNumber[EID_MAX_REGIONALFILENUMBER_LEN + 1];	Regional file number
char brexitMention1[EID_MAX_BREXITMENTION1_LEN + 1];	BREXIT
char brexitMention2[EID_MAX_BREXITMENTION2_LEN + 1];	BREXIT

1.2.19 EidIdentityExW Structure

Identity information stored on EID card - UNICODE version

C++

```
typedef struct tagEidIdentityExW {
  WORD nSize;
  WORD nVersion;
  WCHAR cardNumber[EID_MAX_CARD_NUMBER_LEN + 1];
  WCHAR chipNumber[EID_MAX_CHIP_NUMBER_LEN + 1];
  WCHAR validityDateBegin[EID_MAX_DATE_BEGIN_LEN + 1];
```

```
    WCHAR validityDateEnd[EID_MAX_DATE_END_LEN + 1];
    WCHAR municipality[EID_MAX_DELIVERY_MUNICIPALITY_LEN + 1];
    WCHAR nationalNumber[EID_MAX_NATIONAL_NUMBER_LEN + 1];
    WCHAR name[EID_MAX_NAME_LEN + 1];
    WCHAR firstName1[EID_MAX_FIRST_NAME1_LEN + 1];
    WCHAR firstName2[EID_MAX_FIRST_NAME2_LEN + 1];
    WCHAR nationality[EID_MAX_NATIONALITY_LEN + 1];
    WCHAR birthLocation[EID_MAX_BIRTHPLACE_LEN + 1];
    WCHAR birthDate[EID_MAX_BIRTHDATE_LEN + 1];
    WCHAR sex[EID_MAX_SEX_LEN + 1];
    WCHAR nobleCondition[EID_MAX_NOBLE_CONDITION_LEN + 1];
    int documentType;
    BOOL whiteCane;
    BOOL yellowCane;
    BOOL extendedMinority;
    WCHAR duplicate[EID_MAX_DUPLICATE_LEN + 1];
    WCHAR specialOrganization[EID_MAX_SPECIALORGANIZATION_LEN + 1];
    BOOL memberOfFamily;
    WCHAR dateAndCountryOfProtection[EID_MAX_DATEANDCOUNTRYOFPROTECTION_LEN + 1];
    WCHAR workPermitType[EID_MAX_WORKPERMITTYPE_LEN + 1];
    WCHAR vat1[EID_MAX_VAT1_LEN + 1];
    WCHAR vat2[EID_MAX_VAT2_LEN + 1];
    WCHAR regionalFileNumber[EID_MAX_REGIONALFILENUMBER_LEN + 1];
    WCHAR brexitMention1[EID_MAX_BREXITMENTION1_LEN + 1];
    WCHAR brexitMention2[EID_MAX_BREXITMENTION2_LEN + 1];
} EidIdentityExW, * PEidIdentityExW;
```

File

File: CardStructures.h (see page 178)

Members

Members	Description
WORD nSize;	The size of the structure
WORD nVersion;	The version of the structure
WCHAR cardNumber[EID_MAX_CARD_NUMBER_LEN + 1];	Electronic ID card number
WCHAR chipNumber[EID_MAX_CHIP_NUMBER_LEN + 1];	Electronic ID card physical chip number
WCHAR validityDateBegin[EID_MAX_DATE_BEGIN_LEN + 1];	Card validity start date
WCHAR validityDateEnd[EID_MAX_DATE_END_LEN + 1];	Card validity end date
WCHAR municipality[EID_MAX_DELIVERY_MUNICIPALITY_LEN + 1];	Card delivery municipality
WCHAR nationalNumber[EID_MAX_NATIONAL_NUMBER_LEN + 1];	National number
WCHAR name[EID_MAX_NAME_LEN + 1];	Surname
WCHAR firstName1[EID_MAX_FIRST_NAME1_LEN + 1];	First name (2 first given names)
WCHAR firstName2[EID_MAX_FIRST_NAME2_LEN + 1];	First name part 2 (first letter of the 3rd given name).
WCHAR nationality[EID_MAX_NATIONALITY_LEN + 1];	Nationality
WCHAR birthLocation[EID_MAX_BIRTHPLACE_LEN + 1];	Birth location
WCHAR birthDate[EID_MAX_BIRTHDATE_LEN + 1];	Birth date
WCHAR sex[EID_MAX_SEX_LEN + 1];	Sex
WCHAR nobleCondition[EID_MAX_NOBLE_CONDITION_LEN + 1];	Noble condition
int documentType;	Document type code (Belgian citizen card, Kids Card, Foreigner card)
BOOL whiteCane;	White cane (blind people)

BOOL yellowCane;	Yellow cane (partially sighted people)
BOOL extendedMinority;	Extended minority
WCHAR duplicate[EID_MAX_DUPLICATE_LEN + 1];	Duplicata
WCHAR specialOrganization[EID_MAX_SPECIALORGANIZATION_LEN + 1];	Special Organization
BOOL memberOfFamily;	Member of family
WCHAR dateAndCountryOfProtection[EID_MAX_DATEANDCOUNTRYOFPROTECTION_LEN + 1];	Date and country of protection
WCHAR workPermitType[EID_MAX_WORKPERMITTYPE_LEN + 1];	Work permit type
WCHAR vat1[EID_MAX_VAT1_LEN + 1];	Employer VAT1
WCHAR vat2[EID_MAX_VAT2_LEN + 1];	Employer VAT2
WCHAR regionalFileNumber[EID_MAX_REGIONALFILENUMBER_LEN + 1];	Regional file number
WCHAR brexitMention1[EID_MAX_BREXITMENTION1_LEN + 1];	BREXIT
WCHAR brexitMention2[EID_MAX_BREXITMENTION2_LEN + 1];	BREXIT

1.2.20 EidIdentityW Structure

Identity information stored on EID card - UNICODE version

C++

```
typedef struct tagEidIdentityW {
  WCHAR cardNumber[EID_MAX_CARD_NUMBER_LEN+1];
  WCHAR chipNumber[EID_MAX_CHIP_NUMBER_LEN+1];
  WCHAR validityDateBegin[EID_MAX_DATE_BEGIN_LEN+1];
  WCHAR validityDateEnd[EID_MAX_DATE_END_LEN +1];
  WCHAR municipality[EID_MAX_DELIVERY_MUNICIPALITY_LEN+1];
  WCHAR nationalNumber[EID_MAX_NATIONAL_NUMBER_LEN+1];
  WCHAR name[EID_MAX_NAME_LEN+1];
  WCHAR firstName1[EID_MAX_FIRST_NAME1_LEN+1];
  WCHAR firstName2[EID_MAX_FIRST_NAME2_LEN+1];
  WCHAR nationality[EID_MAX_NATIONALITY_LEN+1];
  WCHAR birthLocation[EID_MAX_BIRTHPLACE_LEN+1];
  WCHAR birthDate[EID_MAX_BIRTHDATE_LEN+1];
  WCHAR sex[EID_MAX_SEX_LEN+1];
  WCHAR nobleCondition[EID_MAX_NOBLE_CONDITION_LEN+1];
  int documentType;
  BOOL whiteCane;
  BOOL yellowCane;
  BOOL extendedMinority;
  WCHAR duplicate[EID_MAX_DUPLICATE_LEN + 1];
  WCHAR specialOrganization[EID_MAX_SPECIALORGANIZATION_LEN + 1];
  BOOL memberOfFamily;
  WCHAR dateAndCountryOfProtection[EID_MAX_DATEANDCOUNTRYOFPROTECTION_LEN + 1];
  WCHAR workPermitType[EID_MAX_WORKPERMITTYPE_LEN + 1];
  WCHAR vat1[EID_MAX_VAT1_LEN + 1];
  WCHAR vat2[EID_MAX_VAT2_LEN + 1];
  WCHAR regionalFileNumber[EID_MAX_REGIONALFILENUMBER_LEN + 1];
  WCHAR brexitMention1[EID_MAX_BREXITMENTION1_LEN + 1];
  WCHAR brexitMention2[EID_MAX_BREXITMENTION2_LEN + 1];
} EidIdentityW, * PEidIdentityW;
```

File

File: CardStructures.h (see page 178)

Members

Members	Description
WCHAR cardNumber[EID_MAX_CARD_NUMBER_LEN+1];	Electronic ID card number
WCHAR chipNumber[EID_MAX_CHIP_NUMBER_LEN+1];	Electronic ID card physical chip number
WCHAR validityDateBegin[EID_MAX_DATE_BEGIN_LEN+1];	Card validity start date
WCHAR validityDateEnd[EID_MAX_DATE_END_LEN +1];	Card validity end date
WCHAR municipality[EID_MAX_DELIVERY_MUNICIPALITY_LEN+1];	Card delivery municipality
WCHAR nationalNumber[EID_MAX_NATIONAL_NUMBER_LEN+1];	National number
WCHAR name[EID_MAX_NAME_LEN+1];	Surname
WCHAR firstName1[EID_MAX_FIRST_NAME1_LEN+1];	First name (2 first given names)
WCHAR firstName2[EID_MAX_FIRST_NAME2_LEN+1];	First name part 2 (first letter of the 3rd given name).
WCHAR nationality[EID_MAX_NATIONALITY_LEN+1];	Nationality
WCHAR birthLocation[EID_MAX_BIRTHPLACE_LEN+1];	Birth location
WCHAR birthDate[EID_MAX_BIRTHDATE_LEN+1];	Birth date
WCHAR sex[EID_MAX_SEX_LEN+1];	Sex
WCHAR nobleCondition[EID_MAX_NOBLE_CONDITION_LEN+1];	Noble condition
int documentType;	Document type code (Belgian citizen card, Kids Card, Foreigner card)
BOOL whiteCane;	White cane (blind people)
BOOL yellowCane;	Yellow cane (partially sighted people)
BOOL extendedMinority;	Extended minority
WCHAR duplicate[EID_MAX_DUPLICATE_LEN + 1];	Duplicata
WCHAR specialOrganization[EID_MAX_SPECIALORGANIZATION_LEN + 1];	Special Organization
BOOL memberOfFamily;	Member of family
WCHAR dateAndCountryOfProtection[EID_MAX_DATEANDCOUNTRYOFPROTECTION_LEN + 1];	Date and country of protection
WCHAR workPermitType[EID_MAX_WORKPERMITTYPE_LEN + 1];	Work permit type
WCHAR vat1[EID_MAX_VAT1_LEN + 1];	Employer VAT1
WCHAR vat2[EID_MAX_VAT2_LEN + 1];	Employer VAT2
WCHAR regionalFileNumber[EID_MAX_REGIONALFILENUMBER_LEN + 1];	Regional file number
WCHAR brexitMention1[EID_MAX_BREXITMENTION1_LEN + 1];	BREXIT
WCHAR brexitMention2[EID_MAX_BREXITMENTION2_LEN + 1];	BREXIT

1.2.21 EidPicture Structure

Raw picture data from EID card

C++

```
typedef struct tagEidPicture {
  BYTE picture[EID_MAX_PICTURE_LEN+1];
```

```
  int pictureLength;
} EidPicture, * PeidPicture;
```

File

File: CardStructures.h (see page 178)

Members

Members	Description
BYTE picture[EID_MAX_PICTURE_LEN+1];	Picture raw data buffer
int pictureLength;	Picture raw data buffer length

1.2.22 ErrorInformation Structure

Information about error when operation with the card or certificate is performed

C++

```
typedef struct structErrorInformation {
  int Code;
  WCHAR Description[ERROR_MAX_DESCRIPTION + 1];
} ErrorInformation, * PErrorInformation;
```

File

File: CardStructures.h (see page 178)

1.2.23 PEidAddressA Structure

EID address information, stored on the card - ANSI version

C++

```
typedef struct tagEidAddressA {
  char street[EID_MAX_STREET_LEN+1];
  char zip[EID_MAX_ZIP_LEN+1];
  char municipality[EID_MAX_MUNICIPALITY_LEN+1];
} EidAddressA, * PEidAddressA;
```

File

File: CardStructures.h (see page 178)

Members

Members	Description
char street[EID_MAX_STREET_LEN+1];	Street name
char zip[EID_MAX_ZIP_LEN+1];	ZIP code
char municipality[EID_MAX_MUNICIPALITY_LEN+1];	Municipality

1.2.24 PEidAddressW Structure

EID address information, stored on the card - UNICODE version

C++

```
typedef struct tagEidAddressW {
  WCHAR street[EID_MAX_STREET_LEN+1];
  WCHAR zip[EID_MAX_ZIP_LEN+1];
```

```
  WCHAR municipality[EID_MAX_MUNICIPALITY_LEN+1];
} EidAddressW, * PEidAddressW;
```

File

File: CardStructures.h (see page 178)

Members

Members	Description
WCHAR street[EID_MAX_STREET_LEN+1];	Street name
WCHAR zip[EID_MAX_ZIP_LEN+1];	ZIP code
WCHAR municipality[EID_MAX_MUNICIPALITY_LEN+1];	Municipality

1.2.25 PEidCertificate Structure

Certificate, stored on EID card

C++

```
typedef struct tagEidCertificate {
  BYTE certificate[EID_MAX_CERT_LEN+1];
  int certificateLength;
} EidCertificate, * PEidCertificate;
```

File

File: CardStructures.h (see page 178)

Members

Members	Description
BYTE certificate[EID_MAX_CERT_LEN+1];	Certificate raw data buffer
int certificateLength;	Certificate data length

1.2.26 PEidIdentityA Structure

Identity information stored on EID card - ANSI version

C++

```
typedef struct tagEidIdentityA {
  char cardNumber[EID_MAX_CARD_NUMBER_LEN+1];
  char chipNumber[EID_MAX_CHIP_NUMBER_LEN+1];
  char validityDateBegin[EID_MAX_DATE_BEGIN_LEN+1];
  char validityDateEnd[EID_MAX_DATE_END_LEN +1];
  char municipality[EID_MAX_DELIVERY_MUNICIPALITY_LEN+1];
  char nationalNumber[EID_MAX_NATIONAL_NUMBER_LEN+1];
  char name[EID_MAX_NAME_LEN+1];
  char firstName1[EID_MAX_FIRST_NAME1_LEN+1];
  char firstName2[EID_MAX_FIRST_NAME2_LEN+1];
  char nationality[EID_MAX_NATIONALITY_LEN+1];
  char birthLocation[EID_MAX_BIRTHPLACE_LEN+1];
  char birthDate[EID_MAX_BIRTHDATE_LEN+1];
  char sex[EID_MAX_SEX_LEN+1];
  char nobleCondition[EID_MAX_NOBLE_CONDITION_LEN+1];
  int documentType;
  BOOL whiteCane;
  BOOL yellowCane;
  BOOL extendedMinority;
  char duplicate[EID_MAX_DUPLICATE_LEN + 1];
  char specialOrganization[EID_MAX_SPECIALORGANIZATION_LEN + 1];
  BOOL memberOfFamily;
  char dateAndCountryOfProtection[EID_MAX_DATEANDCOUNTRYOFPROTECTION_LEN + 1];
```

```
    char workPermitType[EID_MAX_WORKPERMITTYPE_LEN + 1];
    char vat1[EID_MAX_VAT1_LEN + 1];
    char vat2[EID_MAX_VAT2_LEN + 1];
    char regionalFileNumber[EID_MAX_REGIONALFILENUMBER_LEN + 1];
    char brexitMention1[EID_MAX_BREXITMENTION1_LEN + 1];
    char brexitMention2[EID_MAX_BREXITMENTION2_LEN + 1];
} EidIdentityA, * PEidIdentityA;
```

File

File: CardStructures.h (see page 178)

Members

Members	Description
char cardNumber[EID_MAX_CARD_NUMBER_LEN+1];	Electronic ID card number
char chipNumber[EID_MAX_CHIP_NUMBER_LEN+1];	Electronic ID card physical chip number
char validityDateBegin[EID_MAX_DATE_BEGIN_LEN+1];	Card validity start date
char validityDateEnd[EID_MAX_DATE_END_LEN +1];	Card validity end date
char municipality[EID_MAX_DELIVERY_MUNICIPALITY_LEN+1];	Card delivery municipality
char nationalNumber[EID_MAX_NATIONAL_NUMBER_LEN+1];	National number
char name[EID_MAX_NAME_LEN+1];	Surname
char firstName1[EID_MAX_FIRST_NAME1_LEN+1];	First name (2 first given names)
char firstName2[EID_MAX_FIRST_NAME2_LEN+1];	First name part 2 (first letter of the 3rd given name).
char nationality[EID_MAX_NATIONALITY_LEN+1];	Nationality
char birthLocation[EID_MAX_BIRTHPLACE_LEN+1];	Birth location
char birthDate[EID_MAX_BIRTHDATE_LEN+1];	Birth date
char sex[EID_MAX_SEX_LEN+1];	Sex
char nobleCondition[EID_MAX_NOBLE_CONDITION_LEN+1];	Noble condition
int documentType;	Document type code (Belgian citizen card, Kids Card, Foreigner card)
BOOL whiteCane;	White cane (blind people)
BOOL yellowCane;	Yellow cane (partially sighted people)
BOOL extendedMinority;	Extended minority
char duplicate[EID_MAX_DUPLICATE_LEN + 1];	Duplicata
char specialOrganization[EID_MAX_SPECIALORGANIZATION_LEN + 1];	Special Organization
BOOL memberOfFamily;	Member of family
char dateAndCountryOfProtection[EID_MAX_DATEANDCOUNTRYOFPROTECTION_LEN + 1];	Date and country of protection
char workPermitType[EID_MAX_WORKPERMITTYPE_LEN + 1];	Work Permit type
char vat1[EID_MAX_VAT1_LEN + 1];	Employer VAT1
char vat2[EID_MAX_VAT2_LEN + 1];	Employer VAT2
char regionalFileNumber[EID_MAX_REGIONALFILENUMBER_LEN + 1];	Regional file number
char brexitMention1[EID_MAX_BREXITMENTION1_LEN + 1];	BREXIT
char brexitMention2[EID_MAX_BREXITMENTION2_LEN + 1];	BREXIT

1.2.27 PEidIdentityExA Structure

Identity information stored on EID card - ANSI version

C++

```
typedef struct tagEidIdentityExA {
  WORD nSize;
  WORD nVersion;
  char cardNumber[EID_MAX_CARD_NUMBER_LEN + 1];
  char chipNumber[EID_MAX_CHIP_NUMBER_LEN + 1];
  char validityDateBegin[EID_MAX_DATE_BEGIN_LEN + 1];
  char validityDateEnd[EID_MAX_DATE_END_LEN + 1];
  char municipality[EID_MAX_DELIVERY_MUNICIPALITY_LEN + 1];
  char nationalNumber[EID_MAX_NATIONAL_NUMBER_LEN + 1];
  char name[EID_MAX_NAME_LEN + 1];
  char firstName1[EID_MAX_FIRST_NAME1_LEN + 1];
  char firstName2[EID_MAX_FIRST_NAME2_LEN + 1];
  char nationality[EID_MAX_NATIONALITY_LEN + 1];
  char birthLocation[EID_MAX_BIRTHPLACE_LEN + 1];
  char birthDate[EID_MAX_BIRTHDATE_LEN + 1];
  char sex[EID_MAX_SEX_LEN + 1];
  char nobleCondition[EID_MAX_NOBLE_CONDITION_LEN + 1];
  int documentType;
  BOOL whiteCane;
  BOOL yellowCane;
  BOOL extendedMinority;
  char duplicate[EID_MAX_DUPLICATE_LEN + 1];
  char specialOrganization[EID_MAX_SPECIALORGANIZATION_LEN + 1];
  BOOL memberOfFamily;
  char dateAndCountryOfProtection[EID_MAX_DATEANDCOUNTRYOFPROTECTION_LEN + 1];
  char workPermitType[EID_MAX_WORKPERMITTYPE_LEN + 1];
  char vat1[EID_MAX_VAT1_LEN + 1];
  char vat2[EID_MAX_VAT2_LEN + 1];
  char regionalFileNumber[EID_MAX_REGIONALFILENUMBER_LEN + 1];
  char brexitMention1[EID_MAX_BREXITMENTION1_LEN + 1];
  char brexitMention2[EID_MAX_BREXITMENTION2_LEN + 1];
} EidIdentityExA, * PEidIdentityExA;
```

File

File: CardStructures.h (see page 178)

Members

Members	Description
WORD nSize;	The size of the structure
WORD nVersion;	The version of the structure
char cardNumber[EID_MAX_CARD_NUMBER_LEN + 1];	Electronic ID card number
char chipNumber[EID_MAX_CHIP_NUMBER_LEN + 1];	Electronic ID card physical chip number
char validityDateBegin[EID_MAX_DATE_BEGIN_LEN + 1];	Card validity start date
char validityDateEnd[EID_MAX_DATE_END_LEN + 1];	Card validity end date
char municipality[EID_MAX_DELIVERY_MUNICIPALITY_LEN + 1];	Card delivery municipality
char nationalNumber[EID_MAX_NATIONAL_NUMBER_LEN + 1];	National number
char name[EID_MAX_NAME_LEN + 1];	Surname
char firstName1[EID_MAX_FIRST_NAME1_LEN + 1];	First name (2 first given names)
char firstName2[EID_MAX_FIRST_NAME2_LEN + 1];	First name part 2 (first letter of the 3rd given name).

char nationality[EID_MAX_NATIONALITY_LEN + 1];	Nationality
char birthLocation[EID_MAX_BIRTHPLACE_LEN + 1];	Birth location
char birthDate[EID_MAX_BIRTHDATE_LEN + 1];	Birth date
char sex[EID_MAX_SEX_LEN + 1];	Sex
char nobleCondition[EID_MAX_NOBLE_CONDITION_LEN + 1];	Noble condition
int documentType;	Document type code (Belgian citizen card, Kids Card, Foreigner card)
BOOL whiteCane;	White cane (blind people)
BOOL yellowCane;	Yellow cane (partially sighted people)
BOOL extendedMinority;	Extended minority
char duplicate[EID_MAX_DUPLICATE_LEN + 1];	Duplicata
char specialOrganization[EID_MAX_SPECIALORGANIZATION_LEN + 1];	Special Organization
BOOL memberOfFamily;	Member of family
char dateAndCountryOfProtection[EID_MAX_DATEANDCOUNTRYOFPROTECTION_LEN + 1];	Date and country of protection
char workPermitType[EID_MAX_WORKPERMITTYPE_LEN + 1];	Work Permit type
char vat1[EID_MAX_VAT1_LEN + 1];	Employer VAT1
char vat2[EID_MAX_VAT2_LEN + 1];	Employer VAT2
char regionalFileNumber[EID_MAX_REGIONALFILENUMBER_LEN + 1];	Regional file number
char brexitMention1[EID_MAX_BREXITMENTION1_LEN + 1];	BREXIT
char brexitMention2[EID_MAX_BREXITMENTION2_LEN + 1];	BREXIT

1.2.28 PEidIdentityExW Structure

Identity information stored on EID card - UNICODE version

C++

```
typedef struct tagEidIdentityExW {
  WORD nSize;
  WORD nVersion;
  WCHAR cardNumber[EID_MAX_CARD_NUMBER_LEN + 1];
  WCHAR chipNumber[EID_MAX_CHIP_NUMBER_LEN + 1];
  WCHAR validityDateBegin[EID_MAX_DATE_BEGIN_LEN + 1];
  WCHAR validityDateEnd[EID_MAX_DATE_END_LEN + 1];
  WCHAR municipality[EID_MAX_DELIVERY_MUNICIPALITY_LEN + 1];
  WCHAR nationalNumber[EID_MAX_NATIONAL_NUMBER_LEN + 1];
  WCHAR name[EID_MAX_NAME_LEN + 1];
  WCHAR firstName1[EID_MAX_FIRST_NAME1_LEN + 1];
  WCHAR firstName2[EID_MAX_FIRST_NAME2_LEN + 1];
  WCHAR nationality[EID_MAX_NATIONALITY_LEN + 1];
  WCHAR birthLocation[EID_MAX_BIRTHPLACE_LEN + 1];
  WCHAR birthDate[EID_MAX_BIRTHDATE_LEN + 1];
  WCHAR sex[EID_MAX_SEX_LEN + 1];
  WCHAR nobleCondition[EID_MAX_NOBLE_CONDITION_LEN + 1];
  int documentType;
  BOOL whiteCane;
  BOOL yellowCane;
  BOOL extendedMinority;
  WCHAR duplicate[EID_MAX_DUPLICATE_LEN + 1];
  WCHAR specialOrganization[EID_MAX_SPECIALORGANIZATION_LEN + 1];
  BOOL memberOfFamily;
```

```
    WCHAR dateAndCountryOfProtection[EID_MAX_DATEANDCOUNTRYOFPROTECTION_LEN + 1];
    WCHAR workPermitType[EID_MAX_WORKPERMITTYPE_LEN + 1];
    WCHAR vat1[EID_MAX_VAT1_LEN + 1];
    WCHAR vat2[EID_MAX_VAT2_LEN + 1];
    WCHAR regionalFileNumber[EID_MAX_REGIONALFILENUMBER_LEN + 1];
    WCHAR brexitMention1[EID_MAX_BREXITMENTION1_LEN + 1];
    WCHAR brexitMention2[EID_MAX_BREXITMENTION2_LEN + 1];
  } EidIdentityExW, * PEidIdentityExW;
```

File

File: CardStructures.h (see page 178)

Members

Members	Description
WORD nSize;	The size of the structure
WORD nVersion;	The version of the structure
WCHAR cardNumber[EID_MAX_CARD_NUMBER_LEN + 1];	Electronic ID card number
WCHAR chipNumber[EID_MAX_CHIP_NUMBER_LEN + 1];	Electronic ID card physical chip number
WCHAR validityDateBegin[EID_MAX_DATE_BEGIN_LEN + 1];	Card validity start date
WCHAR validityDateEnd[EID_MAX_DATE_END_LEN + 1];	Card validity end date
WCHAR municipality[EID_MAX_DELIVERY_MUNICIPALITY_LEN + 1];	Card delivery municipality
WCHAR nationalNumber[EID_MAX_NATIONAL_NUMBER_LEN + 1];	National number
WCHAR name[EID_MAX_NAME_LEN + 1];	Surname
WCHAR firstName1[EID_MAX_FIRST_NAME1_LEN + 1];	First name (2 first given names)
WCHAR firstName2[EID_MAX_FIRST_NAME2_LEN + 1];	First name part 2 (first letter of the 3rd given name).
WCHAR nationality[EID_MAX_NATIONALITY_LEN + 1];	Nationality
WCHAR birthLocation[EID_MAX_BIRTHPLACE_LEN + 1];	Birth location
WCHAR birthDate[EID_MAX_BIRTHDATE_LEN + 1];	Birth date
WCHAR sex[EID_MAX_SEX_LEN + 1];	Sex
WCHAR nobleCondition[EID_MAX_NOBLE_CONDITION_LEN + 1];	Noble condition
int documentType;	Document type code (Belgian citizen card, Kids Card, Foreigner card)
BOOL whiteCane;	White cane (blind people)
BOOL yellowCane;	Yellow cane (partially sighted people)
BOOL extendedMinority;	Extended minority
WCHAR duplicate[EID_MAX_DUPLICATE_LEN + 1];	Duplicata
WCHAR specialOrganization[EID_MAX_SPECIALORGANIZATION_LEN + 1];	Special Organization
BOOL memberOfFamily;	Member of family
WCHAR dateAndCountryOfProtection[EID_MAX_DATEANDCOUNTRYOFPROTECTION_LEN + 1];	Date and country of protection
WCHAR workPermitType[EID_MAX_WORKPERMITTYPE_LEN + 1];	Work permit type
WCHAR vat1[EID_MAX_VAT1_LEN + 1];	Employer VAT1
WCHAR vat2[EID_MAX_VAT2_LEN + 1];	Employer VAT2
WCHAR regionalFileNumber[EID_MAX_REGIONALFILENUMBER_LEN + 1];	Regional file number

WCHAR brexitMention1[EID_MAX_BREXITMENTION1_LEN + 1];	BREXIT
WCHAR brexitMention2[EID_MAX_BREXITMENTION2_LEN + 1];	BREXIT

1.2.29 PEidIdentityW Structure

Identity information stored on EID card - UNICODE version

C++

```
typedef struct tagEidIdentityW {
  WCHAR cardNumber[EID_MAX_CARD_NUMBER_LEN+1];
  WCHAR chipNumber[EID_MAX_CHIP_NUMBER_LEN+1];
  WCHAR validityDateBegin[EID_MAX_DATE_BEGIN_LEN+1];
  WCHAR validityDateEnd[EID_MAX_DATE_END_LEN +1];
  WCHAR municipality[EID_MAX_DELIVERY_MUNICIPALITY_LEN+1];
  WCHAR nationalNumber[EID_MAX_NATIONAL_NUMBER_LEN+1];
  WCHAR name[EID_MAX_NAME_LEN+1];
  WCHAR firstName1[EID_MAX_FIRST_NAME1_LEN+1];
  WCHAR firstName2[EID_MAX_FIRST_NAME2_LEN+1];
  WCHAR nationality[EID_MAX_NATIONALITY_LEN+1];
  WCHAR birthLocation[EID_MAX_BIRTHPLACE_LEN+1];
  WCHAR birthDate[EID_MAX_BIRTHDATE_LEN+1];
  WCHAR sex[EID_MAX_SEX_LEN+1];
  WCHAR nobleCondition[EID_MAX_NOBLE_CONDITION_LEN+1];
  int documentType;
  BOOL whiteCane;
  BOOL yellowCane;
  BOOL extendedMinority;
  WCHAR duplicate[EID_MAX_DUPLICATE_LEN + 1];
  WCHAR specialOrganization[EID_MAX_SPECIALORGANIZATION_LEN + 1];
  BOOL memberOfFamily;
  WCHAR dateAndCountryOfProtection[EID_MAX_DATEANDCOUNTRYOFPROTECTION_LEN + 1];
  WCHAR workPermitType[EID_MAX_WORKPERMITTYPE_LEN + 1];
  WCHAR vat1[EID_MAX_VAT1_LEN + 1];
  WCHAR vat2[EID_MAX_VAT2_LEN + 1];
  WCHAR regionalFileNumber[EID_MAX_REGIONALFILENUMBER_LEN + 1];
  WCHAR brexitMention1[EID_MAX_BREXITMENTION1_LEN + 1];
  WCHAR brexitMention2[EID_MAX_BREXITMENTION2_LEN + 1];
} EidIdentityW, * PEidIdentityW;
```

File

File: CardStructures.h (see page 178)

Members

Members	Description
WCHAR cardNumber[EID_MAX_CARD_NUMBER_LEN+1];	Electronic ID card number
WCHAR chipNumber[EID_MAX_CHIP_NUMBER_LEN+1];	Electronic ID card physical chip number
WCHAR validityDateBegin[EID_MAX_DATE_BEGIN_LEN+1];	Card validity start date
WCHAR validityDateEnd[EID_MAX_DATE_END_LEN +1];	Card validity end date
WCHAR municipality[EID_MAX_DELIVERY_MUNICIPALITY_LEN+1];	Card delivery municipality
WCHAR nationalNumber[EID_MAX_NATIONAL_NUMBER_LEN+1];	National number
WCHAR name[EID_MAX_NAME_LEN+1];	Surname
WCHAR firstName1[EID_MAX_FIRST_NAME1_LEN+1];	First name (2 first given names)
WCHAR firstName2[EID_MAX_FIRST_NAME2_LEN+1];	First name part 2 (first letter of the 3rd given name).
WCHAR nationality[EID_MAX_NATIONALITY_LEN+1];	Nationality

WCHAR birthLocation[EID_MAX_BIRTHPLACE_LEN+1];	Birth location
WCHAR birthDate[EID_MAX_BIRTHDATE_LEN+1];	Birth date
WCHAR sex[EID_MAX_SEX_LEN+1];	Sex
WCHAR nobleCondition[EID_MAX_NOBLE_CONDITION_LEN+1];	Noble condition
int documentType;	Document type code (Belgian citizen card, Kids Card, Foreigner card)
BOOL whiteCane;	White cane (blind people)
BOOL yellowCane;	Yellow cane (partially sighted people)
BOOL extendedMinority;	Extended minority
WCHAR duplicate[EID_MAX_DUPLICATE_LEN + 1];	Duplicata
WCHAR specialOrganization[EID_MAX_SPECIALORGANIZATION_LEN + 1];	Special Organization
BOOL memberOfFamily;	Member of family
WCHAR dateAndCountryOfProtection[EID_MAX_DATEANDCOUNTRYOFPROTECTION_LEN + 1];	Date and country of protection
WCHAR workPermitType[EID_MAX_WORKPERMITTYPE_LEN + 1];	Work permit type
WCHAR vat1[EID_MAX_VAT1_LEN + 1];	Employer VAT1
WCHAR vat2[EID_MAX_VAT2_LEN + 1];	Employer VAT2
WCHAR regionalFileNumber[EID_MAX_REGIONALFILENUMBER_LEN + 1];	Regional file number
WCHAR brexitMention1[EID_MAX_BREXITMENTION1_LEN + 1];	BREXIT
WCHAR brexitMention2[EID_MAX_BREXITMENTION2_LEN + 1];	BREXIT

1.2.30 PErrorInformation Structure

Information about error when operation with the card or certificate is performed

C++

```
typedef struct structErrorInformation {
  int Code;
  WCHAR Description[ERROR_MAX_DESCRIPTION + 1];
} ErrorInformation, * PErrorInformation;
```

File

File: CardStructures.h (see page 178)

1.2.31 PSISRecordA Structure

Public information stored on Belgian SIS card - ANSI version. The SIS card is the social security card of each Belgian resident (Belgian or foreigner)

C++

```
typedef struct tagSISRecordA {
  char Name[SIS_MAX_NAME_LEN + 1];
  char FirstName[SIS_MAX_FIRSTNAMES_LEN + 1];
  char Initial[SIS_MAX_INITIAL_LEN+ 1];
  char Sex[SIS_MAX_SEX_LEN + 1];
  char BirthDate[SIS_FIELD_MAX_BIRTHDATE_LEN + 1];
```

```
  char SocialSecurityNumber[SIS_FIELD_MAX_SOCIAL_SECURITY_NUMBER_LEN + 1];
  char CaptureDate[SIS_FIELD_MAX_CAPTUREDATE_LEN + 1];
  char ValidityDateBegin[SIS_FIELD_MAX_VALIDBEGIN_LEN + 1];
  char ValidityDateEnd[SIS_FIELD_MAX_VALIDEND_LEN + 1];
  char CardNumber[SIS_FIELD_MAX_CARDNUMBER_LEN + 1];
  char CardName[SIS_MAX_CARDNAME_LEN +1 ];
} SISRecordA, * PSISRecordA;
```

File

File: CardStructures.h (see page 178)

Members

Members	Description
char Name[SIS_MAX_NAME_LEN + 1];	Family name of the card owner
char FirstName[SIS_MAX_FIRSTNAMES_LEN + 1];	First name of the card owner
char Initial[SIS_MAX_INITIAL_LEN+ 1];	Initial of the card owner
char Sex[SIS_MAX_SEX_LEN + 1];	Sex of the card owner
char BirthDate[SIS_FIELD_MAX_BIRTHDATE_LEN + 1];	Birth date of the card owner
char SocialSecurityNumber[SIS_FIELD_MAX_SOCIAL_SECURITY_NUMBER_LEN + 1];	Social security number of the card owner
char CaptureDate[SIS_FIELD_MAX_CAPTUREDATE_LEN + 1];	Date of issue
char ValidityDateBegin[SIS_FIELD_MAX_VALIDBEGIN_LEN + 1];	Card validity begin date
char ValidityDateEnd[SIS_FIELD_MAX_VALIDEND_LEN + 1];	Card validity end date
char CardNumber[SIS_FIELD_MAX_CARDNUMBER_LEN + 1];	Card number
char CardName[SIS_MAX_CARDNAME_LEN +1];	Name of the card

1.2.32 PSISRecordW Structure

Public information stored on Belgian SIS card - UNICODE version. The SIS card is the social security card of each Belgian resident (Belgian or foreigner)

C++

```
typedef struct tagSISRecordW {
  WCHAR Name[SIS_MAX_NAME_LEN + 1];
  WCHAR FirstName[SIS_MAX_FIRSTNAMES_LEN + 1];
  WCHAR Initial[SIS_MAX_INITIAL_LEN+ 1];
  WCHAR Sex[SIS_MAX_SEX_LEN + 1];
  WCHAR BirthDate[SIS_FIELD_MAX_BIRTHDATE_LEN + 1];
  WCHAR SocialSecurityNumber[SIS_FIELD_MAX_SOCIAL_SECURITY_NUMBER_LEN + 1];
  WCHAR CaptureDate[SIS_FIELD_MAX_CAPTUREDATE_LEN + 1];
  WCHAR ValidityDateBegin[SIS_FIELD_MAX_VALIDBEGIN_LEN + 1];
  WCHAR ValidityDateEnd[SIS_FIELD_MAX_VALIDEND_LEN + 1];
  WCHAR CardNumber[SIS_FIELD_MAX_CARDNUMBER_LEN + 1];
  WCHAR CardName[SIS_MAX_CARDNAME_LEN +1 ];
} SISRecordW, * PSISRecordW;
```

File

File: CardStructures.h (see page 178)

Members

Members	Description
WCHAR Name[SIS_MAX_NAME_LEN + 1];	Family name of the card owner
WCHAR FirstName[SIS_MAX_FIRSTNAMES_LEN + 1];	First name of the card owner
WCHAR Initial[SIS_MAX_INITIAL_LEN+ 1];	Initial of the card owner
WCHAR Sex[SIS_MAX_SEX_LEN + 1];	Sex of the card owner

WCHAR BirthDate[SIS_FIELD_MAX_BIRTHDATE_LEN + 1];	Birth date of the card owner
WCHAR SocialSecurityNumber[SIS_FIELD_MAX_SOCIAL_SECURITY_NUMBER_LEN + 1];	Social security number of the card owner
WCHAR CaptureDate[SIS_FIELD_MAX_CAPTUREDATE_LEN + 1];	Date of issue
WCHAR ValidityDateBegin[SIS_FIELD_MAX_VALIDBEGIN_LEN + 1];	Card validity begin date
WCHAR ValidityDateEnd[SIS_FIELD_MAX_VALIDEND_LEN + 1];	Card validity end date
WCHAR CardNumber[SIS_FIELD_MAX_CARDNUMBER_LEN + 1];	Card number
WCHAR CardName[SIS_MAX_CARDNAME_LEN +1];	Name of the card

1.2.33 PeidPicture Structure

Raw picture data from EID card

C++

```
typedef struct tagEidPicture {
  BYTE picture[EID_MAX_PICTURE_LEN+1];
  int pictureLength;
} EidPicture, * PeidPicture;
```

File

File: CardStructures.h (see page 178)

Members

Members	Description
BYTE picture[EID_MAX_PICTURE_LEN+1];	Picture raw data buffer
int pictureLength;	Picture raw data buffer length

1.2.34 SISRecordA Structure

Public information stored on Belgian SIS card - ANSI version. The SIS card is the social security card of each Belgian resident (Belgian or foreigner)

C++

```
typedef struct tagSISRecordA {
  char Name[SIS_MAX_NAME_LEN + 1];
  char FirstName[SIS_MAX_FIRSTNAMES_LEN + 1];
  char Initial[SIS_MAX_INITIAL_LEN+ 1];
  char Sex[SIS_MAX_SEX_LEN + 1];
  char BirthDate[SIS_FIELD_MAX_BIRTHDATE_LEN + 1];
  char SocialSecurityNumber[SIS_FIELD_MAX_SOCIAL_SECURITY_NUMBER_LEN + 1];
  char CaptureDate[SIS_FIELD_MAX_CAPTUREDATE_LEN + 1];
  char ValidityDateBegin[SIS_FIELD_MAX_VALIDBEGIN_LEN + 1];
  char ValidityDateEnd[SIS_FIELD_MAX_VALIDEND_LEN + 1];
  char CardNumber[SIS_FIELD_MAX_CARDNUMBER_LEN + 1];
  char CardName[SIS_MAX_CARDNAME_LEN +1 ];
} SISRecordA, * PSISRecordA;
```

File

File: CardStructures.h (see page 178)

Members

Members	Description
char Name[SIS_MAX_NAME_LEN + 1];	Family name of the card owner

char FirstName[SIS_MAX_FIRSTNAMES_LEN + 1];	First name of the card owner
char Initial[SIS_MAX_INITIAL_LEN+ 1];	Initial of the card owner
char Sex[SIS_MAX_SEX_LEN + 1];	Sex of the card owner
char BirthDate[SIS_FIELD_MAX_BIRTHDATE_LEN + 1];	Birth date of the card owner
char SocialSecurityNumber[SIS_FIELD_MAX_SOCIAL_SECURITY_NUMBER_LEN + 1];	Social security number of the card owner
char CaptureDate[SIS_FIELD_MAX_CAPTUREDATE_LEN + 1];	Date of issue
char ValidityDateBegin[SIS_FIELD_MAX_VALIDBEGIN_LEN + 1];	Card validity begin date
char ValidityDateEnd[SIS_FIELD_MAX_VALIDEND_LEN + 1];	Card validity end date
char CardNumber[SIS_FIELD_MAX_CARDNUMBER_LEN + 1];	Card number
char CardName[SIS_MAX_CARDNAME_LEN +1];	Name of the card

1.2.35 SISRecordW Structure

Public information stored on Belgian SIS card - UNICODE version. The SIS card is the social security card of each Belgian resident (Belgian or foreigner)

C++

```
typedef struct tagSISRecordW {
  WCHAR Name[SIS_MAX_NAME_LEN + 1];
  WCHAR FirstName[SIS_MAX_FIRSTNAMES_LEN + 1];
  WCHAR Initial[SIS_MAX_INITIAL_LEN+ 1];
  WCHAR Sex[SIS_MAX_SEX_LEN + 1];
  WCHAR BirthDate[SIS_FIELD_MAX_BIRTHDATE_LEN + 1];
  WCHAR SocialSecurityNumber[SIS_FIELD_MAX_SOCIAL_SECURITY_NUMBER_LEN + 1];
  WCHAR CaptureDate[SIS_FIELD_MAX_CAPTUREDATE_LEN + 1];
  WCHAR ValidityDateBegin[SIS_FIELD_MAX_VALIDBEGIN_LEN + 1];
  WCHAR ValidityDateEnd[SIS_FIELD_MAX_VALIDEND_LEN + 1];
  WCHAR CardNumber[SIS_FIELD_MAX_CARDNUMBER_LEN + 1];
  WCHAR CardName[SIS_MAX_CARDNAME_LEN +1 ];
} SISRecordW, * PSISRecordW;
```

File

File: CardStructures.h (see page 178)

Members

Members	Description
WCHAR Name[SIS_MAX_NAME_LEN + 1];	Family name of the card owner
WCHAR FirstName[SIS_MAX_FIRSTNAMES_LEN + 1];	First name of the card owner
WCHAR Initial[SIS_MAX_INITIAL_LEN+ 1];	Initial of the card owner
WCHAR Sex[SIS_MAX_SEX_LEN + 1];	Sex of the card owner
WCHAR BirthDate[SIS_FIELD_MAX_BIRTHDATE_LEN + 1];	Birth date of the card owner
WCHAR SocialSecurityNumber[SIS_FIELD_MAX_SOCIAL_SECURITY_NUMBER_LEN + 1];	Social security number of the card owner
WCHAR CaptureDate[SIS_FIELD_MAX_CAPTUREDATE_LEN + 1];	Date of issue
WCHAR ValidityDateBegin[SIS_FIELD_MAX_VALIDBEGIN_LEN + 1];	Card validity begin date
WCHAR ValidityDateEnd[SIS_FIELD_MAX_VALIDEND_LEN + 1];	Card validity end date
WCHAR CardNumber[SIS_FIELD_MAX_CARDNUMBER_LEN + 1];	Card number
WCHAR CardName[SIS_MAX_CARDNAME_LEN +1];	Name of the card

1.3 Files

The following table lists files in this documentation.

Files

Name	Description
Algorithms.h (see page 178)	
CardEvents.h (see page 178)	The definition of the possible card events
CardStructures.h (see page 178)	The definition of the information storage structures
Encryption.h (see page 179)	Encryption and decryption operations
FileOperations.h (see page 180)	Files and folders manipulations
Graphics.h (see page 182)	
NationalityConverter.h (see page 183)	Nationality to ISO code cenverter
Network.h (see page 183)	
Swelio.h (see page 183)	Belgian electronic Id card access engine
System.h (see page 188)	Windows related routines
SystemInfo.h (see page 189)	Routines for querying information about operating system
quricol.h (see page 190)	QR Code generator

1.3.1 Algorithms.h

1.3.2 CardEvents.h

The definition of the possible card events

Enumerations

	Name	Description
	tagCardEventType (see page 150)	The type of the reader event
	CardEventType (see page 159)	The type of the reader event

1.3.3 CardStructures.h

The definition of the information storage structures

Structures

	Name	Description
	structErrorInformation (see page 149)	Information about error when operation with the card or certificate is performed
	tagEidAddressA (see page 150)	EID address information, stored on the card - ANSI version
	tagEidAddressW (see page 150)	EID address information, stored on the card - UNICODE version
	tagEidCertificate (see page 151)	Certificate, stored on EID card

	tagEidIdentityA (see page 151)	Identity information stored on EID card - ANSI version
	tagEidIdentityExA (see page 153)	Identity information stored on EID card - ANSI version
	tagEidIdentityExW (see page 154)	Identity information stored on EID card - UNICODE version
	tagEidIdentityW (see page 156)	Identity information stored on EID card - UNICODE version
	tagEidPicture (see page 157)	Raw picture data from EID card
	tagSISRecordA (see page 157)	Public information stored on Belgian SIS card - ANSI version. The SIS card is the social security card of each Belgian resident (Belgian or foreigner)
	tagSISRecordW (see page 158)	Public information stored on Belgian SIS card - UNICODE version. The SIS card is the social security card of each Belgian resident (Belgian or foreigner)
	EidAddressA (see page 159)	EID address information, stored on the card - ANSI version
	EidAddressW (see page 160)	EID address information, stored on the card - UNICODE version
	EidCertificate (see page 160)	Certificate, stored on EID card
	EidIdentityA (see page 160)	Identity information stored on EID card - ANSI version
	EidIdentityExA (see page 162)	Identity information stored on EID card - ANSI version
	EidIdentityExW (see page 163)	Identity information stored on EID card - UNICODE version
	EidIdentityW (see page 165)	Identity information stored on EID card - UNICODE version
	EidPicture (see page 166)	Raw picture data from EID card
	ErrorInformation (see page 167)	Information about error when operation with the card or certificate is performed
	PEidAddressA (see page 167)	EID address information, stored on the card - ANSI version
	PEidAddressW (see page 167)	EID address information, stored on the card - UNICODE version
	PEidCertificate (see page 168)	Certificate, stored on EID card
	PEidIdentityA (see page 168)	Identity information stored on EID card - ANSI version
	PEidIdentityExA (see page 170)	Identity information stored on EID card - ANSI version
	PEidIdentityExW (see page 171)	Identity information stored on EID card - UNICODE version
	PEidIdentityW (see page 173)	Identity information stored on EID card - UNICODE version
	PErrorInformation (see page 174)	Information about error when operation with the card or certificate is performed
	PSISRecordA (see page 174)	Public information stored on Belgian SIS card - ANSI version. The SIS card is the social security card of each Belgian resident (Belgian or foreigner)
	PSISRecordW (see page 175)	Public information stored on Belgian SIS card - UNICODE version. The SIS card is the social security card of each Belgian resident (Belgian or foreigner)
	PeidPicture (see page 176)	Raw picture data from EID card
	SISRecordA (see page 176)	Public information stored on Belgian SIS card - ANSI version. The SIS card is the social security card of each Belgian resident (Belgian or foreigner)
	SISRecordW (see page 177)	Public information stored on Belgian SIS card - UNICODE version. The SIS card is the social security card of each Belgian resident (Belgian or foreigner)

1.3.4 Encryption.h

Encryption and decryption operations

Functions

	Name	Description
	CardDecryptFileA (see page 17)	Decrypt file using Belgian Id card

	CardDecryptFileW (see page 17)	Decrypt file using Belgian Id card
	CardEncryptFileA (see page 17)	Encrypt file using Belgian Id card
	CardEncryptFileW (see page 18)	Encrypt file using Belgian Id card
	CheckMD5 (see page 23)	Checks the MD5 hash value of the memory buffer
	CheckSHA1 (see page 24)	Checks the SHA1 hash value of the memory buffer
	CheckSHA256 (see page 24)	Checks the SHA256 hash value of the memory buffer
	DecryptFileAESA (see page 32)	Decrypts file using AES algorithm.
	DecryptFileAESW (see page 32)	Decrypts file using AES algorithm.
	EncryptFileAESA (see page 42)	Encrypts file using AES algorithm.
	EncryptFileAESW (see page 42)	Encrypts file using AES algorithm.
	GetFileMD5A (see page 66)	Gets the MD5 hash value for the file
	GetFileMD5W (see page 67)	Gets the MD5 hash value for the file
	GetFileSHA1A (see page 67)	Gets the SHA1 hash value for the file
	GetFileSHA1W (see page 68)	Gets the SHA1 hash value for the file
	GetFileSHA256A (see page 68)	Gets the SHA256 hash value for the file
	GetFileSHA256W (see page 68)	Gets the SHA256 hash value for the file
	GetFileSHA384A (see page 69)	Gets the SHA384 hash value for the file
	GetFileSHA384W (see page 69)	Gets the SHA384 hash value for the file
	GetFileSHA512A (see page 70)	Gets the SHA512 hash value for the file
	GetFileSHA512W (see page 70)	Gets the SHA512 hash value for the file
	GetMD5 (see page 73)	Gets the MD5 hash value for the content of the memory buffer
	GetSHA1 (see page 77)	Gets the SHA1 hash value for the content of the memory buffer
	GetSHA256 (see page 78)	Gets the SHA256 hash value for the content of the memory buffer
	GetSHA384 (see page 78)	Gets the SHA384 hash value for the content of the memory buffer
	GetSHA512 (see page 79)	Gets the SHA512 hash value for the content of the memory buffer

1.3.5 FileOperations.h

Files and folders manipulations

Functions

	Name	Description
	AllocateBuffer (see page 12)	Allocates the buffer in memory
	ClearFileAttributesA (see page 25)	This function sets the file attributes to normal.
	ClearFileAttributesW (see page 25)	This function sets the file attributes to normal.
	CreateUnicodeFileA (see page 28)	Creates UNICODE file
	CreateUnicodeFileW (see page 29)	Creates UNICODE file
	DeallocateBuffer (see page 31)	Deallocates the memory buffer
	DeleteToRecycleBinA (see page 33)	Deletes file to the Windows Recycle Bin
	DeleteToRecycleBinW (see page 33)	Deletes file to WIndows Recycle Bin
	DirectoryExistsA (see page 34)	Determines whether a specified directory exists.
	DirectoryExistsW (see page 35)	Determines whether a specified directory exists.

	FileCloseA (see page 43)	Concludes input/output (I/O) to a file opened using the FileCreateRewrite function.
	FileCloseW (see page 43)	Concludes input/output (I/O) to a file opened using the FileCreateRewrite function.
	FileCopyA (see page 43)	The CopyFile function copies an existing file to a new file.
	FileCopyW (see page 44)	The CopyFile function copies an existing file to a new file.
	FileCreateRewriteA (see page 44)	Creates new or overwrites existing file
	FileCreateRewriteW (see page 44)	Creates new or overwrites existing file
	FileDeleteA (see page 45)	Deletes a file from disk.
	FileDeleteW (see page 45)	Deletes a file from disk.
	FileExistsA (see page 46)	Tests whether a specified file exists.
	FileExistsW (see page 46)	Tests whether a specified file exists.
	FileExtensionIsA (see page 46)	Checks the file extension
	FileExtensionIsW (see page 47)	Checks the file extension
	FileGetSizeA (see page 47)	Retrieves the size of a specified file.
	FileGetSizeW (see page 47)	Retrieves the size of a specified file.
	FileIsExeA (see page 48)	Checks if the file is a Windows executable
	FileIsExeW (see page 48)	Checks if the file is a Windows executable
	FileIsIconA (see page 49)	Checks if the file is a Windows icon (.ico) file
	FileIsIconW (see page 49)	Checks if the file is a Windows icon (.ico) file
	FileIsImageA (see page 49)	Checks if the file is an image file
	FileIsImageW (see page 50)	Checks if the file is an image file
	FileIsLink (see page 50)	Checks to see if the file specified by file name is a Microsoft Windows shortcut (.Lnk) file (and is neither a file nor a folder).
	FileOrFolderExistsA (see page 50)	Checks if the file or folder with the given name exists
	FileOrFolderExistsW (see page 51)	Checks if the file or folder with the given name exists
	FileRenameA (see page 51)	Renames the file
	FileRenameW (see page 51)	Renames the file
	FileWriteA (see page 52)	Writes string to the file
	FileWriteCharA (see page 52)	Writes one character to the file
	FileWriteCharW (see page 52)	Writes one character to the file
	FileWriteNewLineA (see page 53)	Writes new line sequence to the file
	FileWriteNewLineW (see page 53)	Writes new line sequence to the file
	FileWriteW (see page 53)	Writes string to the file
	FullPathA (see page 54)	Gets the full path to the file based on file name
	FullPathW (see page 54)	Gets the full path to the file based on file name
	GetAllFiles (see page 62)	Returns the names of files in a specified directory.
	GetFilesCountA (see page 71)	Calculates the number of files in the given folder
	GetFilesCountW (see page 71)	Calculates the number of files in the given folder
	IsAnimatedGIFA (see page 84)	Checks if the file is an animated GIF image file
	IsAnimatedGIFW (see page 84)	Checks if the file is an animated GIF image file
	IsDirectoryA (see page 87)	Verifies that a path is a valid directory.
	IsDirectoryW (see page 87)	Verifies that a path is a valid directory.
	IsUnicodeFileA (see page 93)	Checks if the file is UNICODE file
	IsUnicodeFileW (see page 93)	Checks if the file is UNICODE file
	IsValidFileNameA (see page 94)	Checks if provided string is a valid file name
	IsValidFileNameW (see page 94)	Checks if provided string is a valid file name

	IsValidPathNameA (see page 95)	Checks if provided string is a valid file path
	IsValidPathNameW (see page 95)	Checks if provided string is a valid file path
	ReadBufferFromFileA (see page 108)	Reads the content of the file to the memory buffer
	ReadBufferFromFileW (see page 108)	Reads the content of the file to the memory buffer
	ShellCopyFileA (see page 140)	Copies file to the new location
	ShellCopyFileW (see page 140)	Copies file to the new location
	StripFileNameA (see page 143)	Replaces environment variable names with values
	StripFileNameW (see page 143)	Replaces environment variable names with values
	WriteBufferToFileA (see page 147)	Writes the memory buffer to file
	WriteBufferToFileW (see page 147)	Writes the memory buffer to file

1.3.6 Graphics.h

Functions

	Name	Description
	AlphaBlendBitmap (see page 15)	Performs alpha blending of a bitmap onto a device context
	AlphaBlendNative (see page 16)	Performs alpha blending between two device contexts
	CloneFont (see page 26)	Clones an existing font
	CopyNativeBitmap (see page 27)	Copies a native bitmap to a device context
	CreateNativeBitmap (see page 28)	Creates a new native bitmap
	CreateWindowsFont (see page 29)	Creates a Windows font with specified properties
	DestroyFont (see page 34)	Destroys a font and releases its resources
	DpiY (see page 36)	Gets the horizontal DPI of the display Returns: The horizontal DPI value
	DrawAlphaText (see page 36)	Draws text with alpha blending onto a device context
	DrawAlphaTextRect (see page 36)	Draws text with alpha blending within a specified rectangle on a device context
	DrawNativeBitmap (see page 37)	Draws a native bitmap onto another bitmap
	DrawTextDirect (see page 38)	Draws text directly onto a device context
	DrawTextDirectEx (see page 38)	Draws text directly onto a device context with specified background color
	DrawTextGlow (see page 38)	Draws text with a glowing effect onto a device context
	DrawTextLine (see page 39)	Draws a single line of text onto a device context
	DrawTextOutline (see page 39)	Draws text with an outline effect onto a device context
	DrawTextRect (see page 40)	Draws text within a specified rectangle on a device context
	EmToPixels (see page 40)	Converts em units to pixels
	GetTextLineSize (see page 80)	Calculates the size of a single line of text in pixels
	GetTextSize (see page 81)	Calculates the size of the text in pixels
	GetTextSizeEx (see page 81)	Gets the size of text when rendered with a specific font
	LoadBitmapJPG (see page 99)	Loads a JPEG bitmap from a file
	LoadBitmapPNG (see page 100)	Loads a PNG bitmap from a file
	LoadPNGResource (see page 101)	Loads a PNG bitmap from a resource
	MakeCompatibleBitmap (see page 103)	Creates a bitmap compatible with the specified source bitmap

	PointsToPixels (see page 105)	Converts points to pixels
	StretchNativeBitmap (see page 142)	Stretches a native bitmap to fit the specified dimensions

1.3.7 NationalityConverter.h

Nationality to ISO code cenverter

Functions

	Name	Description
	GetISOCodeA (see page 72)	Returns the country ISO code based on the nationality string
	GetISOCodeW (see page 73)	Returns the country ISO code based on the nationality string

1.3.8 Network.h

Functions

	Name	Description
	ServerAccessible (see page 136)	Checks if the server is accessible

1.3.9 Swelio.h

Belgian electronic Id card access engine

Functions

	Name	Description
	ActivateCard (see page 10)	Established communication between the card and the reader
	ActivateCardEx (see page 11)	Established communication between the card and the reader
	AddFileToContainer (see page 11)	Add existing file to the container
	CardSignCMS (see page 18)	Sign data with eID card according to CMS standard
	CardSignCMSEx (see page 19)	Sign data with eID card according to CMS standard
	CardSignCadesT (see page 19)	Sign data with eID card according to CADES-T standard
	CardSignCadesTEx (see page 20)	Sign data with eID card according to CADES-T standard
	CertSignCMS (see page 20)	Sign data with the certificate file according to CMS standard
	CertSignCMSData (see page 21)	Sign data with the certificate according to CMS standard
	CertSignCMSEx (see page 21)	Sign data with the certificate file according to CMS standard
	CertSignCadesT (see page 22)	Sign data with the certificate file according to CADES-T standard
	CertSignCadesTData (see page 22)	Sign data with the certificate according to CADES-T standard
	CertSignCadesTEx (see page 23)	Sign data with the certificate file according to CADES-T standard
	ContainerCertificate (see page 26)	Assign certificate for signing ASIC container
	ContainerEidCertificate (see page 26)	Select EID card certificate to sign ASIC container

	ContainerPickCertificate (see page 27)	Pick certificate to sign ASIC container
	CreateCardBuffer (see page 28)	Creates XML buffer
	DeactivateCard (see page 30)	Terminates a cennection between a smart card and a reader
	DeactivateCardEx (see page 30)	Terminates a connection between a smart card and a reader
	DeleteCardBuffer (see page 33)	Deletes XML buffer
	DisplayCertificate (see page 35)	Displays the dialog window with certificate information
	EncodeCertificate (see page 41)	Performs Base64 encoding of the certificate
	EncodePhoto (see page 41)	Performs Base64 encoding of the photo
	FreeContainer (see page 54)	Deallocates ASIC container
	GenerateAuthenticationSignatureA (see page 55)	Generate authentication signature
	GenerateAuthenticationSignatureExA (see page 55)	Generate authentication signature
	GenerateAuthenticationSignatureExW (see page 56)	Generate authentication signature
	GenerateAuthenticationSignatureW (see page 56)	Generate authentication signature
	GenerateNonRepudiationSignatureA (see page 58)	Generate non repudiation signature
	GenerateNonRepudiationSignatureExA (see page 58)	Generate non repudiation signature
	GenerateNonRepudiationSignatureExW (see page 59)	Generate non repudiation signature
	GenerateNonRepudiationSignatureW (see page 59)	Generate non repudiation signature
	GenerateQRCodeA (see page 60)	Read eID card and save the identity information and address to PNG QR Code file
	GenerateQRCodeExA (see page 61)	Read eID card and save the identity information and address to PNG QR Code file
	GenerateQRCodeExW (see page 61)	Read eID card and save the identity information and address to PNG QR Code file
	GenerateQRCodeW (see page 62)	Read eID card and save the identity information and address to PNG QR Code file
	GetCardBufferA (see page 62)	Gets XML or CSV information from the memory buffer
	GetCardBufferSize (see page 63)	This function returns the size of the buffer needed to hold the information from the eID card in the XML or CSV format
	GetCardBufferW (see page 63)	Gets XML or CSV information from the memory buffer
	GetCardSerialNumber (see page 64)	Get the serial number of EID card
	GetCardVersion (see page 64)	Get the applet version number for card in the reader with specified number
	GetContainerError (see page 65)	Gets the value of the container validation error message based on the message index
	GetContainerErrorsCount (see page 65)	Get the number of error messages availavle in the container validation report
	GetEncodedCertificateSize (see page 65)	Returns the size of the Base64 encoded certificate
	GetEncodedPhotoSize (see page 66)	Calculates buffer size for Base64 encoded photo
	GetReaderIndexA (see page 75)	Returns the zero-based reader index with specified name
	GetReaderIndexW (see page 75)	Returns the zero-based reader index with specified name
	GetReaderNameA (see page 75)	Returns the name of the card reader
	GetReaderNameLenA (see page 76)	Returns the length of the reader name
	GetReaderNameLenW (see page 76)	Returns the length of the reader name

	GetReaderNameW (see page 76)	Returns the name of the card reader
	GetReadersCount (see page 77)	Get number of card readers connected to PC
	GetSelectedReaderIndex (see page 79)	Returns the index of the active smart card reader
	GetSupportSIS (see page 80)	Checks if the SIS cards are supported by the engine
	GetTimeServer (see page 81)	Get the time server URI for digital signature of the ASIC container
	IgnoreHardwareEvents (see page 82)	Ignore USB reader insert / remove events
	IgnoreServiceEvents (see page 82)	Ignore smartcard service stop events when reporting readers list change
	IgnoreUnknownCards (see page 83)	Ignore Unknown cards insert / remove events
	InitializeContainer (see page 83)	Initializes ASIC container
	IsCardActivated (see page 84)	Checks the connection between a smart card and a reader
	IsCardActivatedEx (see page 85)	Checks the connection between a smart card and a reader
	IsCardPresent (see page 85)	Checks if the card is present in the card reader
	IsCardPresentEx (see page 85)	Checks if the card is present in the card reader
	IsCardStillInserted (see page 86)	Checks if the card is still inserted in the card reader
	IsCardStillInsertedEx (see page 86)	Checks if the card is still inserted in the card reader
	IsEIDCard (see page 88)	Check if Belgian EID card is inserted into card reader
	IsEIDCardEx (see page 88)	Check if Belgian EID card is inserted into card reader
	IsEngineActive (see page 88)	Checks if the Swelio Engine is activated
	IsFemaleA (see page 89)	Checks if the card owner is female
	IsFemaleW (see page 89)	Checks if the card owner is female
	IsMaleA (see page 89)	Checks if the card owner is male
	IsMaleW (see page 90)	Checks if the card owner is male
	IsSISCard (see page 92)	Check if Belgian SIS card is inserted into card reader
	IsSISCardEx (see page 92)	Check if Belgian SIS card is inserted into card reader
	LoadAddressA (see page 98)	Reads the raw address information from a file
	LoadAddressW (see page 99)	Reads the raw address information from a file
	LoadCertificateA (see page 100)	Reads the certificate from a file
	LoadCertificateW (see page 100)	Reads the certificate from a file
	LoadIdentityA (see page 101)	Reads the raw identity information from a file
	LoadIdentityW (see page 101)	Reads the raw identity information from a file
	LoadPhotoA (see page 102)	Loads photo from a file
	LoadPhotoW (see page 102)	Loads photo from a file
	NonRepudiationSignatureAlgoA (see page 104)	This is function NonRepudiationSignatureAlgoA.
	NonRepudiationSignatureAlgoW (see page 105)	This is function NonRepudiationSignatureAlgoW.
	ReadAddressA (see page 106)	Read address information from Belgian eID card
	ReadAddressExA (see page 106)	Read address information from Belgian eID card
	ReadAddressExW (see page 106)	Read address information from Belgian eID card
	ReadAddressW (see page 107)	Read address information from Belgian eID card
	ReadAuthenticationCertificate (see page 107)	Read Authentication Certificate to memory
	ReadAuthenticationCertificateEx (see page 107)	Read Authentication Certificate to memory
	ReadCaCertificate (see page 109)	Read Ca Certificate to memory
	ReadCaCertificateEx (see page 109)	Read Ca Certificate to memory
	ReadIdentityA (see page 109)	Read identity information from Belgian eID card
	ReadIdentityExA (see page 110)	Read identity information from Belgian eID card

	ReadIdentityExW (see page 110)	Read identity information from Belgian eID card
	ReadIdentityW (see page 110)	Read identity information from Belgian eID card
	ReadNonRepudiationCertificate (see page 111)	Read Non Repudiation Certificate to memory
	ReadNonRepudiationCertificateEx (see page 111)	Read Non Repudiation Certificate to memory
	ReadPhoto (see page 112)	Reads a photo from a card
	ReadPhotoAsBitmap (see page 112)	Reads the picture from the card, converts it to bitmap and returns the bitmap handle Decription: Reads the photo from the Belgian eID card and returns the bitmap handle Reading the photo from the card is a time consuming operation. Do it only when needed.
	ReadPhotoAsBitmapEx (see page 112)	Reads the picture from the card, converts it to bitmap and returns the bitmap handle Decription: Reads the photo from the Belgian eID card and returns the Windows bitmap handle Reading the photo from the card is a time consuming operation. Do it only when needed.
	ReadPhotoEx (see page 113)	Reads a photo from a card
	ReadRootCaCertificate (see page 113)	Read Root Ca Certificate to memory
	ReadRootCaCertificateEx (see page 114)	Read Root Ca Certificate to memory
	ReadRrnCertificate (see page 114)	Read Rrn Certificate to memory
	ReadRrnCertificateEx (see page 114)	Read Rrn Certificate to memory
	ReadSISCardA (see page 115)	Read Belgian SIS card.
	ReadSISCardExA (see page 115)	Read Belgian SIS card.
	ReadSISCardExW (see page 116)	Read Belgian SIS card.
	ReadSISCardW (see page 116)	Read Belgian SIS card.
	ReloadReadersList (see page 117)	Reloads the list of the available card readers
	RemoveCallback (see page 117)	Remove callback procedure for card events
	SaveAddressA (see page 119)	Saves address information to a file
	SaveAddressW (see page 119)	Saves address information to a file
	SaveAuthenticationCertificateA (see page 119)	Save Authentication Certificate to a file
	SaveAuthenticationCertificateExW (see page 120)	Save Authentication Certificate to a file
	SaveAuthenticationCertificateW (see page 120)	Save Authentication Certificate to a file
	SaveCaCertificateA (see page 120)	Save Ca Certificate to a file
	SaveCaCertificateExW (see page 121)	Save Ca Certificate to a file
	SaveCaCertificateW (see page 121)	Save Ca Certificate to a file
	SaveCardToToXMLStreamExA (see page 122)	Read eID card and save the information to XML buffer
	SaveCardToToXMLStreamExW (see page 122)	Read eID card and save the information to XML buffer
	SaveCardToXmlA (see page 122)	Read eID card and save the information to XML file
	SaveCardToXmlExA (see page 123)	Read eID card and save the information to XML file
	SaveCardToXmlExW (see page 123)	Read eID card and save the information to XML file
	SaveCardToXmlW (see page 124)	Read eID card and save the information to XML file
	SaveContainer (see page 124)	Save container to the file
	SaveIdentityA (see page 124)	Saves indentity information to a file
	SaveIdentityW (see page 125)	Saves indentity information to a file
	SaveNonRepudiationCertificateA (see page 125)	Save Non Repudiation Certificate to a file

	SaveNonRepudiationCertificateExW (see page 126)	Save Non Repudiation Certificate to a file
	SaveNonRepudiationCertificateW (see page 126)	Save Non Repudiation Certificate to a file
	SavePersonCsvToStreamA (see page 126)	Read eID card and save the identity information to CSV memory buffer
	SavePersonCsvToStreamW (see page 127)	Read eID card and save the identity information to CSV memory buffer
	SavePersonToCsvA (see page 127)	Read eID card and save the identity information and address to CSV file
	SavePersonToCsvExA (see page 128)	Read eID card and save the identity information and address to CSV file
	SavePersonToCsvExW (see page 128)	Read eID card and save the identity information and address to CSV file
	SavePersonToCsvW (see page 128)	Read eID card and save the identity information and address to CSV file
	SavePhotoA (see page 129)	Save photo to a file
	SavePhotoAsBitmapA (see page 129)	Save the picture from the card to Windows Bitmap file Decription: Reads the photo from the Belgian eID card and writes it to the file as bitmap image. Reading the photo from the card is a time consuming operation. Do it only when needed.
	SavePhotoAsBitmapExA (see page 130)	Reads the picture from the card and saves it to Windows Bitmap file Decription: Reads the photo from the Belgian eID card and writes it to the file as bitmap image. Reading the photo from the card is a time consuming operation. Do it only when needed.
	SavePhotoAsBitmapExW (see page 130)	Reads the picture from the card and saves it to Windows Bitmap file Decription: Reads the photo from the Belgian eID card and writes it to the file as bitmap image. Reading the photo from the card is a time consuming operation. Do it only when needed.
	SavePhotoAsBitmapW (see page 130)	Save the picture from the card to Windows Bitmap file Decription: Reads the photo from the Belgian eID card and writes it to the file as bitmap image. Reading the photo from the card is a time consuming operation. Do it only when needed.
	SavePhotoAsJpegA (see page 131)	Save the picture from the card to JPG file Decription: Reads the photo from the Belgian eID card and writes it to the file as JPG image. Reading the photo from the card is a time consuming operation. Do it only when needed.
	SavePhotoAsJpegExA (see page 131)	Save the picture from the card to JPG file Decription: Reads the photo from the Belgian eID card and writes it to the file as JPG image. Reading the photo from the card is a time consuming operation. Do it only when needed.
	SavePhotoAsJpegExW (see page 131)	Save the picture from the card to JPG file Decription: Reads the photo from the Belgian eID card and writes it to the file as JPG image. Reading the photo from the card is a time consuming operation. Do it only when needed.
	SavePhotoAsJpegW (see page 132)	Save the picture from the card to JPG file Decription: Reads the photo from the Belgian eID card and writes it to the file as JPG image. Reading the photo from the card is a time consuming operation. Do it only when needed.
	SavePhotoW (see page 132)	Saves photo to a file
	SaveRootCaCertificateA (see page 133)	Save Root Ca Certificate to a file
	SaveRootCaCertificateExW (see page 133)	Save Root Ca Certificate to a file
	SaveRootCaCertificateW (see page 133)	Save Root Ca Certificate to a file
	SaveRrnCertificateA (see page 134)	Save RRN Certificate to a file

	SaveRrnCertificateExW (see page 134)	Save RRN Certificate to a file
	SaveRrnCertificateW (see page 134)	Save RRN Certificate to a file
	SelectReader (see page 135)	When more than 1 reader connected, select the reader with specified number
	SelectReaderByNameA (see page 135)	Select active smart card reader by providing the reader name
	SelectReaderByNameW (see page 136)	Select active smart card reader by providing the reader name
	SendAPDU (see page 136)	This is function SendAPDU.
	SetCallback (see page 137)	Activates callback procedure for card status change event
	SetContainerCanonization (see page 137)	Set XML canonization standard of the ASIC container
	SetContainerTimeServer (see page 137)	Set time server URI for digital signature of the ASIC container
	SetMWCompatibility (see page 138)	Set the compatibility mode with the old version of the oficial EID MiddleWare
	SetSupportSIS (see page 139)	Activates or deactivates SIS card support by engine
	SetTimeServer (see page 139)	Set time server URI for digital signature of the ASIC container
	SignPdfFile (see page 141)	Digitally sign PDF file using Belgian EID card
	SignPdfFileEx (see page 141)	Digitally sign PDF file using Belgian EID card
	StartEngine (see page 142)	Activates the Swelio Engine.
	StopEngine (see page 142)	Deactivates the Swelio Engine
	VerifyContainer (see page 145)	Verify the signatures and the integrity of the ASIC container
	VerifyPinA (see page 145)	Verify PIN code
	VerifyPinExA (see page 145)	Verify PIN code
	VerifyPinExW (see page 146)	Verify PIN code
	VerifyPinW (see page 146)	Verify PIN code
	VerifySignature (see page 146)	Verifies the signature from the specified hash value.

1.3.10 System.h

Windows related routines

Functions

	Name	Description
	AddRemoveMessageFilter (see page 12)	Adds or removes a message from the User Interface Privilege Isolation (UIPI) message filter.
	AllocateDefaultHWNDA (see page 13)	This function creates the invisible tool window
	AllocateDefaultHWNDW (see page 13)	This function creates the invisible tool window
	AllocateHWNDA (see page 13)	This function creates the invisible tool window using the provided window procedure
	AllocateHWNDW (see page 14)	This function creates the invisible tool window using the provided window procedure
	AllocateLayeredWindowA (see page 14)	This function creates the layered window using the provided window class name
	AllocateLayeredWindowW (see page 14)	This function creates the layered window using the provided window class name

	AllocateWindowClassA (see page 15)	This function creates the standard window using the provided window class name
	AllocateWindowClassW (see page 15)	This function creates the standard window using the provided window class name
	BringWindowToFront (see page 16)	This function brings the specified window to the top of the z-order.
	ClearUnusedMemory (see page 25)	Clears unused memory and minimized the application memory usage
	DeallocateHWNDA (see page 31)	This function destroys the specified window.
	DeallocateHWNDW (see page 31)	This function destroys the specified window.
	DrawLayeredWindow (see page 37)	Repaints the surface of the layered window
	EmptyRecycleBin (see page 41)	Empties the recycle bin
	GetStartupA (see page 79)	Checks if the application is registered to run when Windows starts
	GetStartupW (see page 80)	Checks if the application is registered to run when Windows starts
	HibernateWindows (see page 82)	Hibernates Windows
	LayeredWndProcA (see page 98)	The default window procedure for the layered window
	LayeredWndProcW (see page 98)	The default window procedure for the layered window
	MakeSoundFromFileA (see page 103)	Plays the wave sound from the file
	MakeSoundFromFileW (see page 103)	Plays the wave sound from the file
	MakeSoundFromResourceA (see page 104)	Plays the wave sound from the resource
	MakeSoundFromResourceW (see page 104)	Plays the wave sound from the resource
	RemoveStartupA (see page 117)	Removes the application from the list of the automatically started applications
	RemoveStartupW (see page 118)	Removes the application from the list of the automatically started applications
	RestoreWindowSubclassA (see page 118)	Restores window standard procedure
	RestoreWindowSubclassW (see page 118)	Restores window standard procedure
	SetStartupA (see page 138)	Register application to run when Windows starts
	SetStartupW (see page 138)	Register application to run when Windows starts
	ShutdownWindows (see page 140)	Logs off the interactive user, shuts down the system.
	SuspendWindows (see page 144)	Suspends Windows
	TurnMonitorOff (see page 144)	Turns the monitor off
	TurnMonitorOn (see page 144)	Turns the monitor on
	UpdateWindowPosition (see page 144)	Updated the window position
	fpreset (see page 148)	Resets the floating-point unit (FPU) to its default state.

1.3.11 SystemInfo.h

Routines for querying information about operating system

Functions

	Name	Description
	CurrentIPAddressA (see page 29)	Returns the IP address
	CurrentIPAddressW (see page 30)	Returns the IP address
	IsCitrixSession (see page 86)	Checks if application is running in Citrix session
	IsConnectedToInternet (see page 87)	Checks if PC is connected to Internet
	IsMediaCenter (see page 90)	Checks if the Media Center version of Windows is installed
	IsMetroActive (see page 91)	Checks if metro interface is active
	IsMultiTouchReady (see page 91)	Checks if the system is multi touch ready
	IsNativeWin64 (see page 91)	Checks if the application is native 64 bit executable
	IsRemoteSession (see page 92)	Checks if application is running in RDP session
	IsTabletPC (see page 93)	Checks if the application is running on the Tablet PC
	IsWindows10 (see page 95)	Checks if PC is running Windows 10 or better
	IsWindows11 (see page 96)	Checks if PC is running Windows 11 or better
	IsWindows7 (see page 96)	Checks if PC is running Windows 7 or better
	IsWindows8 (see page 96)	Checks if PC is Running Windows 8 or better
	IsWindowsVista (see page 97)	Checks if PC is running Windows Vista or better
	IsWindowsXP (see page 97)	Checks if PC is running Windows XP
	IsWindowsXPSP2 (see page 97)	Checks if PC is running Windows XP with Service Pack 2 installed
	IsWow64 (see page 98)	Checks if the 32 bit application runs on 64 bit Windows
	PortAvailable (see page 105)	Checks if the port with specified number is available
	RecycleBinEmpty (see page 117)	Checks if Windows Recycle Bin is empty

1.3.12 quricol.h

QR Code generator

Functions

	Name	Description
	DestroyImageBuffer (see page 34)	Destroys the memory buffer
	GenerateBMPA (see page 57)	Generates Windows Bitmap file with QR Code image
	GenerateBMPW (see page 57)	Generates Windows Bitmap file with QR Code image
	GeneratePNGA (see page 60)	Generates PNG file with QR Code image
	GeneratePNGW (see page 60)	Generates PNG file with QR Code image
	GetHBitmapA (see page 71)	Generates Windows Bitmap in memory with QR Code image
	GetHBitmapW (see page 72)	Generates Windows Bitmap in memory with QR Code image
	GetPNGA (see page 74)	Writes PNG image to the memory buffer.
	GetPNGW (see page 74)	Writes PNG image to the memory buffer.

Index

A

ActivateCard 10
ActivateCard function 10
ActivateCardEx 11
ActivateCardEx function 11
AddFileToContainer 11
AddFileToContainer function 11
AddRemoveMessageFilter 12
AddRemoveMessageFilter function 12
Algorithms.h 178
AllocateBuffer 12
AllocateBuffer function 12
AllocateDefaultHWNDA 13
AllocateDefaultHWNDA function 13
AllocateDefaultHWNDW 13
AllocateDefaultHWNDW function 13
AllocateHWNDA 13
AllocateHWNDA function 13
AllocateHWNDW 14
AllocateHWNDW function 14
AllocateLayeredWindowA 14
AllocateLayeredWindowA function 14
AllocateLayeredWindowW 14
AllocateLayeredWindowW function 14
AllocateWindowClassA 15
AllocateWindowClassA function 15
AllocateWindowClassW 15
AllocateWindowClassW function 15
AlphaBlendBitmap 15
AlphaBlendBitmap function 15
AlphaBlendNative 16
AlphaBlendNative function 16

B

BringWindowToFront 16
BringWindowToFront function 16

C

CardDecryptFileA 17
CardDecryptFileA function 17
CardDecryptFileW 17
CardDecryptFileW function 17
CardEncryptFileA 17
CardEncryptFileA function 17
CardEncryptFileW 18
CardEncryptFileW function 18
CardEventType 159
CardEventType enumeration 159
CardEvents.h 178
CardSignCMS 18
CardSignCMS function 18
CardSignCMSEx 19
CardSignCMSEx function 19
CardSignCadesT 19
CardSignCadesT function 19
CardSignCadesTEx 20
CardSignCadesTEx function 20
CardStructures.h 178
CertSignCMS 20
CertSignCMS function 20
CertSignCMSData 21
CertSignCMSData function 21
CertSignCMSEx 21
CertSignCMSEx function 21
CertSignCadesT 22
CertSignCadesT function 22
CertSignCadesTData 22
CertSignCadesTData function 22
CertSignCadesTEx 23
CertSignCadesTEx function 23
CheckMD5 23
CheckMD5 function 23
CheckSHA1 24
CheckSHA1 function 24
CheckSHA256 24
CheckSHA256 function 24
ClearFileAttributesA 25
ClearFileAttributesA function 25
ClearFileAttributesW 25
ClearFileAttributesW function 25
ClearUnusedMemory 25

ClearUnusedMemory function 25
CloneFont 26
CloneFont function 26
ContainerCertificate 26
ContainerCertificate function 26
ContainerEidCertificate 26
ContainerEidCertificate function 26
ContainerPickCertificate 27
ContainerPickCertificate function 27
CopyNativeBitmap 27
CopyNativeBitmap function 27
CreateCardBuffer 28
CreateCardBuffer function 28
CreateNativeBitmap 28
CreateNativeBitmap function 28
CreateUnicodeFileA 28
CreateUnicodeFileA function 28
CreateUnicodeFileW 29
CreateUnicodeFileW function 29
CreateWindowsFont 29
CreateWindowsFont function 29
CurrentIPAddressA 29
CurrentIPAddressA function 29
CurrentIPAddressW 30
CurrentIPAddressW function 30

D

DeactivateCard 30
DeactivateCard function 30
DeactivateCardEx 30
DeactivateCardEx function 30
DeallocateBuffer 31
DeallocateBuffer function 31
DeallocateHWNDA 31
DeallocateHWNDA function 31
DeallocateHWNDW 31
DeallocateHWNDW function 31
DecryptFileAESA 32
DecryptFileAESA function 32
DecryptFileAESW 32
DecryptFileAESW function 32
DeleteCardBuffer 33
DeleteCardBuffer function 33
DeleteToRecycleBinA 33
DeleteToRecycleBinA function 33
DeleteToRecycleBinW 33
DeleteToRecycleBinW function 33
DestroyFont 34
DestroyFont function 34
DestroyImageBuffer 34
DestroyImageBuffer function 34
DirectoryExistsA 34
DirectoryExistsA function 34
DirectoryExistsW 35
DirectoryExistsW function 35
DisplayCertificate 35
DisplayCertificate function 35
DpiY 36
DpiY function 36
DrawAlphaText 36
DrawAlphaText function 36
DrawAlphaTextRect 36
DrawAlphaTextRect function 36
DrawLayeredWindow 37
DrawLayeredWindow function 37
DrawNativeBitmap 37
DrawNativeBitmap function 37
DrawTextDirect 38
DrawTextDirect function 38
DrawTextDirectEx 38
DrawTextDirectEx function 38
DrawTextGlow 38
DrawTextGlow function 38
DrawTextLine 39
DrawTextLine function 39
DrawTextOutline 39
DrawTextOutline function 39
DrawTextRect 40
DrawTextRect function 40

E

EidAddressA 159
EidAddressA structure 159
EidAddressW 160

EidAddressW structure 160
EidCertificate 160
EidCertificate structure 160
EidIdentityA 160
EidIdentityA structure 160
EidIdentityExA 162
EidIdentityExA structure 162
EidIdentityExW 163
EidIdentityExW structure 163
EidIdentityW 165
EidIdentityW structure 165
EidPicture 166
EidPicture structure 166
EmToPixels 40
EmToPixels function 40
EmptyRecycleBin 41
EmptyRecycleBin function 41
EncodeCertificate 41
EncodeCertificate function 41
EncodePhoto 41
EncodePhoto function 41
EncryptFileAESA 42
EncryptFileAESA function 42
EncryptFileAESW 42
EncryptFileAESW function 42
Encryption.h 179
ErrorInformation 167
ErrorInformation structure 167
FileCloseA 43
FileCloseA function 43
FileCloseW 43
FileCloseW function 43
FileCopyA 43
FileCopyA function 43
FileCopyW 44
FileCopyW function 44
FileCreateRewriteA 44
FileCreateRewriteA function 44
FileCreateRewriteW 44
FileCreateRewriteW function 44
FileDeleteA 45
FileDeleteA function 45
FileDeleteW 45
FileDeleteW function 45
FileExistsA 46
FileExistsA function 46
FileExistsW 46
FileExistsW function 46
FileExtensionIsA 46
FileExtensionIsA function 46
FileExtensionIsW 47
FileExtensionIsW function 47
FileGetSizeA 47
FileGetSizeA function 47
FileGetSizeW 47
FileGetSizeW function 47
FileIsExeA 48
FileIsExeA function 48
FileIsExeW 48
FileIsExeW function 48
FileIsIconA 49
FileIsIconA function 49
FileIsIconW 49
FileIsIconW function 49
FileIsImageA 49
FileIsImageA function 49
FileIsImageW 50
FileIsImageW function 50
FileIsLink 50
FileIsLink function 50
FileOperations.h 180
FileOrFolderExistsA 50
FileOrFolderExistsA function 50
FileOrFolderExistsW 51
FileOrFolderExistsW function 51
FileRenameA 51
FileRenameA function 51
FileRenameW 51
FileRenameW function 51
FileWriteA 52
FileWriteA function 52
FileWriteCharA 52
FileWriteCharA function 52
FileWriteCharW 52

FileWriteCharW function 52
FileWriteNewLineA 53
FileWriteNewLineA function 53
FileWriteNewLineW 53
FileWriteNewLineW function 53
FileWriteW 53
FileWriteW function 53
Files 178
FreeContainer 54
FreeContainer function 54
FullPathA 54
FullPathA function 54
FullPathW 54
FullPathW function 54
Functions 1
GenerateAuthenticationSignatureA 55
GenerateAuthenticationSignatureA function 55
GenerateAuthenticationSignatureExA 55
GenerateAuthenticationSignatureExA function 55
GenerateAuthenticationSignatureExW 56
GenerateAuthenticationSignatureExW function 56
GenerateAuthenticationSignatureW 56
GenerateAuthenticationSignatureW function 56
GenerateBMPA 57
GenerateBMPA function 57
GenerateBMPW 57
GenerateBMPW function 57
GenerateNonRepudiationSignatureA 58
GenerateNonRepudiationSignatureA function 58
GenerateNonRepudiationSignatureExA 58
GenerateNonRepudiationSignatureExA function 58
GenerateNonRepudiationSignatureExW 59
GenerateNonRepudiationSignatureExW function 59
GenerateNonRepudiationSignatureW 59
GenerateNonRepudiationSignatureW function 59
GeneratePNGA 60
GeneratePNGA function 60
GeneratePNGW 60
GeneratePNGW function 60
GenerateQRCodeA 60
GenerateQRCodeA function 60
GenerateQRCodeExA 61
GenerateQRCodeExA function 61
GenerateQRCodeExW 61
GenerateQRCodeExW function 61
GenerateQRCodeW 62
GenerateQRCodeW function 62
GetAllFiles 62
GetAllFiles function 62
GetCardBufferA 62
GetCardBufferA function 62
GetCardBufferSize 63
GetCardBufferSize function 63
GetCardBufferW 63
GetCardBufferW function 63
GetCardSerialNumber 64
GetCardSerialNumber function 64
GetCardVersion 64
GetCardVersion function 64
GetContainerError 65
GetContainerError function 65
GetContainerErrorsCount 65
GetContainerErrorsCount function 65
GetEncodedCertificateSize 65
GetEncodedCertificateSize function 65
GetEncodedPhotoSize 66
GetEncodedPhotoSize function 66
GetFileMD5A 66
GetFileMD5A function 66
GetFileMD5W 67
GetFileMD5W function 67
GetFileSHA1A 67
GetFileSHA1A function 67
GetFileSHA1W 68
GetFileSHA1W function 68
GetFileSHA256A 68
GetFileSHA256A function 68
GetFileSHA256W 68
GetFileSHA256W function 68
GetFileSHA384A 69
GetFileSHA384A function 69
GetFileSHA384W 69
GetFileSHA384W function 69
GetFileSHA512A 70

GetFileSHA512A function 70
GetFileSHA512W 70
GetFileSHA512W function 70
GetFilesCountA 71
GetFilesCountA function 71
GetFilesCountW 71
GetFilesCountW function 71
GetHBitmapA 71
GetHBitmapA function 71
GetHBitmapW 72
GetHBitmapW function 72
GetISOCodeA 72
GetISOCodeA function 72
GetISOCodeW 73
GetISOCodeW function 73
GetMD5 73
GetMD5 function 73
GetPNGA 74
GetPNGA function 74
GetPNGW 74
GetPNGW function 74
GetReaderIndexA 75
GetReaderIndexA function 75
GetReaderIndexW 75
GetReaderIndexW function 75
GetReaderNameA 75
GetReaderNameA function 75
GetReaderNameLenA 76
GetReaderNameLenA function 76
GetReaderNameLenW 76
GetReaderNameLenW function 76
GetReaderNameW 76
GetReaderNameW function 76
GetReadersCount 77
GetReadersCount function 77
GetSHA1 77
GetSHA1 function 77
GetSHA256 78
GetSHA256 function 78
GetSHA384 78
GetSHA384 function 78
GetSHA512 79
GetSHA512 function 79
GetSelectedReaderIndex 79
GetSelectedReaderIndex function 79
GetStartupA 79
GetStartupA function 79
GetStartupW 80
GetStartupW function 80
GetSupportSIS 80
GetSupportSIS function 80
GetTextLineSize 80
GetTextLineSize function 80
GetTextSize 81
GetTextSize function 81
GetTextSizeEx 81
GetTextSizeEx function 81
GetTimeServer 81
GetTimeServer function 81
Graphics.h 182
HibernateWindows 82
HibernateWindows function 82
IgnoreHardwareEvents 82
IgnoreHardwareEvents function 82
IgnoreServiceEvents 82
IgnoreServiceEvents function 82
IgnoreUnknownCards 83
IgnoreUnknownCards function 83
InitializeContainer 83
InitializeContainer function 83
IsAnimatedGIFA 84
IsAnimatedGIFA function 84
IsAnimatedGIFW 84
IsAnimatedGIFW function 84
IsCardActivated 84
IsCardActivated function 84
IsCardActivatedEx 85
IsCardActivatedEx function 85
IsCardPresent 85
IsCardPresent function 85
IsCardPresentEx 85
IsCardPresentEx function 85
IsCardStillInserted 86
IsCardStillInserted function 86

IsCardStillInsertedEx 86
IsCardStillInsertedEx function 86
IsCitrixSession 86
IsCitrixSession function 86
IsConnectedToInternet 87
IsConnectedToInternet function 87
IsDirectoryA 87
IsDirectoryA function 87
IsDirectoryW 87
IsDirectoryW function 87
IsEIDCard 88
IsEIDCard function 88
IsEIDCardEx 88
IsEIDCardEx function 88
IsEngineActive 88
IsEngineActive function 88
IsFemaleA 89
IsFemaleA function 89
IsFemaleW 89
IsFemaleW function 89
IsMaleA 89
IsMaleA function 89
IsMaleW 90
IsMaleW function 90
IsMediaCenter 90
IsMediaCenter function 90
IsMetroActive 91
IsMetroActive function 91
IsMultiTouchReady 91
IsMultiTouchReady function 91
IsNativeWin64 91
IsNativeWin64 function 91
IsRemoteSession 92
IsRemoteSession function 92
IsSISCard 92
IsSISCard function 92
IsSISCardEx 92
IsSISCardEx function 92
IsTabletPC 93
IsTabletPC function 93
IsUnicodeFileA 93
IsUnicodeFileA function 93
IsUnicodeFileW 93
IsUnicodeFileW function 93
IsValidFileNameA 94
IsValidFileNameA function 94
IsValidFileNameW 94
IsValidFileNameW function 94
IsValidPathNameA 95
IsValidPathNameA function 95
IsValidPathNameW 95
IsValidPathNameW function 95
IsWindows10 95
IsWindows10 function 95
IsWindows11 96
IsWindows11 function 96
IsWindows7 96
IsWindows7 function 96
IsWindows8 96
IsWindows8 function 96
IsWindowsVista 97
IsWindowsVista function 97
IsWindowsXP 97
IsWindowsXP function 97
IsWindowsXPSP2 97
IsWindowsXPSP2 function 97
IsWow64 98
IsWow64 function 98
LayeredWndProcA 98
LayeredWndProcA function 98
LayeredWndProcW 98
LayeredWndProcW function 98
LoadAddressA 98
LoadAddressA function 98
LoadAddressW 99
LoadAddressW function 99
LoadBitmapJPG 99
LoadBitmapJPG function 99
LoadBitmapPNG 100
LoadBitmapPNG function 100
LoadCertificateA 100
LoadCertificateA function 100
LoadCertificateW 100
LoadCertificateW function 100

LoadIdentityA 101
LoadIdentityA function 101
LoadIdentityW 101
LoadIdentityW function 101
LoadPNGResource 101
LoadPNGResource function 101
LoadPhotoA 102
LoadPhotoA function 102
LoadPhotoW 102
LoadPhotoW function 102
MakeCompatibleBitmap 103
MakeCompatibleBitmap function 103
MakeSoundFromFileA 103
MakeSoundFromFileA function 103
MakeSoundFromFileW 103
MakeSoundFromFileW function 103
MakeSoundFromResourceA 104
MakeSoundFromResourceA function 104
MakeSoundFromResourceW 104
MakeSoundFromResourceW function 104
NationalityConverter.h 183
Network.h 183
NonRepudiationSignatureAlgoA 104
NonRepudiationSignatureAlgoA function 104
NonRepudiationSignatureAlgoW 105
NonRepudiationSignatureAlgoW function 105
PEidAddressA 167
PEidAddressA structure 167
PEidAddressW 167
PEidAddressW structure 167
PEidCertificate 168
PEidCertificate structure 168
PEidIdentityA 168
PEidIdentityA structure 168
PEidIdentityExA 170
PEidIdentityExA structure 170
PEidIdentityExW 171
PEidIdentityExW structure 171
PEidIdentityW 173
PEidIdentityW structure 173
PErrorInformation 174
PErrorInformation structure 174
PSISRecordA 174
PSISRecordA structure 174
PSISRecordW 175
PSISRecordW structure 175
PeidPicture 176
PeidPicture structure 176
PointsToPixels 105
PointsToPixels function 105
PortAvailable 105
PortAvailable function 105
ReadAddressA 106
ReadAddressA function 106
ReadAddressExA 106
ReadAddressExA function 106
ReadAddressExW 106
ReadAddressExW function 106
ReadAddressW 107
ReadAddressW function 107
ReadAuthenticationCertificate 107
ReadAuthenticationCertificate function 107
ReadAuthenticationCertificateEx 107
ReadAuthenticationCertificateEx function 107
ReadBufferFromFileA 108
ReadBufferFromFileA function 108
ReadBufferFromFileW 108
ReadBufferFromFileW function 108
ReadCaCertificate 109
ReadCaCertificate function 109
ReadCaCertificateEx 109
ReadCaCertificateEx function 109
ReadIdentityA 109
ReadIdentityA function 109
ReadIdentityExA 110
ReadIdentityExA function 110
ReadIdentityExW 110
ReadIdentityExW function 110
ReadIdentityW 110
ReadIdentityW function 110
ReadNonRepudiationCertificate 111
ReadNonRepudiationCertificate function 111
ReadNonRepudiationCertificateEx 111
ReadNonRepudiationCertificateEx function 111

ReadPhoto 112
ReadPhoto function 112
ReadPhotoAsBitmap 112
ReadPhotoAsBitmap function 112
ReadPhotoAsBitmapEx 112
ReadPhotoAsBitmapEx function 112
ReadPhotoEx 113
ReadPhotoEx function 113
ReadRootCaCertificate 113
ReadRootCaCertificate function 113
ReadRootCaCertificateEx 114
ReadRootCaCertificateEx function 114
ReadRrnCertificate 114
ReadRrnCertificate function 114
ReadRrnCertificateEx 114
ReadRrnCertificateEx function 114
ReadSISCardA 115
ReadSISCardA function 115
ReadSISCardExA 115
ReadSISCardExA function 115
ReadSISCardExW 116
ReadSISCardExW function 116
ReadSISCardW 116
ReadSISCardW function 116
RecycleBinEmpty 117
RecycleBinEmpty function 117
ReloadReadersList 117
ReloadReadersList function 117
RemoveCallback 117
RemoveCallback function 117
RemoveStartupA 117
RemoveStartupA function 117
RemoveStartupW 118
RemoveStartupW function 118
RestoreWindowSubclassA 118
RestoreWindowSubclassA function 118
RestoreWindowSubclassW 118
RestoreWindowSubclassW function 118
SISRecordA 176
SISRecordA structure 176
SISRecordW 177
SISRecordW structure 177
SaveAddressA 119
SaveAddressA function 119
SaveAddressW 119
SaveAddressW function 119
SaveAuthenticationCertificateA 119
SaveAuthenticationCertificateA function 119
SaveAuthenticationCertificateExW 120
SaveAuthenticationCertificateExW function 120
SaveAuthenticationCertificateW 120
SaveAuthenticationCertificateW function 120
SaveCaCertificateA 120
SaveCaCertificateA function 120
SaveCaCertificateExW 121
SaveCaCertificateExW function 121
SaveCaCertificateW 121
SaveCaCertificateW function 121
SaveCardToToXMLStreamExA 122
SaveCardToToXMLStreamExA function 122
SaveCardToToXMLStreamExW 122
SaveCardToToXMLStreamExW function 122
SaveCardToXmlA 122
SaveCardToXmlA function 122
SaveCardToXmlExA 123
SaveCardToXmlExA function 123
SaveCardToXmlExW 123
SaveCardToXmlExW function 123
SaveCardToXmlW 124
SaveCardToXmlW function 124
SaveContainer 124
SaveContainer function 124
SaveIdentityA 124
SaveIdentityA function 124
SaveIdentityW 125
SaveIdentityW function 125
SaveNonRepudiationCertificateA 125
SaveNonRepudiationCertificateA function 125
SaveNonRepudiationCertificateExW 126
SaveNonRepudiationCertificateExW function 126
SaveNonRepudiationCertificateW 126
SaveNonRepudiationCertificateW function 126
SavePersonCsvToStreamA 126
SavePersonCsvToStreamA function 126

SavePersonCsvToStreamW 127
SavePersonCsvToStreamW function 127
SavePersonToCsvA 127
SavePersonToCsvA function 127
SavePersonToCsvExA 128
SavePersonToCsvExA function 128
SavePersonToCsvExW 128
SavePersonToCsvExW function 128
SavePersonToCsvW 128
SavePersonToCsvW function 128
SavePhotoA 129
SavePhotoA function 129
SavePhotoAsBitmapA 129
SavePhotoAsBitmapA function 129
SavePhotoAsBitmapExA 130
SavePhotoAsBitmapExA function 130
SavePhotoAsBitmapExW 130
SavePhotoAsBitmapExW function 130
SavePhotoAsBitmapW 130
SavePhotoAsBitmapW function 130
SavePhotoAsJpegA 131
SavePhotoAsJpegA function 131
SavePhotoAsJpegExA 131
SavePhotoAsJpegExA function 131
SavePhotoAsJpegExW 131
SavePhotoAsJpegExW function 131
SavePhotoAsJpegW 132
SavePhotoAsJpegW function 132
SavePhotoW 132
SavePhotoW function 132
SaveRootCaCertificateA 133
SaveRootCaCertificateA function 133
SaveRootCaCertificateExW 133
SaveRootCaCertificateExW function 133
SaveRootCaCertificateW 133
SaveRootCaCertificateW function 133
SaveRrnCertificateA 134
SaveRrnCertificateA function 134
SaveRrnCertificateExW 134
SaveRrnCertificateExW function 134
SaveRrnCertificateW 134
SaveRrnCertificateW function 134
SelectReader 135
SelectReader function 135
SelectReaderByNameA 135
SelectReaderByNameA function 135
SelectReaderByNameW 136
SelectReaderByNameW function 136
SendAPDU 136
SendAPDU function 136
ServerAccessible 136
ServerAccessible function 136
SetCallback 137
SetCallback function 137
SetContainerCanonization 137
SetContainerCanonization function 137
SetContainerTimeServer 137
SetContainerTimeServer function 137
SetMWCompatibility 138
SetMWCompatibility function 138
SetStartupA 138
SetStartupA function 138
SetStartupW 138
SetStartupW function 138
SetSupportSIS 139
SetSupportSIS function 139
SetTimeServer 139
SetTimeServer function 139
ShellCopyFileA 140
ShellCopyFileA function 140
ShellCopyFileW 140
ShellCopyFileW function 140
ShutdownWindows 140
ShutdownWindows function 140
SignPdfFile 141
SignPdfFile function 141
SignPdfFileEx 141
SignPdfFileEx function 141
StartEngine 142
StartEngine function 142
StopEngine 142
StopEngine function 142
StretchNativeBitmap 142
StretchNativeBitmap function 142

StripFileNameA 143
StripFileNameA function 143
StripFileNameW 143
StripFileNameW function 143
Structs, Records, Enums 148
SuspendWindows 144
SuspendWindows function 144
Swelio.h 183
System.h 188
SystemInfo.h 189
TurnMonitorOff 144
TurnMonitorOff function 144
TurnMonitorOn 144
TurnMonitorOn function 144
UpdateWindowPosition 144
UpdateWindowPosition function 144
VerifyContainer 145
VerifyContainer function 145
VerifyPinA 145
VerifyPinA function 145
VerifyPinExA 145
VerifyPinExA function 145
VerifyPinExW 146
VerifyPinExW function 146
VerifyPinW 146
VerifyPinW function 146
VerifySignature 146
VerifySignature function 146
WriteBufferToFileA 147
WriteBufferToFileA function 147
WriteBufferToFileW 147
WriteBufferToFileW function 147
ewtCardInsert enumeration member 150
ewtCardRemove enumeration member 150
ewtReadersChange enumeration member 150
ewtUnknownEvent enumeration member 150

F

fpreset 148
fpreset function 148

Q

quricol.h 190

S

structErrorInformation 149
structErrorInformation structure 149

T

tagCardEventType 150
tagCardEventType enumeration 150
tagEidAddressA 150
tagEidAddressA structure 150
tagEidAddressW 150
tagEidAddressW structure 150
tagEidCertificate 151
tagEidCertificate structure 151
tagEidIdentityA 151
tagEidIdentityA structure 151
tagEidIdentityExA 153
tagEidIdentityExA structure 153
tagEidIdentityExW 154
tagEidIdentityExW structure 154
tagEidIdentityW 156
tagEidIdentityW structure 156
tagEidPicture 157
tagEidPicture structure 157
tagSISRecordA 157
tagSISRecordA structure 157
tagSISRecordW 158
tagSISRecordW structure 158